

Laporan Tugas Besar

IF2211 Strategi Algoritma

Implementasi Strategi Greedy pada Bot Battlecode 2025

Semester II Tahun 2025/2026



Kelompok DendengBalado

Gabriella Botimada Lubis 13524006

Agatha Tatianingseto 13524008

Junior Natra Situmorang 13524055

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1	Deskripsi Tugas	3
1.1	Latar Belakang	3
1.2	Komponen Permainan	3
1.2.1	Objectives	3
1.2.2	Aturan Tiebreaker	3
1.2.3	Peta	3
1.2.4	Jarak Pandang dan Pergerakan	4
1.2.5	Tower dan Ruins	4
1.2.6	Pola Sumber Daya Khusus	4
1.2.7	Cat	4
1.2.8	Chips	4
1.2.9	Robot	4
1.2.10	Jenis Robot	4
1.2.11	Towers	5
1.2.12	Serangan	5
1.2.13	Penambahan	5
1.2.14	Komunikasi	5
1.2.15	Penanda	5
1.2.16	Action dan Cooldown	5
2	Landasan Teori	7
2.1	Algoritma Greedy	7
2.2	Cara Kerja Program	9
3	Aplikasi Strategi Greedy	11
3.1	Mapping Persoalan	11
3.2	Mapping Permasalahan Battlecode ke Komponen Algoritma Greedy	11
3.2.1	Himpunan Kandidat	11
3.2.2	Himpunan Solusi	13
3.2.3	Fungsi Seleksi	13
3.2.4	Fungsi Kelayakan	14
3.2.5	Fungsi Solusi	14
3.2.6	Fungsi Objektif	15
3.3	Alternatif Solusi Greedy 1	15
3.3.1	Heuristik yang Digunakan	15
3.3.2	Fokus Utama Setiap Unit	15
3.3.3	Logika Greedy Sederhana	16
3.3.4	Strategi Jalan	16
3.3.5	Strategi Target Cat	17
3.3.6	Manajemen Sumber Daya	19
3.3.7	Spawn Strategy	19
3.3.8	Pemanfaatan Pola Khusus (SRP dan Tower)	19

3.4	Alternatif Solusi Greedy 2	20
3.5	Alternatif Solusi Greedy 3	21
3.5.1	State Robot	22
3.5.2	Alur Setiap Robot	23
3.5.3	Alur Robot Setiap Phase	24
3.5.4	Threshold	24
3.6	Analisis Efisiensi	25
3.6.1	Alternatif Solusi Greedy 1	25
3.6.2	Alternatif Solusi Greedy 2	26
3.6.3	Alternatif Solusi Greedy 3	27
3.7	Strategi yang Dipilih	27
4	Implementasi dan Pengujian	29
4.1	Pseudocode	29
4.1.1	Algoritma Solusi Greedy 1	29
4.1.2	Algoritma Solusi Greedy 2	89
4.1.3	Algoritma Solusi Greedy 3	109
4.2	Struktur Data dan Method	159
4.2.1	G.java	159
4.2.2	RobotPlayer.java	160
4.2.3	Soldier.java	160
4.2.4	Splasher.java	163
4.2.5	Mopper.java	164
4.2.6	Tower.java	166
4.2.7	Nav.java	167
4.2.8	MapMemory.java	168
4.2.9	Rand.java	170
4.3	Pengujian	170
4.4	Analisis Hasil	172
4.4.1	Main Bot	172
4.4.2	Alternative 1	175
4.4.3	Alternative 2	179
4.4.4	Hasil Pengujian	183
5	Kesimpulan dan Saran	185
5.1	Kesimpulan	185
5.2	Saran	185

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

1.2 Komponen Permainan

Komponen-komponen dari permainan ini antara lain:

1.2.1 Objectives

Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

- Luas area yang sudah diwarnai di peta
- Jumlah menara sekutu yang masih hidup
- Total jumlah uang (chips)
- Jumlah total cat (paint) yang dimiliki semua robot dan menara
- Jumlah robot sekutu yang masih hidup
- Jika masih seri, tim dipilih secara acak (uniform random)

1.2.3 Peta

Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20×20 hingga 60×60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0,0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek MapLocation yang menyimpan nilai x dan y. Untuk menjaga keseimbangan permainan, peta dijamin simetris melalui rotasi atau refleksi.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat fitur peta dan robot lain dalam jarak hingga $\sqrt{20}$ atau sekitar 4.47 petak. Unit tidak dapat bergerak melewati menara atau robot lain. Peta juga dapat memiliki tembok (walls) yang tidak dapat dilewati maupun diwarnai. Jumlah tembok maksimal adalah 20% dari seluruh peta. Selain tembok, terdapat reruntuhan (ruins) yang juga tidak dapat dilewati atau diwarnai oleh robot.

1.2.5 Tower dan Ruins

Menara berfungsi sebagai zona spawn tim sekaligus sebagai unit yang dapat menyerang musuh. Untuk membangun menara, tim harus mewarnai pola tertentu pada reruntuhan yang tersedia di peta. Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah tersedia di peta. Reruntuhan tidak dapat dipindahkan atau dihancurkan. Pusat reruntuhan berjarak minimal lima petak satu sama lain, dan area 5×5 di sekitarnya dijamin bebas dari tembok. Setiap tim memulai permainan dengan dua menara, yaitu satu menara penghasil cat dan satu menara penghasil uang.

1.2.6 Pola Sumber Daya Khusus

Pola sumber daya khusus atau Special Resource Pattern (SRP) merupakan pola yang dapat meningkatkan produksi sumber daya tim. Jika sebuah pola berhasil diselesaikan dan tetap utuh selama 50 ronde, pola tersebut akan menjadi aktif. Setiap pola aktif akan memberikan tambahan tiga sumber daya per giliran pada semua menara tambang sekutu.

1.2.7 Cat

Cat merupakan sumber daya utama dalam permainan. Cat digunakan untuk mengklaim wilayah pada peta dan menentukan skor tim. Setiap robot dan menara memiliki kapasitas penyimpanan cat yang berbeda-beda. Jika jumlah cat yang dimiliki robot terlalu sedikit, robot akan mengalami peningkatan cooldown atau bahkan tidak dapat melakukan aksi.

1.2.8 Chips

Chips digunakan sebagai sumber daya untuk membangun unit dan menara. Chips dihasilkan setiap ronde oleh menara penghasil uang. Setiap tim memulai permainan dengan 2500 chips yang dapat digunakan oleh semua robot sekutu.

1.2.9 Robot

Semua robot dapat melakukan berbagai aksi seperti bergerak, merasakan lingkungan sekitar, dan berkomunikasi dengan robot lain. Permainan berjalan secara bergiliran dalam ronde, dan setiap robot akan menjalankan kode yang ditulis oleh pemain. Setiap robot memiliki batas penggunaan bytecode pada setiap giliran. Jika bytecode habis sebelum robot selesai menjalankan programnya, maka giliran robot tersebut akan langsung berakhir.

1.2.10 Jenis Robot

Terdapat tiga jenis robot utama dalam permainan: Soldier, Splasher, dan Mopper. Masing-masing robot memiliki kemampuan, kapasitas cat, serta biaya produksi yang berbeda.

1.2.11 Towers

Semua menara dibangun dengan prosedur yang sama, yaitu dengan mewarnai pola tertentu pada area 5×5 di sekitar reruntuhan. Langkah pembangunan menara adalah:

1. Memanggil fungsi `markTowerPattern()`
2. Mewarnai pola yang ditentukan
3. Memanggil `completeTowerPattern()` untuk memunculkan menara

1.2.12 Serangan

Menara dapat melakukan dua jenis serangan:

- Serangan satu petak yang menargetkan satu unit musuh
- Serangan area efek (Area of Effect)

Dalam satu giliran, menara dapat melakukan satu serangan satu petak dan satu serangan area.

1.2.13 Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan menambang sumber daya secara pasif. Cat yang dihasilkan disimpan di dalam menara dan dapat diambil oleh robot sekutu, sedangkan chips dapat langsung digunakan oleh semua unit sekutu.

1.2.14 Komunikasi

Robot tidak dapat berbagi variabel secara langsung sehingga koordinasi antar robot harus dilakukan melalui komunikasi. Komunikasi dapat dilakukan melalui dua cara:

- Pesan (Messages)
- Penanda (Markers)

Pesan dapat berisi informasi apa pun tetapi dibatasi hingga 4 byte. Robot dapat mengirim maksimal satu pesan per ronde, sedangkan menara dapat mengirim hingga dua puluh pesan per ronde.

1.2.15 Penanda

Penanda merupakan informasi yang dapat ditempatkan pada peta oleh robot untuk memberikan petunjuk kepada robot sekutu lainnya mengenai pola atau area tertentu. Penanda hanya dapat dilihat oleh robot sekutu dan tidak memengaruhi kepemilikan wilayah pada peta.

1.2.16 Action dan Cooldown

Robot dapat melakukan berbagai aksi untuk berinteraksi dengan lingkungan permainan. Beberapa aksi memiliki cooldown sehingga tidak dapat dilakukan secara berulang dalam waktu singkat. Aksi utama yang dapat dilakukan robot meliputi:

- Menyerang atau mewarnai petak
- Bergerak ke petak lain

- Menandai petak
- Mentransfer cat
- Menarik cat dari menara
- Spawn robot dari menara
- Mengirim pesan
- Menghancurkan diri sendiri

2 Landasan Teori

2.1 Algoritma Greedy

Algoritma greedy adalah metode penyelesaian masalah yang bekerja secara bertahap (step by step). Pada setiap langkah, algoritma ini memilih keputusan yang dianggap paling optimal pada saat itu tanpa mempertimbangkan konsekuensi dari keputusan tersebut terhadap langkah-langkah berikutnya.

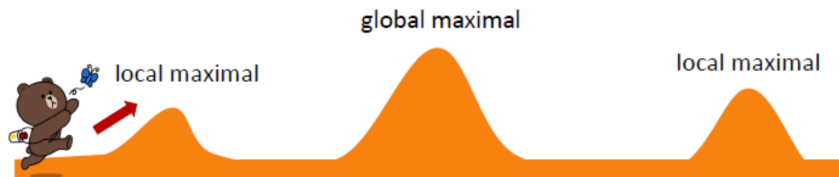


Figure 1: Greedy Algorithm Visualization
Source: [Greedy Algorithm](#)

Prinsip utama dari algoritma greedy adalah mengambil pilihan terbaik yang tersedia pada kondisi saat ini. Pendekatan ini sering disebut sebagai prinsip “rakus” (greedy), yaitu memilih keuntungan terbesar yang dapat diperoleh segera, dengan asumsi bahwa pilihan lokal terbaik tersebut pada akhirnya akan menghasilkan solusi global yang optimal.

Secara umum, algoritma greedy memiliki dua karakteristik utama, yaitu:

a. **Memilih solusi optimal lokal pada setiap langkah**

Pada setiap tahap penyelesaian, algoritma akan memilih pilihan terbaik yang tersedia pada saat itu tanpa mempertimbangkan kemungkinan dampaknya pada langkah selanjutnya.

b. **Mengharapkan tercapainya solusi optimal global**

Dengan memilih solusi terbaik pada setiap langkah, algoritma berharap bahwa rangkaian pilihan lokal tersebut akan menghasilkan solusi optimal secara keseluruhan.

Pendekatan greedy banyak digunakan dalam berbagai permasalahan optimasi karena memiliki implementasi yang sederhana serta efisiensi waktu yang baik. Namun, metode ini tidak selalu menghasilkan solusi optimal untuk semua jenis permasalahan, terutama pada kasus yang tidak memenuhi sifat greedy choice property dan optimal substructure. Algoritma greedy memiliki beberapa sifat utama yang mendukung penggunaannya dalam menyelesaikan berbagai permasalahan optimasi. Dua sifat penting dari algoritma greedy adalah sebagai berikut.

a. **Greedy Choice Property**

Greedy Choice Property merupakan sifat yang menyatakan bahwa solusi optimal dari suatu permasalahan dapat diperoleh dengan memilih keputusan terbaik secara lokal pada setiap langkah. Dengan kata lain, pada setiap tahap penyelesaian masalah, algoritma memilih opsi yang memberikan keuntungan paling besar pada saat itu tanpa mempertimbangkan kemungkinan keputusan di masa depan.

Jika suatu permasalahan memiliki sifat ini, maka pendekatan greedy dapat digunakan karena pilihan lokal terbaik pada setiap langkah akan tetap mengarah pada solusi optimal secara keseluruhan.

b. **Optimal Substructure**

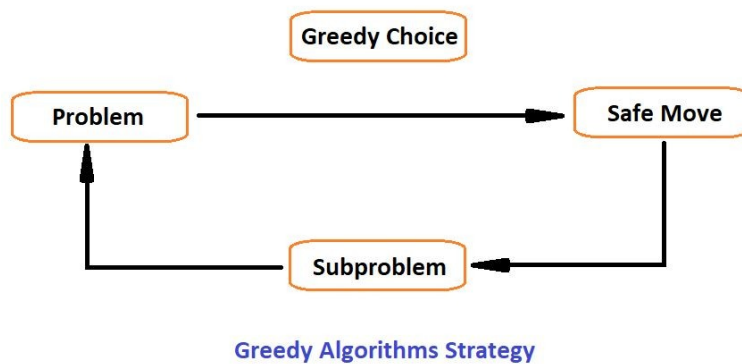


Figure 2: Greedy Algorithm Strategy

Source: Greedy Strategy

Optimal Substructure adalah sifat yang menyatakan bahwa solusi optimal dari suatu permasalahan dapat dibentuk dari solusi optimal dari sub-permasalahan yang lebih kecil. Artinya, jika suatu masalah dipecah menjadi beberapa bagian yang lebih kecil, maka penyelesaian terbaik untuk keseluruhan masalah dapat diperoleh dengan menggabungkan solusi terbaik dari masing-masing sub-permasalahan tersebut.

Sifat ini sangat penting dalam algoritma greedy karena memungkinkan proses penyelesaian dilakukan secara bertahap. Dengan menyelesaikan sub-permasalahan secara optimal pada setiap langkah, solusi akhir yang diperoleh juga akan menjadi solusi optimal untuk permasalahan secara keseluruhan.

Dalam merancang algoritma greedy untuk menyelesaikan suatu persoalan optimasi, terdapat beberapa langkah atau prosedur yang perlu dilakukan agar pendekatan greedy dapat diterapkan secara tepat. Prosedur tersebut meliputi hal-hal berikut.

- **Merumuskan persoalan optimasi dengan pilihan greedy**

Langkah pertama adalah merumuskan persoalan optimasi sedemikian rupa sehingga pada setiap tahap penyelesaian terdapat pilihan yang dapat diambil secara greedy. Persoalan dipecah menjadi serangkaian keputusan yang dilakukan secara bertahap, di mana setiap keputusan memilih opsi yang dianggap paling menguntungkan pada saat itu. Setelah suatu pilihan diambil, persoalan yang tersisa akan menjadi sub-persoalan yang lebih kecil untuk diselesaikan pada langkah berikutnya.

- **Menunjukkan adanya optimal substructure**

Langkah selanjutnya adalah menunjukkan bahwa persoalan tersebut memiliki sifat optimal substructure. Artinya, solusi optimal dari persoalan utama dapat diperoleh dengan menggabungkan solusi optimal dari sub-persoalan yang dihasilkan pada setiap langkah. Dengan adanya sifat ini, penyelesaian secara bertahap menggunakan pendekatan greedy tetap dapat menghasilkan solusi yang optimal.

- **Membuktikan adanya solusi greedy-optimal**

Langkah terakhir adalah membuktikan secara matematis bahwa selalu terdapat solusi optimal yang dapat diperoleh dengan mengikuti pilihan greedy pada setiap langkah. Pembuktian ini penting untuk

memastikan bahwa keputusan lokal yang diambil oleh algoritma greedy tidak akan menghalangi tercapainya solusi optimal secara keseluruhan.

2.2 Cara Kerja Program

Program yang dikembangkan merupakan sebuah bot yang dirancang untuk bermain secara otomatis pada permainan Battlecode. Bot ini bekerja dengan membaca kondisi lingkungan permainan pada setiap ronde dan kemudian menentukan aksi yang paling sesuai untuk dilakukan berdasarkan informasi yang diperoleh. Proses pengambilan keputusan tersebut dilakukan secara berulang selama robot masih aktif di dalam permainan.

Pada saat permainan dimulai, setiap robot akan menjalankan fungsi utama yang berisi sebuah perulangan (loop) yang berjalan terus menerus. Di dalam setiap iterasi perulangan tersebut, robot akan melakukan beberapa langkah utama, yaitu membaca kondisi lingkungan, menentukan strategi yang akan digunakan, memilih aksi terbaik, dan mengeksekusi aksi tersebut. Informasi yang digunakan oleh robot dapat berupa posisi robot saat ini, keberadaan unit musuh, lokasi menara musuh, serta kondisi tile di sekitar robot.

Bot dalam program ini terdiri dari beberapa jenis unit, yaitu Soldier, Splasher, dan Mopper, yang masing-masing memiliki peran yang berbeda. Unit Soldier bertugas melakukan eksplorasi peta, menyerang menara musuh, serta membangun menara baru pada lokasi tertentu. Unit Splasher berfokus pada penyebaran cat (paint) dan membantu menyerang wilayah musuh. Sementara itu, unit Mopper bertugas mendukung unit lain dengan membersihkan area di sekitar musuh dan menyerang target yang berada dalam jangkauan.

Implementasi algoritma greedy pada bot dilakukan dengan cara menentukan prioritas aksi yang harus dilakukan pada setiap ronde permainan. Pada setiap langkah, bot akan memilih aksi yang memberikan keuntungan paling besar pada saat itu berdasarkan kondisi lingkungan yang terdeteksi. Sebagai contoh, jika terdapat menara musuh yang berada dalam jangkauan serangan, maka bot akan langsung memprioritaskan aksi menyerang menara tersebut. Jika tidak ada target yang dapat diserang, bot akan memilih arah pergerakan yang paling mendekati lokasi strategis seperti posisi menara musuh atau wilayah yang dikuasai musuh.

Pendekatan greedy juga digunakan dalam pemilihan arah pergerakan robot. Bot akan mengevaluasi beberapa kemungkinan arah pergerakan yang tersedia, kemudian memilih arah yang paling mendekatkan robot pada tujuan utamanya. Dengan cara ini, bot dapat mengambil keputusan secara cepat tanpa perlu melakukan perhitungan strategi jangka panjang yang kompleks.

Untuk menjalankan bot, kode program perlu dikompilasi dan dijalankan menggunakan sistem permainan Battlecode. Setelah program dijalankan, bot akan secara otomatis mengikuti aturan permainan dan menjalankan strategi yang telah diimplementasikan. Selama permainan berlangsung, bot akan terus melakukan pengambilan keputusan pada setiap ronde hingga permainan berakhir.

3 Aplikasi Strategi Greedy

3.1 Mapping Persoalan

3.2 Mapping Permasalahan Battlecode ke Komponen Algoritma Greedy

Permainan Battlecode 2025 mengharuskan setiap robot dan menara untuk mengambil keputusan pada setiap ronde permainan. Keputusan tersebut meliputi berbagai aksi seperti bergerak, menyerang, mewarnai wilayah, membersihkan cat musuh, maupun melakukan spawn robot baru. Karena setiap unit memiliki keterbatasan bytecode dan waktu komputasi, maka pemilihan aksi harus dilakukan secara efisien. Pendekatan algoritma greedy digunakan untuk memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi permainan saat itu. Namun, setiap jenis unit pada Battlecode memiliki fungsi dan kemampuan yang berbeda. Oleh karena itu, proses pengambilan keputusan greedy tidak dapat disamaratakan untuk semua unit. Setiap jenis robot dan tower memiliki himpunan kandidat dan prioritas aksi yang berbeda. Untuk memformalkan permasalahan ini ke dalam kerangka algoritma greedy, diperlukan definisi komponen-komponen berikut: himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, fungsi solusi, dan fungsi objektif.

3.2.1 Himpunan Kandidat

Himpunan kandidat merupakan kumpulan seluruh aksi yang dapat dilakukan oleh suatu unit pada suatu ronde permainan. Karena setiap jenis robot memiliki kemampuan yang berbeda, maka himpunan kandidat untuk setiap robot juga berbeda.

Jenis Robot	Himpunan Kandidat (Aksi yang Dapat Dilakukan)
Soldier	• Bergerak ke salah satu dari delapan arah • Menyerang robot musuh dalam radius serangan • Mewarnai petak netral • Menghapus cat musuh • Bergerak menuju wilayah yang belum dicat • Membantu menyelesaikan pola pembangunan tower • Membantu menyelesaikan pola resource
Splasher	• Bergerak ke salah satu arah • Melakukan serangan area (splash attack) • Mewarnai beberapa petak sekaligus melalui serangan area • Bergerak menuju kelompok musuh • Bergerak menuju wilayah yang banyak dikuasai musuh
Mopper	• Bergerak menuju wilayah yang dicat musuh • Menghapus cat musuh • Melakukan serangan mop swing • Mentransfer cat ke robot sekutu • Mengambil cat dari menara • Bergerak menuju robot sekutu yang kekurangan cat

Table 1: Himpunan kandidat untuk setiap jenis robot

Selain robot, tower juga memiliki himpunan kandidat yang berbeda tergantung pada jenis tower yang dimiliki oleh tim.

Jenis Tower	Himpunan Kandidat
Money Tower	• Spawn robot baru • Menyerang musuh dalam jangkauan • Melakukan serangan area • Upgrade tower
Paint Tower	• Spawn robot • Menyerang robot musuh • Menyediakan cat bagi robot sekutu • Upgrade tower
Defense Tower	• Menyerang robot musuh • Melakukan serangan area • Melindungi wilayah sekitar • Upgrade tower

Table 2: Himpunan kandidat untuk setiap jenis tower

3.2.2 Himpunan Solusi

Himpunan solusi merupakan kumpulan aksi yang dipilih oleh bot untuk dilakukan oleh seluruh unit pada suatu ronde permainan. Setiap robot dan tower akan memilih satu aksi dari himpunan kandidat yang tersedia berdasarkan prioritas strategi yang digunakan. Contoh himpunan solusi pada suatu ronde dapat berupa:

- Soldier bergerak menuju wilayah netral
- Soldier lainnya menyerang robot musuh
- Splasher melakukan serangan area
- Mopper membersihkan cat musuh
- Tower melakukan spawn robot baru

Gabungan dari seluruh aksi tersebut membentuk keputusan strategi bot pada ronde tersebut.

3.2.3 Fungsi Seleksi

Fungsi seleksi digunakan untuk menentukan aksi mana yang memiliki prioritas tertinggi dari himpunan kandidat. Prioritas aksi berbeda untuk setiap jenis robot karena setiap robot memiliki peran yang berbeda dalam permainan.

Jenis Robot	Prioritas Aksi (Greedy Selection)
Soldier	1. Menyerang musuh dalam jangkauan 2. Mewarnai petak netral 3. Menghapus cat musuh 4. Bergerak menuju wilayah yang belum dicat 5. Membantu menyelesaikan pola tower
Splasher	1. Menyerang area dengan banyak musuh 2. Menyerang wilayah dengan banyak cat musuh 3. Bergerak menuju area konflik 4. Bergerak menuju wilayah musuh
Mopper	1. Menghapus cat musuh 2. Mentransfer cat ke robot sekutu 3. Mengambil cat dari tower 4. Bergerak menuju wilayah dengan banyak cat musuh

Table 3: Fungsi seleksi untuk setiap jenis robot

3.2.4 Fungsi Kelayakan

Fungsi kelayakan digunakan untuk memeriksa apakah suatu aksi dapat dilakukan tanpa melanggar aturan permainan. Beberapa kondisi yang diperiksa antara lain:

- Robot hanya dapat bergerak jika petak tujuan tidak ditempati robot lain dan bukan tembok.
- Robot hanya dapat menyerang jika musuh berada dalam radius serangan.
- Robot hanya dapat mewarnai petak jika memiliki cat yang cukup.
- Robot hanya dapat mentransfer cat jika robot sekutu berada dalam jarak transfer.
- Tower hanya dapat melakukan spawn jika memiliki sumber daya yang cukup dan petak spawn kosong.

Jika suatu kandidat tidak memenuhi kondisi tersebut, maka kandidat tersebut tidak dapat dipilih sebagai bagian dari solusi.

3.2.5 Fungsi Solusi

Fungsi solusi menentukan kapan proses pemilihan aksi dianggap selesai. Dalam konteks Battlecode, solusi dianggap selesai ketika setiap unit telah menentukan aksi yang akan dilakukan pada ronde tersebut. Dengan demikian, proses pemilihan aksi berhenti ketika:

- setiap robot telah memilih satu aksi dari himpunan kandidat
- setiap tower telah menentukan aksinya
- tidak ada aksi tambahan yang dapat dilakukan pada ronde tersebut

3.2.6 Fungsi Objektif

Fungsi objektif mendefinisikan tujuan utama dari strategi greedy yang digunakan oleh bot. Berdasarkan aturan permainan Battlecode, tujuan utama permainan adalah mewarnai lebih dari 70% wilayah peta atau menghancurkan seluruh unit musuh. Dengan demikian, fungsi objektif dari strategi greedy dapat dirumuskan sebagai berikut:

- Memaksimalkan jumlah petak wilayah yang berhasil dicat oleh tim.
- Meminimalkan wilayah yang dikuasai oleh tim lawan.
- Memaksimalkan jumlah menara sekutu yang bertahan.
- Memaksimalkan jumlah robot sekutu yang aktif.
- Meminimalkan kerugian unit akibat serangan musuh.

Strategi greedy yang digunakan oleh bot akan berusaha memilih aksi yang memberikan keuntungan terbesar terhadap tujuan-tujuan tersebut pada setiap ronde permainan.

3.3 Alternatif Solusi Greedy 1

3.3.1 Heuristik yang Digunakan

Strategi greedy pertama yang dirancang berfokus pada dominasi wilayah permainan dengan tetap menjaga keberlangsungan ekonomi tim. Tujuan utama dari strategi ini adalah menguasai wilayah seluas mungkin sambil memastikan suplai *paint* dan *chips* tidak terputus. Dominasi wilayah tanpa dukungan ekonomi yang stabil akan menyebabkan robot tidak dapat bergerak karena kehabisan *paint* atau tim tidak dapat melakukan spawn robot baru ketika unit sebelumnya hancur. Oleh karena itu, strategi ini menggunakan dua pendekatan heuristik utama:

- **Greedy on Coverage**, memaksimalkan jumlah petak wilayah yang berhasil dicat.
- **Greedy on Resource**, memanfaatkan *ruins* dan *SRP* untuk meningkatkan ekonomi tim.

Fungsi seleksi dasar yang digunakan untuk memilih aksi adalah:

$$Score = (10 \times JumlahPetakWarnaBaru) + (50 \text{ jika mendekati Ruins})$$

Robot akan memilih aksi dengan skor tertinggi pada kondisi permainan saat itu.

3.3.2 Fokus Utama Setiap Unit

Setiap jenis robot dan tower memiliki peran yang berbeda dalam mendukung strategi ini.

Unit	Fokus Utama
Splasher	Menjadi unit utama untuk melakukan ekspansi wilayah secara besar-besaran pada area kosong atau wilayah musuh.
Soldier	Digunakan untuk menyelesaikan pola pembangunan tower secara presisi pada lokasi ruins.
Mopper	Menjaga area sekitar tower agar tetap bersih dari cat musuh serta mempertahankan wilayah yang sudah dikuasai.
Tower	Pada awal permainan berfokus pada pembangunan Money Tower untuk menghasilkan chips, kemudian Paint Tower untuk mendukung ekspansi Splasher.

3.3.3 Logika Greedy Sederhana

Logika greedy yang digunakan pada strategi ini dapat diringkas sebagai berikut:

- Jika terdapat *ruins* atau SRP di dekat robot, maka robot memprioritaskan bergerak menuju lokasi tersebut.
- Jika tidak terdapat target sumber daya, robot akan mencari petak netral atau wilayah musuh terdekat untuk diwarnai.
- Robot selalu memilih aksi yang memberikan nilai tertinggi terhadap kontrol wilayah atau peningkatan sumber daya.

3.3.4 Strategi Jalan

Pergerakan robot ditentukan menggunakan fungsi skor berikut:

$$ScoreMove(d) = \alpha \cdot CoverageGain(d) + \beta \cdot ResourceGain(d) + \gamma \cdot Safety(d) + \delta \cdot Connectivity(d) - \epsilon \cdot CostRisk(d)$$

Robot akan memilih arah pergerakan d dengan skor tertinggi.

Keterangan variabel

- *CoverageGain* : peluang robot dapat mengecat lebih banyak petak netral atau musuh.
- *ResourceGain* : bonus jika pergerakan mendekati ruin, SRP, atau paint tower sekutu.
- *Safety* : bonus jika robot berada pada wilayah sekutu atau menjauhi tower musuh.
- *Connectivity* : bonus jika pergerakan menjaga konektivitas jalur cat sekutu.
- *CostRisk* : penalti jika pergerakan masuk wilayah musuh dengan paint rendah, mendekati tower musuh, atau menuju area buntu.

Robot menggunakan beberapa mode operasi: EXPAND, BUILD_TOWER, BUILD_SRP, DEFEND_PAINT_LINE, REFILL, RETREAT

Table 4: Strategi jalan untuk masing-masing robot

Robot	Strategi Jalan
Splasher	• Mengenumerasi semua petak yang mungkin dijadikan pusat serangan dalam jangkauan aksi. • Menghitung jumlah petak netral atau petak musuh yang dapat terdampak oleh serangan area (AoE). • Memilih arah gerakan yang mendekatkan robot pada pusat serangan dengan nilai dampak terbesar. • Memberikan bonus besar pada area dengan cluster petak musuh atau netral yang padat. • Memberikan bonus sedang pada frontier wilayah kosong yang luas. • Memberikan penalti jika hasil AoE mengenai banyak petak yang sudah dimiliki sendiri tanpa membuka frontier baru. • Memberikan penalti tinggi jika setelah melakukan serangan sisa paint menjadi terlalu rendah.
Soldier	• Jika terdapat <i>ruin</i> yang terdeteksi, robot akan langsung bergerak menuju lokasi tersebut. • Jika sedang mengerjakan pembangunan <i>tower</i> atau <i>Special Resource Pattern (SRP)</i>, robot memilih gerakan yang meminimalkan waktu penyelesaian pola. • Jika tidak terdapat tugas ekonomi aktif, robot akan bergerak untuk menutup frontier wilayah di sekitar tower yang baru dibangun. • Petak musuh di sekitar <i>ruin</i>, tower, atau SRP memiliki prioritas target tertinggi. • Petak musuh pada jalur komunikasi cat sekutu menjadi prioritas kedua. • Petak musuh biasa diprioritaskan setelah kedua kondisi di atas.
Mopper	• Bergerak menuju area dengan banyak cat musuh di sekitar ruin, tower, atau SRP. • Memprioritaskan pembersihan wilayah yang penting bagi stabilitas ekonomi tim. • Bergerak ke jalur komunikasi cat sekutu yang sering rusak untuk memperbaiki konektivitas wilayah. • Mendukung robot sekutu yang memiliki paint rendah dengan melakukan transfer paint apabila diperlukan. • Membersihkan area inti tim dengan cepat ketika terdapat gangguan dari cat musuh.

3.3.5 Strategi Target Cat

Strategi target cat berfokus pada aksi pewarnaan yang memberikan peningkatan kontrol wilayah terbesar dengan biaya paint yang efisien.

Splasher

Splasher memilih pusat serangan yang memaksimalkan dampak area.

$$ScoreSplash(c) = 8E + 5N + 2F + 20R - 6A - 10W - 4Overpaint$$

Keterangan:

- E : jumlah petak musuh yang dapat ditimpa
- N : jumlah petak netral yang dapat diwarnai
- F : jumlah petak yang memperluas frontier wilayah
- R : bonus jika dekat ruin atau SRP strategis
- A : jumlah petak sekutu yang terbangun
- W : jumlah petak tidak berguna
- Overpaint : area yang sudah aman milik sendiri

Table 5: Prioritas strategi target cat untuk masing-masing robot

Robot	Prioritas Target Cat
Splasher	• Petak yang memungkinkan serangan area mengenai banyak petak musuh sekaligus. • Petak netral yang berada pada cluster besar sehingga memberikan ekspansi wilayah yang signifikan. • Petak pada frontier wilayah yang dapat membuka akses ekspansi ke area baru. • Petak musuh yang memutus konektivitas wilayah cat sekutu. • Petak netral biasa yang berada di sekitar posisi robot.
Soldier	• Petak yang merupakan bagian dari pola pembangunan tower pada lokasi <i>ruin</i>. • Petak yang merupakan bagian dari pola <i>Special Resource Pattern (SRP)</i>. • Petak musuh yang berada di sekitar ruin atau tower sekutu. • Petak musuh yang mengganggu jalur komunikasi cat sekutu. • Petak netral yang berada di sekitar posisi robot.
Mopper	• Petak yang dicat oleh musuh di sekitar tower sekutu. • Petak musuh yang berada di sekitar ruin atau SRP yang sedang dibangun. • Petak musuh yang memutus jalur konektivitas cat sekutu. • Petak musuh yang berada pada wilayah inti tim. • Petak musuh biasa yang berada di sekitar posisi robot.

3.3.6 Manajemen Sumber Daya

Early Game	Mid Game	Late Game
• stabilkan income chips dan paint • buka 1–2 garis ekspansi	• percepat coverage • amankan suplai paint untuk Splasher	• maximize map coverage • convert ekonomi jadi spam unit / refill paint / menjaga tiebreaker
• ruin dekat + aman → bangun tower • prefer Money Tower dulu jika chips menjadi bottleneck spawn • jangan terlalu banyak investasi SRP terlalu dini	• mulai tambah Paint Tower • bangun SRP jika area sekitar cukup aman dijaga 50 ronde • Mopper lebih aktif menjaga pola dan wilayah sekitar tower	• Money Tower tambahan hanya jika masih benar-benar menaikkan tempo spawn • kalau bottleneck utama sudah paint, ubah prioritas ke Paint Tower

Table 6: Strategi manajemen sumber daya berdasarkan fase permainan

Manajemen sumber daya dilakukan dengan menghitung kebutuhan tim pada setiap ronde.

- $NeedPaint = jumlah\ splasher\ aktif \times konsumsi\ paint$
- $NeedChips = target\ spawn\ rate \times biaya\ unit$
- $NeedDefense = jumlah\ ruin/SRP\ yang\ rawan$

Keputusan strategi:

- Jika chips rendah dan paint cukup → prioritaskan Money Tower
- Jika paint rendah dan unit aktif banyak → prioritaskan Paint Tower
- Jika area rawan besar → tambah Mopper
- Jika ekspansi lambat tetapi resource aman → tambah Splasher

3.3.7 Spawn Strategy

- Early game : Soldier lebih banyak untuk membuka ruin
- Mid game : Splasher menjadi unit dominan
- Jika banyak kontestasi ruin : jumlah Mopper meningkat
- Jika ekonomi paint stabil : produksi Splasher meningkat

3.3.8 Pemanfaatan Pola Khusus (SRP dan Tower)

Pemilihan ruin untuk pembangunan tower menggunakan skor berikut:

$$ScoreRuin = 50 - 2 \times Distance + 20 \times Safety + 25 \times ExpansionValue - 15 \times EnemyInterference$$

Prioritas pemilihan ruin:

1. jarak relatif dekat
2. area sekitar tidak didominasi cat musuh
3. tersedia paint untuk menyelesaikan pola
4. tidak terlalu banyak unit lain yang sedang mengerjakan
5. tower yang dihasilkan mendukung ekspansi selanjutnya

3.4 Alternatif Solusi Greedy 2

Strategi agresif berfokus pada ekspansi wilayah secara cepat dan penekanan terhadap wilayah musuh. Robot diprioritaskan untuk bergerak menuju area yang memiliki konsentrasi petak musuh atau petak netral yang dapat segera diambil alih.

Arah pergerakan robot ditentukan menggunakan fungsi heuristik berikut:

$$\begin{aligned} score(d) = & -10 \cdot dist^2(nextLoc(d), target) + \sum dist^2(nextLoc(d), ally_i) - \sum dist^2(nextLoc(d), enemy_j) \\ & + 8 \quad (\text{jika tile EMPTY}) + 20 \quad (\text{jika tile ENEMY_PRIMARY atau ENEMY_SECONDARY}) \end{aligned}$$

Heuristik ini memberikan prioritas pergerakan menuju area yang mengandung petak musuh atau petak netral, sehingga robot dapat secara agresif memperluas wilayah tim sekaligus menekan wilayah lawan.

Early Phase	
Unit	Strategi
Tower (Round $\leq$ 300)	Fokus memproduksi robot <i>Soldier</i> . Robot Soldier diarahkan menuju lokasi kandidat tower musuh untuk menghancurkan tower tersebut sejak awal permainan.
Soldier (Round $\leq$ 300)	Bergerak menuju dua tower awal milik musuh dengan tujuan menghancurkan tower tersebut pada fase awal permainan.
Splasher (300 $\leq$ round $\leq$ 600)	Berfokus mengambil wilayah pada area tengah peta serta mengurangi persentase wilayah yang dimiliki oleh musuh.
Mopper (300 $\leq$ round $\leq$ 600)	Mencari robot musuh di sekitar wilayah permainan dan melakukan serangan untuk mengganggu ekspansi wilayah lawan.

Table 7: Strategi agresif pada Early Phase

Mid Phase	
Unit	Strategi
Tower (Round $\leq$ 800)	Tower memproduksi semua jenis robot dengan komposisi rasio sekitar 1:2:1 untuk Soldier, Splasher, dan Mopper.
Soldier (Round $\leq$ 1000)	Fokus bergerak menuju robot musuh untuk melakukan penyerangan. Jika tidak terdapat musuh di sekitar wilayah tersebut, robot akan kembali menggunakan strategi seperti pada Early Phase.
Splasher (600 $\leq$ round $\leq$ 1000)	Berfokus untuk mengambil wilayah yang dimiliki musuh serta mencari area yang dapat diperluas.
Mopper (600 $\leq$ round $\leq$ 1000)	Mencari keberadaan robot musuh dan melakukan serangan untuk mengganggu kontrol wilayah lawan.

Table 8: Strategi agresif pada Mid Phase

Late Phase	
Unit	Strategi
Tower (Round $\geq$ 1000)	Tower memfokuskan produksi pada robot Splasher dan Mopper untuk membersihkan wilayah musuh serta mengubahnya menjadi wilayah milik tim.
Soldier (Round $\geq$ 1000)	Jika masih terdapat robot Soldier yang aktif, robot tersebut akan mencari lokasi <i>ruins</i> untuk membangun tower tambahan guna memperoleh sumber daya tambahan yang dapat digunakan untuk memproduksi robot Splasher dan Mopper.
Splasher	Bergerak menuju arah tower musuh untuk mencari tower yang masih bertahan. Jika tidak ditemukan, robot akan bergerak menuju sisi atau sudut wilayah musuh untuk memperluas kontrol wilayah tim.
Mopper	Bergerak menuju robot sekutu terdekat dan mengikuti pergerakannya untuk memberikan dukungan sekaligus membersihkan wilayah yang masih dikuasai musuh.

Table 9: Strategi agresif pada Late Phase

3.5 Alternatif Solusi Greedy 3

Strategi ini menggunakan pendekatan berbasis *state* yang ditentukan oleh beberapa faktor utama, yaitu jumlah paint yang dimiliki robot, jarak terhadap tower sekutu, posisi musuh, serta peluang untuk mewarnai wilayah secara efisien.

Robot akan menentukan tindakan yang diambil berdasarkan *state* yang sedang dialami, sehingga keputusan yang diambil bersifat adaptif terhadap kondisi permainan. Setiap *state* menentukan prioritas tindakan robot seperti melakukan ekspansi wilayah, mempertahankan wilayah, kembali ke tower untuk mengisi ulang paint, atau melakukan penarikan diri dari wilayah musuh.

3.5.1 State Robot

State	Deskripsi dan Perilaku
Refill	Aktif ketika $\text{paint} \leq 20\%$. Pada kondisi ini robot diperkirakan akan mengalami penalti besar jika tetap bergerak atau menyerang dalam beberapa ronde ke depan. • Bergerak menuju paint tower sekutu terdekat • Mendekati Mopper sekutu apabila berada di dekatnya • Bergerak menuju wilayah sekutu dan menjauhi wilayah musuh
Return to Tower	Aktif ketika $20\% \leq \text{paint} \leq 40\%$ dan robot berada jauh dari tower atau terlalu dalam di wilayah musuh. • Mundur menuju wilayah sekutu • Mengecat tile yang murah jika memungkinkan • Memprioritaskan tile yang dapat memperluas jalur sekutu menuju tower
Maintain	Aktif ketika jumlah paint cukup dan tidak ada kebutuhan mendesak untuk ekspansi atau pengisian ulang. • Mengecat tile kosong yang dapat membangun koridor aman • Memperluas area di sekitar tower • Membantu menyelesaikan pola pembangunan • Menghindari tile yang dapat membuat robot terisolasi
Expand	Aktif ketika $\text{paint} > 50\%$ dan terdapat jalur aman untuk bergerak maju. • Menargetkan tile netral yang mudah diambil • Bergerak ke frontier antara wilayah sekutu dan wilayah netral • Memperluas wilayah tanpa terlalu dalam memasuki area musuh • Menargetkan tile di sekitar ruin atau SRP
Contest Light	Aktif ketika terdapat cat musuh di dekat frontier dan robot memiliki peluang menyerang dengan paint yang cukup. • Menghapus atau menimpa tile musuh yang strategis • Memfokuskan pada pengurangan kontrol wilayah musuh • Menjaga efisiensi penggunaan paint
Retreat	Aktif ketika paint rendah dan terdapat musuh di sekitar atau ketika posisi robot berada di wilayah musuh. • Mundur menuju wilayah sekutu terdekat • Memilih tile dengan penalti penggunaan paint minimum • Menghindari kerumunan robot sekutu

Table 10: State robot dalam strategi greedy

3.5.2 Alur Setiap Robot

Robot	Peran dan Perilaku
Soldier	Berperan sebagai unit garis depan yang bertugas melakukan ekspansi wilayah secara aman. • Mengisi frontier wilayah sekutu • Membantu penyelesaian pola pembangunan • Menyerang tower musuh apabila biaya serangan relatif murah dan aman Perilaku berdasarkan paint: • Paint rendah: kembali ke tower • Paint sedang: mendukung frontier • Paint tinggi: melakukan ekspansi
Mopper	Berperan sebagai unit logistik utama yang menjaga stabilitas pasokan paint. • Mendistribusikan paint kepada robot sekutu • Membersihkan cat musuh pada frontier • Menjadi penghubung antara tower dan robot sekutu di garis depan Perilaku tambahan: • Transfer paint jika terdapat sekutu dengan paint rendah • Membersihkan tile musuh jika jumlahnya banyak • Mendekati Soldier atau Splasher yang jauh dari tower
Splasher	Unit situasional yang digunakan pada kondisi tertentu. • Tidak diproduksi pada awal permainan • Digunakan ketika terdapat cluster tile bernilai tinggi • Dipanggil ketika sumber daya tim sudah stabil • Bergerak bersama jalur suplai sekutu • Menyerang jika area cakupan serangan cukup besar

Table 11: Alur perilaku setiap jenis robot

3.5.3 Alur Robot Setiap Phase

Phase	Strategi
Early Game	• Menghindari ekspansi berlebihan • Membangun wilayah aman di sekitar tower • Membuka jalur paint yang stabil • Spawn unit secara hemat dan stabil (Soldier dan beberapa Mopper) • Memperluas radius wilayah aman • Menjaga robot tetap berada di atas batas aman paint
Mid Game	• Mengontrol frontier wilayah • Menjaga jalur suplai antara tower dan unit garis depan • Melakukan serangan ringan terhadap wilayah musuh • Membantu pembangunan pola tower atau SRP jika kondisi aman
Late Game	• Mempertahankan luas wilayah yang dimiliki • Menghindari kehilangan unit yang tidak perlu • Melakukan spawn unit secara kontinu • Menjaga kondisi tiebreaker seperti luas area, jumlah tower, total paint, dan jumlah unit hidup

Table 12: Alur strategi bot berdasarkan fase permainan

3.5.4 Threshold

Robot	Threshold Paint	Tindakan
Soldier	Paint > 50%	Boleh melakukan ekspansi wilayah
	$30\% < paint \leq 50\%$	Mendukung frontier yang aman
	$paint \leq 30\%$	Kembali menuju tower
	$paint \leq 15\%$	Wajib melakukan refill
Mopper	Paint > 60%	Mendukung frontier dan membersihkan wilayah musuh
	$25\% < paint \leq 60\%$	Tetap berada dekat robot sekutu
	$paint \leq 25\%$	Kembali ke tower untuk mengambil paint
Splasher	Paint > 65%	Boleh melakukan serangan AoE
	$paint \leq 65\%$	Menahan serangan agar tidak membuang paint secara tidak efisien

Table 13: Threshold keputusan robot berdasarkan jumlah paint

3.6 Analisis Efisiensi

Analisis berikut membahas efisiensi dan efektivitas dari setiap bot. Efisiensi berkaitan dengan penggunaan sumber daya komputasi maupun sumber daya permainan seperti paint dan chips, serta penggunaan bytecode pada setiap giliran. Sementara itu, efektivitas berkaitan dengan sejauh mana strategi tersebut mampu mencapai tujuan utama permainan, yaitu menguasai wilayah peta dan memenangkan pertandingan.

3.6.1 Alternatif Solusi Greedy 1

Bot pertama menggunakan strategi greedy yang berfokus pada pembangunan menara dan penguatan ekonomi tim. Berbeda dengan bot kedua yang memprioritaskan ekspansi agresif, bot ini lebih menekankan stabilitas jangka panjang melalui pembangunan tower pada lokasi ruins yang tersedia di peta.

Dari sisi efisiensi komputasi, strategi ini sedikit lebih kompleks karena robot harus mempertimbangkan lebih banyak kondisi sebelum memilih aksi. Robot perlu mendeteksi keberadaan ruins di sekitar peta, mengevaluasi apakah lokasi tersebut aman untuk pembangunan tower, serta menentukan apakah sumber daya yang dimiliki cukup untuk menyelesaikan pola pembangunan. Walaupun demikian, kompleksitas ini masih dapat dianggap efisien karena sebagian besar perhitungan tetap bersifat lokal. Robot hanya perlu memeriksa informasi mengenai ruins terdekat, kondisi wilayah di sekitar lokasi tersebut, serta jumlah paint yang dimiliki. Tidak ada pencarian global terhadap seluruh peta yang dapat menghabiskan bytecode secara signifikan. Dengan demikian, strategi ini masih berada dalam batas penggunaan bytecode yang wajar untuk setiap giliran.

Kelemahan dari sisi efisiensi muncul pada banyaknya mode atau state yang digunakan dalam strategi ini. Robot dapat berpindah antara beberapa mode seperti eksplorasi, pembangunan tower, pengisian ulang paint, atau pertahanan wilayah. Jika transisi antar mode ini tidak dikendalikan dengan baik, robot dapat menghabiskan beberapa ronde hanya untuk berpindah mode tanpa menghasilkan kontribusi signifikan terhadap kontrol wilayah. Misalnya, robot dapat berpindah dari mode eksplorasi ke mode pembangunan, kemudian kembali lagi ke eksplorasi sebelum pola pembangunan selesai. Meskipun demikian, secara keseluruhan strategi ini masih cukup efisien dalam penggunaan sumber daya permainan karena pembangunan tower secara konsisten akan meningkatkan produksi paint dan chips pada jangka panjang.

Dari sisi efektivitas, bot pertama ini dapat dianggap sebagai strategi yang paling stabil di antara ketiga alternatif yang dirancang. Pembangunan tower memberikan keuntungan jangka panjang yang signifikan karena tower tidak hanya berfungsi sebagai unit pertahanan tetapi juga sebagai sumber produksi sumber daya. Money tower akan meningkatkan jumlah chips yang tersedia untuk spawn robot baru, sementara paint tower akan meningkatkan suplai paint yang dapat digunakan oleh robot sekutu. Dengan bertambahnya jumlah tower yang dimiliki tim, kapasitas ekspansi wilayah juga meningkat karena robot dapat melakukan refill paint lebih sering tanpa harus kembali ke tower awal. Selain itu, tower juga

memberikan perlindungan tambahan terhadap serangan musuh sehingga wilayah yang sudah dikuasai menjadi lebih sulit direbut kembali oleh lawan. Walaupun strategi ini membutuhkan waktu lebih lama pada fase awal permainan untuk menyelesaikan pola pembangunan tower, keuntungan ekonomi yang dihasilkan akan sangat terasa pada fase mid game dan late game. Dengan ekonomi yang lebih kuat, tim dapat memproduksi lebih banyak robot serta mempertahankan wilayah dengan lebih stabil.

Namun, strategi ini juga memiliki beberapa kelemahan. Fokus yang terlalu besar pada pembangunan tower dapat menyebabkan ekspansi wilayah menjadi lebih lambat pada fase awal permainan. Jika lawan

menggunakan strategi agresif dan berhasil menguasai sebagian besar wilayah peta sejak awal, bot ini mungkin kesulitan untuk mengejar ketertinggalan tersebut.

3.6.2 Alternatif Solusi Greedy 2

Bot kedua menggunakan pendekatan greedy agresif yang memprioritaskan serangan terhadap musuh sebagai aksi utama pada setiap giliran. Strategi ini dirancang untuk menghasilkan tekanan tinggi pada wilayah lawan sejak awal permainan dengan cara mempercepat kontak antara robot sekutu dan wilayah musuh.

Dari sisi efisiensi komputasi, strategi ini relatif sederhana. Keputusan robot sebagian besar hanya didasarkan pada informasi lokal yang dapat diperoleh melalui fungsi sensing pada radius penglihatan robot. Robot tidak perlu melakukan evaluasi global terhadap seluruh peta, melainkan cukup memeriksa apakah terdapat musuh, wilayah netral, atau wilayah musuh di sekitar posisi saat ini. Pendekatan ini membuat proses evaluasi aksi menjadi cepat dan ringan sehingga penggunaan bytecode pada setiap giliran tetap rendah. Hal ini penting karena dalam Battlecode setiap robot memiliki batas penggunaan bytecode pada setiap ronde, dan jika batas tersebut terlampaui maka eksekusi program robot akan berhenti sebelum seluruh logika selesai dijalankan. Selain itu, pembagian strategi ke dalam tiga fase permainan yaitu early phase, mid phase, dan late phase juga memberikan keuntungan dari sisi efisiensi desain. Dengan membagi permainan berdasarkan jumlah ronde, setiap robot dapat menggunakan aturan yang lebih sederhana pada masing-masing fase tanpa harus melakukan perhitungan kompleks mengenai kondisi permainan secara keseluruhan. Pendekatan ini membuat implementasi program menjadi lebih modular dan lebih mudah dipahami.

Namun, dari sisi efisiensi penggunaan sumber daya permainan, bot ini memiliki beberapa kekurangan. Heuristik yang digunakan untuk memilih aksi cenderung terlalu kasar sehingga tidak selalu mempertimbangkan kondisi sumber daya secara optimal. Misalnya, pemilihan unit yang di-spawn oleh tower sebagian besar ditentukan oleh fase permainan, bukan oleh kebutuhan aktual di lapangan. Selain itu, pengelolaan cadangan paint pada robot tidak terlalu diperhatikan secara eksplisit. Akibatnya, robot sering tetap bergerak atau menyerang secara agresif meskipun cadangan paint sedang rendah. Padahal dalam aturan permainan, robot yang memiliki paint rendah akan mengalami peningkatan cooldown pada aksi tertentu. Hal ini dapat menyebabkan robot menjadi tidak efektif dalam melakukan ekspansi wilayah atau serangan. Dengan kata lain, walaupun strategi ini efisien dari sisi komputasi, efisiensi penggunaan sumber daya permainan masih dapat ditingkatkan.

Dari sisi efektivitas, bot kedua memiliki beberapa keunggulan pada fase awal permainan. Strategi agresif memungkinkan robot untuk dengan cepat menjangkau wilayah musuh dan mulai mengganggu kontrol wilayah lawan. Pada peta yang relatif terbuka dan tidak terlalu banyak rintangan, bot ini dapat menyebar dengan cepat dan menekan wilayah lawan sebelum mereka sempat membangun struktur atau menyiapkan pertahanan yang kuat. Selain itu, karena peta pada Battlecode bersifat simetris, strategi ini sering dapat memanfaatkan informasi mengenai posisi awal musuh untuk langsung mengarahkan robot menuju lokasi yang kemungkinan besar menjadi pusat aktivitas lawan. Hal ini meningkatkan peluang terjadinya konflik awal yang dapat menguntungkan tim jika berhasil memenangkan pertarungan tersebut.

Namun, efektivitas strategi ini cenderung menurun pada fase permainan yang lebih panjang. Hal ini disebabkan oleh kurangnya pengelolaan ekonomi dan logistik dalam strategi tersebut. Pembangunan menara tidak selalu menjadi prioritas, sehingga kapasitas produksi paint dan chips tim dapat tertinggal dibandingkan lawan yang lebih fokus pada pembangunan struktur ekonomi. Selain itu, peran robot Mopper dalam strategi

ini relatif terbatas dan lebih bersifat reaktif, yaitu hanya membersihkan cat musuh di sekitar wilayah sekutu tanpa berperan aktif dalam menjaga suplai paint bagi robot garis depan. Akibatnya, meskipun bot ini dapat memberikan tekanan tinggi pada awal permainan, performanya sering kali menurun pada fase mid game dan late game ketika lawan sudah memiliki ekonomi yang lebih stabil serta produksi unit yang lebih konsisten. Dalam kondisi tersebut, bot ini dapat kesulitan mempertahankan wilayah yang telah diperoleh sebelumnya.

3.6.3 Alternatif Solusi Greedy 3

Bot ketiga menggunakan pendekatan greedy yang berfokus pada efisiensi penggunaan sumber daya dalam setiap keputusan. Strategi ini mengadopsi pendekatan berbasis state di mana robot menentukan tindakan berdasarkan kondisi internal seperti jumlah paint, posisi relatif terhadap tower sekutu, keberadaan musuh di sekitar, serta peluang ekspansi wilayah.

Dari sisi efisiensi komputasi, bot ini merupakan yang paling kompleks di antara ketiga strategi yang dikembangkan. Setiap robot perlu mengevaluasi berbagai kemungkinan aksi berdasarkan beberapa faktor sekaligus, seperti keuntungan wilayah yang diperoleh, biaya paint yang dikeluarkan, risiko terhadap serangan musuh, serta keberlanjutan posisi robot pada beberapa ronde berikutnya. Meskipun demikian, strategi ini tetap dirancang agar sesuai dengan batas bytecode yang ditetapkan oleh permainan. Sebagian besar evaluasi keputusan hanya menggunakan data lokal yang diperoleh dari fungsi sensing pada ronde tersebut. Tidak ada penggunaan algoritma pencarian yang kompleks seperti pathfinding global atau perhitungan optimasi yang memerlukan iterasi besar. Selain itu, beberapa optimasi juga diterapkan untuk mengurangi biaya komputasi, seperti menyimpan informasi mengenai lokasi ruin dalam cache untuk beberapa ronde serta membatasi target ekspansi robot hanya pada area tertentu. Dengan pendekatan ini, kompleksitas keputusan tetap dapat dikendalikan sehingga robot tidak melampaui batas bytecode yang tersedia.

Dari sisi efektivitas, strategi ini memiliki pendekatan yang sangat berhati-hati dalam menggunakan sumber daya. Robot akan selalu mempertimbangkan rasio antara keuntungan yang diperoleh dan biaya yang harus dikeluarkan sebelum melakukan suatu aksi. Hal ini membuat penggunaan paint menjadi lebih hemat dan mengurangi risiko robot kehabisan paint di wilayah musuh.

Namun, fokus yang terlalu besar pada efisiensi lokal ini juga menimbulkan beberapa kelemahan. Karena robot cenderung memilih aksi yang paling efisien secara langsung, mereka sering menghindari tindakan yang memiliki risiko lebih tinggi tetapi berpotensi memberikan keuntungan strategis yang lebih besar. Misalnya, robot mungkin memilih untuk memperluas wilayah secara perlahan di sekitar area aman daripada melakukan penetrasi lebih dalam ke wilayah musuh. Akibatnya, tekanan terhadap wilayah lawan menjadi relatif lebih rendah dibandingkan dengan strategi agresif pada bot pertama. Selain itu, pembangunan tower juga tidak selalu menjadi prioritas utama dalam strategi ini karena robot sering lebih fokus pada optimalisasi penggunaan paint pada wilayah yang sudah dikuasai.

Secara keseluruhan, strategi ini sangat baik dalam menjaga stabilitas sumber daya dan meminimalkan kerugian unit. Namun, dalam beberapa situasi permainan, strategi ini dapat menjadi terlalu defensif sehingga kehilangan momentum ekspansi wilayah yang penting untuk memenangkan permainan.

3.7 Strategi yang Dipilih

Dari tiga alternatif strategi greedy yang telah dirancang dan dianalisis sebelumnya, strategi yang dipilih untuk diimplementasikan dalam program adalah strategi pada Alternatif Solusi Greedy 1, yaitu strategi

greedy yang berfokus pada pembangunan menara (tower) dan penguatan ekonomi tim.

Pemilihan strategi ini didasarkan pada beberapa pertimbangan utama, seperti stabilitas permainan, efisiensi penggunaan sumber daya, serta efektivitas dalam jangka panjang. Dari sisi efektivitas permainan, strategi Alternatif 1 dinilai lebih mampu memberikan performa yang stabil sepanjang permainan. Berbeda dengan Alternatif 2 yang lebih agresif dalam melakukan ekspansi wilayah sejak awal permainan, Alternatif 1 menempatkan pembangunan tower sebagai prioritas utama ketika menemukan lokasi ruins yang tersedia. Tower memiliki peran penting dalam permainan karena dapat menghasilkan sumber daya tambahan seperti paint dan chips, serta berfungsi sebagai titik pertahanan terhadap serangan musuh. Dengan meningkatnya jumlah tower yang dimiliki tim, kapasitas produksi sumber daya akan meningkat sehingga tim dapat memproduksi lebih banyak robot serta mempertahankan wilayah dengan lebih baik.

Secara efisiensi penggunaan sumber daya, strategi Alternatif 1 memberikan pengelolaan sumber daya yang lebih terstruktur. Pembangunan tower memungkinkan tim memperoleh suplai paint dan chips secara berkelanjutan sehingga robot tidak terlalu sering mengalami kekurangan sumber daya saat melakukan ekspansi wilayah atau serangan. Selain itu, keberadaan tower juga mempermudah robot untuk melakukan pengisian ulang paint tanpa harus kembali ke titik awal, sehingga waktu dan langkah pergerakan robot dapat digunakan secara lebih efisien.

Strategi Alternatif 1 dinilai memiliki keseimbangan antara ekspansi wilayah dan pembangunan infrastruktur tim. Walaupun pembangunan tower membutuhkan waktu pada fase awal permainan, keuntungan ekonomi yang dihasilkan akan memberikan dampak yang signifikan pada fase permainan selanjutnya. Dengan ekonomi yang lebih kuat, tim dapat memproduksi robot secara lebih konsisten serta memperluas wilayah dengan lebih stabil.

Strategi ini juga lebih adaptif terhadap berbagai kondisi peta. Pada peta yang memiliki banyak ruins, Alternatif 1 dapat memanfaatkan peluang tersebut untuk membangun lebih banyak tower dan memperkuat kontrol wilayah secara bertahap. Hal ini memberikan keuntungan strategis dibandingkan pendekatan yang hanya berfokus pada ekspansi agresif tanpa mempertimbangkan pembangunan ekonomi.

Berdasarkan pertimbangan tersebut, strategi greedy pada Alternatif 1 dipilih sebagai strategi utama yang akan diimplementasikan dalam program. Strategi ini dinilai memberikan kombinasi yang lebih seimbang antara efisiensi penggunaan sumber daya, efektivitas penguasaan wilayah, serta stabilitas performa sepanjang permainan.

4 Implementasi dan Pengujian

4.1 Pseudocode

4.1.1 Algoritma Solusi Greedy 1

G.java

procedure markVisited(loc : MapLocation)

Menandai bahwa grid kasar yang memuat lokasi ini telah dikunjungi pada round sekarang.

Kamus:

gy, gx : integer

Algorithm

gy $\leftarrow$ loc.y div 2

gx $\leftarrow$ loc.x div 2

if gy ≥ 0 and gy < 30 and gx ≥ 0 and gx < 30 then

visited[gy][gx] $\leftarrow$ round

endif

function getVisited(loc : MapLocation) $\rightarrow$ integer

Mengembalikan round terakhir lokasi ini dikunjungi.

Kamus:

gy, gx : integer

Algorithm

gy $\leftarrow$ loc.y div 2

gx $\leftarrow$ loc.x div 2

if gy < 0 or gy ≥ 30 or gx < 0 or gx ≥ 30 then

return 0

endif

return visited[gy][gx]

function cheby(a : MapLocation, b : MapLocation) $\rightarrow$ integer

Menghitung jarak Chebyshev antara dua titik pada grid.

Kamus:

dx, dy : integer

Algorithm

dx $\leftarrow$ abs(a.x - b.x)

```
dy ← abs(a.y - b.y)
return max(dx, dy)
```

MapMemory.java

procedure initializeMapMemory()

Mengisi `towerGrid` dengan -1 dan `ruinClaimRound` dengan nilai awal kecil.

Kamus:

i, j, k : integer

Algorithm

```
for i ← 0 to 11 do
  for j ← 0 to 11 do
    towerGrid[i][j] ← -1
  endfor
endfor
for k ← 0 to MAX_RUINS - 1 do
  ruinClaimRound[k] ← -9999
endfor
```

procedure registerRuin(loc : MapLocation, team : Team, type : UnitType)

Mendaftarkan ruin atau tower yang baru terlihat, atau memperbarui entri lama.

Kamus:

gx, gy : integer

idx : integer

Algorithm

```
gx ← loc.x div 5
gy ← loc.y div 5
if gx ≥ 12 or gy ≥ 12 then
  return
endif
idx ← towerGrid[gy][gx]
if idx ≥ 0 then
  ruinTeams[idx] ← team
  ruinTypes[idx] ← type
```

```
    ruinStatus[idx] ← teamToStatus(team)
    if team = G.team then
        call recomputeAllyCount()
    endif
    return
endif
if numRuins ≥ MAX_RUINS then
    return
endif
idx ← numRuins
numRuins ← numRuins + 1
towerGrid[gy][gx] ← idx
ruinLocs[idx] ← loc
ruinTeams[idx] ← team
ruinTypes[idx] ← type
ruinStatus[idx] ← teamToStatus(team)
if ruinStatus[idx] = 1 then
    ruinClaimRound[idx] ← -9999
endif
call recomputeAllyCount()
```

function teamToStatus(t : Team) → integer

Mengubah team menjadi status ruin.

Kamus:

-

Algorithm

```
if t = null or t = Team.NEUTRAL then
    return 0
endif
if t = G.team then
    return 1
endif
return 2
```

procedure recomputeAllyCount()

Menghitung ulang jumlah tower ally berdasarkan ruin yang sudah terdaftar.

Kamus:

i : integer

base : UnitType

Algorithm

allyPaintTowers $\leftarrow$ 0

allyMoneyTowers $\leftarrow$ 0

allyDefenseTowers $\leftarrow$ 0

totalAllyTowers $\leftarrow$ 0

for i $\leftarrow$ numRuins - 1 downto 0 do

 if ruinTeams[i] = G.team then

 totalAllyTowers $\leftarrow$ totalAllyTowers + 1

 if ruinTypes[i] $\neq$ null then

 base $\leftarrow$ ruinTypes[i].getBaseType()

 if base = LEVEL_ONE_PAINT_TOWER then

 allyPaintTowers $\leftarrow$ allyPaintTowers + 1

 else if base = LEVEL_ONE_MONEY_TOWER then

 allyMoneyTowers $\leftarrow$ allyMoneyTowers + 1

 else if base = LEVEL_ONE_DEFENSE_TOWER then

 allyDefenseTowers $\leftarrow$ allyDefenseTowers + 1

 endif

 endif

endif

endfor

function chooseTowerType() $\rightarrow$ UnitType

Memilih tipe tower secara greedy berdasarkan keseimbangan ekonomi, paint, dan defense.

Kamus:

paintRatio : real

moneyRatio : real

Algorithm

if allyPaintTowers = 0 then


```
    return LEVEL.ONE_PAINT_TOWER
endif

if totalAllyTowers < 3 or allyMoneyTowers < max(1, allyPaintTowers * 0.7) then
    return LEVEL.ONE_MONEY_TOWER
endif

if G.lastKnownEnemyTower ≠ null and G.round - G.lastEnemyTowerRound < 60 and
allyDefenseTowers = 0 then
    return LEVEL.ONE_DEFENSE_TOWER
endif

paintRatio ← allyPaintTowers / max(1, totalAllyTowers)
moneyRatio ← allyMoneyTowers / max(1, totalAllyTowers)
if paintRatio < 0.45 then
    return LEVEL.ONE_PAINT_TOWER
endif
if moneyRatio < 0.35 then
    return LEVEL.ONE_MONEY_TOWER
endif
return LEVEL.ONE_PAINT_TOWER
```

function getEnemyMirror(allyLoc : MapLocation) → MapLocation

Mengembalikan lokasi cermin musuh berdasarkan dugaan simetri.

Kamus:

-

Algorithm

```
if symmetry[0] then
    return MapLocation(G.mapWidth - 1 - allyLoc.x, allyLoc.y)
endif
if symmetry[1] then
    return MapLocation(allyLoc.x, G.mapHeight - 1 - allyLoc.y)
endif
return MapLocation(G.mapWidth - 1 - allyLoc.x, G.mapHeight - 1 - allyLoc.y)
```

procedure scanNearby()

Memindai ruin dan enemy tower di sekitar lalu memasukkannya ke memory.

Kamus:

i : integer

ruin : MapLocation

t : Team

ut : UnitType

ri : RobotInfo

r : RobotInfo

Algorithm

```
for i ← length(G.nearbyRuins) - 1 downto 0 do
  ruin ← G.nearbyRuins[i]
  t ← Team.NEUTRAL
  ut ← null
  if G.rc.canSenseRobotAtLocation(ruin) then
    ri ← G.rc.senseRobotAtLocation(ruin)
    if ri ≠ null then
      t ← ri.team
      ut ← ri.type
    endif
  endif
  call registerRuin(ruin, t, ut)
endfor

for i ← length(G.nearbyEnemies) - 1 downto 0 do
  r ← G.nearbyEnemies[i]
  if not r.type.isRobotType() then
    call registerRuin(r.location, G.opponentTeam, r.type)
    G.lastKnownEnemyTower ← r.location
    G.lastEnemyTowerRound ← G.round
  endif
endfor
```

function closestNeutralRuin(from : MapLocation) → MapLocation

Mengembalikan ruin netral terdekat.

Kamus:

best : MapLocation

bestDist : integer

d, i : integer

Algorithm

best $\leftarrow$ null

bestDist $\leftarrow +\infty$

for i $\leftarrow$ numRuins - 1 downto 0 do

 if ruinStatus[i] = 0 then

 d $\leftarrow$ from.distanceSquaredTo(ruinLocs[i])

 if d < bestDist then

 bestDist $\leftarrow$ d

 best $\leftarrow$ ruinLocs[i]

 endif

 endif

endfor

return best

function closestEnemyTower(from : MapLocation) $\rightarrow$ MapLocation

Mengembalikan enemy tower terdekat yang sudah diketahui.

Kamus:

best : MapLocation

bestDist : integer

d, i : integer

Algorithm

best $\leftarrow$ null

bestDist $\leftarrow +\infty$

for i $\leftarrow$ numRuins - 1 downto 0 do

 if ruinStatus[i] = 2 then

 d $\leftarrow$ from.distanceSquaredTo(ruinLocs[i])

 if d < bestDist then

 bestDist $\leftarrow$ d

 best $\leftarrow$ ruinLocs[i]

```
endif  
endif  
endfor  
return best
```

Nav.java

procedure moveToward(target : MapLocation)

Bergerak menuju target dengan greedy step, lalu fallback ke bug navigation jika terhalang.

Kamus:

ideal : Direction
offset : integer
dr, dl : Direction
attempt : Direction
i : integer

Algorithm

```
if not G.rc.isMovementReady() then  
    return  
endif  
if G.me = target then  
    return  
endif  
ideal ← G.me.directionTo(target)  
if bugTarget = null or bugTarget ≠ target or G.me.distanceSquaredTo(target) < bugStartDist then  
    bugDir ← null  
    bugTarget ← target  
    bugStartDist ← G.me.distanceSquaredTo(target)  
    bugSteps ← 0  
endif  
if bugDir = null then  
    if tryMove(ideal) then  
        bugDir ← null  
        return  
    endif
```

```
for offset  $\leftarrow$  1 to 4 do
  dr  $\leftarrow$  ideal
  repeat offset times
    dr  $\leftarrow$  dr.rotateRight()
  endrepeat
  if tryMove(dr) then
    bugDir  $\leftarrow$  dr
    bugRot  $\leftarrow$  1
    bugSteps  $\leftarrow$  0
    return
  endif
  dl  $\leftarrow$  ideal
  repeat offset times
    dl  $\leftarrow$  dl.rotateLeft()
  endrepeat
  if tryMove(dl) then
    bugDir  $\leftarrow$  dl
    bugRot  $\leftarrow$  -1
    bugSteps  $\leftarrow$  0
    return
  endif
endfor
else
  bugSteps  $\leftarrow$  bugSteps + 1
  if bugSteps > BUG_RESET then
    bugDir  $\leftarrow$  null
    bugSteps  $\leftarrow$  0
    return
  endif
  if tryMove(ideal) then
    bugDir  $\leftarrow$  null
    return
  endif
endif
```

```
if bugRot = 1 then
    attempt ← bugDir.rotateLeft()
else
    attempt ← bugDir.rotateRight()
endif
for i ← 0 to 7 do
    if tryMove(attempt) then
        bugDir ← attempt
        return
    endif
    if bugRot = 1 then
        attempt ← attempt.rotateRight()
    else
        attempt ← attempt.rotateLeft()
    endif
endfor
endif
```

procedure moveAway(avoid : MapLocation)

Bergerak menjauh dari satu lokasi.

Kamus:

away : Direction

Algorithm

```
if not G.rc.isMovementReady() then
    return
endif
away ← avoid.directionTo(G.me)
if tryMove(away) then return endif
if tryMove(away.rotateLeft()) then return endif
if tryMove(away.rotateRight()) then return endif
```

function tryMove(d : Direction) → boolean

Mencoba bergerak ke arah d.

Kamus:

-

Algorithm

```
if d = null or d = CENTER then
    return false
endif
if G.rc.canMove(d) then
    G.rc.move(d)
    G.me ← G.rc.getLocation()
    call G.markVisited(G.me)
    return true
endif
return false
```

function pickExploreTarget() → MapLocation

Memilih target eksplorasi yang cenderung belum dikunjungi dan cukup jauh.

Kamus:

best : MapLocation
bestScore : integer
attempt : integer
x, y : integer
candidate : MapLocation
score : integer
v : integer

Algorithm

```
best ← null
bestScore ← -1
for attempt ← 0 to 7 do
    x ← Rand.nextInt(G.mapWidth)
    y ← Rand.nextInt(G.mapHeight)
    candidate ← MapLocation(x, y)
    score ← 0
    v ← G.getVisited(candidate)
```

```
    if v = 0 then
        score ← score + 100
    else
        score ← score + max(0, G.round - v - 50)
    endif
    score ← score + G.me.distanceSquaredTo(candidate) div 10
    if score > bestScore then
        bestScore ← score
        best ← candidate
    endif
endfor
if best ≠ null then
    return best
endif
return MapLocation(Rand.nextInt(G.mapWidth), Rand.nextInt(G.mapHeight))
```

Rand.java

function next() → integer

Menghasilkan bilangan pseudorandom non-negatif berikutnya.

Kamus:

-

Algorithm

state ← state xor (state << 13)

state ← state xor (state >> 17)

state ← state xor (state << 5)

return state and 0x7ffffff

function nextInt(bound : integer) → integer

Menghasilkan bilangan acak dalam rentang 0 sampai bound-1.

Kamus:

-

Algorithm

return next() mod bound

RobotPlayer.java

procedure run(rc : RobotController)

Entry point utama. Inisialisasi global, lalu jalankan logic unit sesuai tipenya setiap round.

Kamus:

r : integer

Algorithm

try

G.rc ← rc

G.mapWidth ← rc.getMapWidth()

G.mapHeight ← rc.getMapHeight()

G.mapArea ← G.mapWidth * G.mapHeight

G.mapCenter ← MapLocation(G.mapWidth div 2, G.mapHeight div 2)

G.team ← rc.getTeam()

G.opponentTeam ← G.team.opponent()

G.roundSpawned ← rc.getRoundNum()

G.indicator ← new StringBuilder()

Rand.state ← rc.getID() * 0x1A3C5E7 + 0x9B2D4F

call updateRound()

switch rc.getType()

case SOLDIER : call Soldier.init()

case SPLASHER : call Splasher.init()

case MOPPER : call Mopper.init()

default : call Tower.init()

endswitch

while true do

r ← rc.getRoundNum()

try

call updateRound()

switch rc.getType()

case SOLDIER : call Soldier.run()

case SPLASHER : call Splasher.run()

case MOPPER : call Mopper.run()

default : call Tower.run()

```
endswitch  
rc.setIndicatorString(G.indicator.toString())  
G.indicator ← new StringBuilder()  
catch GameActionException e  
    print error game action  
    G.indicator ← new StringBuilder()  
catch Exception e  
    print general error  
    G.indicator ← new StringBuilder()  
    print stack trace  
endtry  
if rc.getRoundNum() ≠ r then  
    print bytecode overflow warning  
endif  
Clock.yield()  
endwhile  
catch Exception e  
    print fatal init error  
    print stack trace  
endtry
```

procedure updateRound()

Memperbarui semua state global G untuk round saat ini.

Kamus:

-

Algorithm

```
G.round ← G.rc.getRoundNum()  
G.me ← G.rc.getLocation()  
G.nearbyAllies ← G.rc.senseNearbyRobots(-1, G.team)  
G.nearbyEnemies ← G.rc.senseNearbyRobots(-1, G.opponentTeam)  
G.nearbyMapInfos ← G.rc.senseNearbyMapInfos()  
G.nearbyRuins ← G.rc.senseNearbyRuins(-1)  
call G.markVisited(G.me)
```

call MapMemory.scanNearby()

Mopper.java

procedure init()

Inisialisasi seed random untuk Mopper.

Kamus:

-

Algorithm

$\text{Rand.state} \leftarrow \text{G.rc.getID()} * 31415926 + 271$

procedure run()

Prosedur utama Mopper per round.

Kamus:

paint : integer

Algorithm

$\text{paint} \leftarrow \text{G.rc.getPaint}()$

if $\text{paint} < 20$ and $\text{mode} \neq \text{RETREAT}$ then

$\text{mode} \leftarrow \text{RETREAT}$

endif

if $\text{mode} = \text{RETREAT}$ and $\text{paint} \geq 70$ then

$\text{mode} \leftarrow \text{EXPLORE}$

endif

call tryCompletePatterns()

switch mode

 case EXPLORE : call doExplore()

 case SUPPORT : call doSupport()

 case RETREAT : call doRetreat()

endswitch

append indicator string

procedure doExplore()

Mode eksplorasi: bantu ally jika perlu, bergerak ke area enemy paint, lalu mop.

Kamus:

needyAlly : RobotInfo

bestDir : Direction

Algorithm

needyAlly $\leftarrow$ findNeedyAlly()

if needyAlly $\neq$ null then

 mode $\leftarrow$ SUPPORT

 call doSupport()

 return

endif

if G.rc.isMovementReady() then

 bestDir $\leftarrow$ pickMopDirection()

 if bestDir $\neq$ null and G.rc.canMove(bestDir) then

 G.rc.move(bestDir)

 G.me $\leftarrow$ G.rc.getLocation()

 call G.markVisited(G.me)

 else

 if exploreTarget = null or G.me.isWithinDistanceSquared(exploreTarget, 4) or exploreAge > EXPLORE_TIMEOUT then

 exploreTarget $\leftarrow$ Nav.pickExploreTarget()

 exploreAge $\leftarrow$ 0

 endif

 exploreAge $\leftarrow$ exploreAge + 1

 call Nav.moveToward(exploreTarget)

 endif

endif

if G.rc.isActionReady() then

 call doMopAction()

endif

function pickMopDirection() $\rightarrow$ Direction

Memilih arah gerak yang mendekati region dengan enemy paint terbanyak.

Kamus:

bestDir : Direction

```
bestScore : integer
```

```
d : Direction
```

```
next : MapLocation
```

```
score : integer
```

```
info : MapInfo
```

```
i : integer
```

Algorithm

```
bestDir ← null
```

```
bestScore ← 0
```

```
for each d in G.DIRECTIONS do
```

```
    if not G.rc.canMove(d) then
```

```
        continue
```

```
    endif
```

```
    next ← G.me.add(d)
```

```
    score ← 0
```

```
    for each info in G.nearbyMapInfos do
```

```
        if not next.isWithinDistanceSquared(info.getMapLocation(), 8) then
```

```
            continue
```

```
        endif
```

```
        if info.getPaint().isEnemy() then
```

```
            score ← score + 14
```

```
        endif
```

```
    endfor
```

```
    for i ← length(G.nearbyEnemies) - 1 downto 0 do
```

```
        if G.nearbyEnemies[i].type.isRobotType() and next.isWithinDistanceSquared(G.nearbyEnemies[i].location,  
8) then
```

```
            score ← score + 8
```

```
        endif
```

```
    endfor
```

```
    if G.getVisited(next) = 0 then
```

```
        score ← score + 3
```

```
    endif
```

```
    if score > bestScore then
```

```
        bestScore ← score
```

```
        bestDir ← d
    endif
endfor
return bestDir
```

procedure doMopAction()

Memilih aksi mop terbaik: swing bila banyak robot musuh, jika tidak mop target tunggal terbaik.

Kamus:

bestSwing : Direction
bestTarget : MapLocation
bestScore : integer
info : MapInfo
loc : MapLocation
paint : PaintType
score : integer
i : integer

Algorithm

```
bestSwing ← findBestSwing()
if bestSwing ≠ null then
    if G.rc.canAttack(G.me.add(bestSwing)) then
        G.rc.attack(G.me.add(bestSwing))
        return
    endif
endif
bestTarget ← null
bestScore ← 0
for each info in G.nearbyMapInfos do
    loc ← info.getMapLocation()
    if not G.me.isWithinDistanceSquared(loc, 2) then continue endif
    if not G.rc.canAttack(loc) then continue endif
    score ← 0
    paint ← info.getPaint()
    if paint.isEnemy() then
```

```
    score ← score + 20
  endif
  for i ← length(G.nearbyEnemies) - 1 downto 0 do
    if G.nearbyEnemies[i].type.isRobotType() and G.nearbyEnemies[i].location = loc then
      score ← score + 10
      break
    endif
  endfor
  if score > bestScore then
    bestScore ← score
    bestTarget ← loc
  endif
endfor
if bestTarget ≠ null then
  G.rc.attack(bestTarget)
endif
```

function findBestSwing() → Direction

Memilih cardinal direction yang mengenai robot musuh terbanyak.

Kamus:

bestDir : Direction

bestHits : integer

d : Direction

step1 : MapLocation

step2 : MapLocation

hits : integer

i : integer

enemy : RobotInfo

Algorithm

bestDir ← null

bestHits ← 1

for each d in {NORTH, SOUTH, EAST, WEST} do

step1 ← G.me.add(d)

```
step2 ← step1.add(d)
hits ← 0
for i ← length(G.nearbyEnemies) - 1 downto 0 do
    enemy ← G.nearbyEnemies[i]
    if not enemy.type.isRobotType() then
        continue
    endif
    if enemy.location.isWithinDistanceSquared(step1, 1) or enemy.location.isWithinDistanceSquared(step2,
1) then
        hits ← hits + 1
    endif
endfor
if hits > bestHits then
    bestHits ← hits
    bestDir ← d
endif
endfor
return bestDir
```

function findNeedyAlly() → RobotInfo

Mencari ally robot di sekitar yang paint-nya rendah.

Kamus:

i : integer

ally : RobotInfo

paintCap : integer

Algorithm

```
for i ← length(G.nearbyAllies) - 1 downto 0 do
    ally ← G.nearbyAllies[i]
    if not ally.type.isRobotType() then continue endif
    if not G.me.isWithinDistanceSquared(ally.location, 2) then continue endif
    paintCap ← ally.type.paintCapacity
    if paintCap > 0 and ally.paintAmount < paintCap * 0.3 then
        return ally
    endif
endfor
```



```
endif  
endfor  
return null
```

procedure doSupport()

Bergerak ke ally yang butuh paint lalu transfer paint.

Kamus:

ally : RobotInfo
give : integer

Algorithm

```
ally ← findNeedyAlly()  
if ally = null then  
    mode ← EXPLORE  
    return  
endif  
if G.rc.isMovementReady() then  
    call Nav.moveToward(ally.location)  
endif  
if G.rc.isActionReady() and G.me.isWithinDistanceSquared(ally.location, 2) then  
    give ← min(G.rc.getPaint() - 10, ally.type.paintCapacity - ally.paintAmount)  
    if give > 0 and G.rc.canTransferPaint(ally.location, give) then  
        G.rc.transferPaint(ally.location, give)  
        mode ← EXPLORE  
    endif  
endif  
endif
```

procedure doRetreat()

Mundur ke paint tower terdekat untuk refill.

Kamus:

i : integer
ally : RobotInfo
need : integer

tower : MapLocation

Algorithm

```
for i ← length(G.nearbyAllies) - 1 downto 0 do
  ally ← G.nearbyAllies[i]
  if ally.type.isRobotType() then continue endif
  if ally.type.getBaseType() ≠ LEVEL_ONE_PAINT_TOWER then continue endif
  if not G.me.isWithinDistanceSquared(ally.location, 2) then continue endif
  need ← G.rc.getType().paintCapacity - G.rc.getPaint()
  if need > 0 and G.rc.canTransferPaint(ally.location, -need) then
    G.rc.transferPaint(ally.location, -need)
  return
endif
endfor
tower ← findNearestPaintTowerLoc()
if tower ≠ null and G.rc.isMovementReady() then
  call Nav.moveToward(tower)
endif
```

procedure tryCompletePatterns()

Menyelesaikan tower pattern yang sudah siap di ruin sekitar.

Kamus:

i : integer
ruin : MapLocation
tt : UnitType

Algorithm

```
for i ← length(G.nearbyRuins) - 1 downto 0 do
  ruin ← G.nearbyRuins[i]
  for each tt in Soldier.TOWER_TYPES do
    if G.rc.canCompleteTowerPattern(tt, ruin) then
      G.rc.completeTowerPattern(tt, ruin)
      call MapMemory.registerRuin(ruin, G.team, tt)
      break
    endif
```

```
endfor  
endfor
```

function findNearestPaintTowerLoc() → MapLocation

Mengembalikan lokasi ally paint tower terdekat.

Kamus:

best : MapLocation

bestDist : integer

d, i : integer

Algorithm

best ← null

bestDist ← $+\infty$

for i ← MapMemory.numRuins - 1 downto 0 do

 if MapMemory.ruinTeams[i] = G.team and MapMemory.ruinTypes[i] ≠ null then

 if MapMemory.ruinTypes[i].getBaseType() = LEVEL_ONE_PAINT_TOWER then

 d ← G.me.distanceSquaredTo(MapMemory.ruinLocs[i])

 if d < bestDist then

 bestDist ← d

 best ← MapMemory.ruinLocs[i]

 endif

 endif

 endif

endfor

return best

function modeStr() → string

Mengubah kode mode Mopper menjadi string.

Kamus:

-

Algorithm

switch mode

 case EXPLORE : return "EXP"

 case SUPPORT : return "SUP"

```
case RETREAT : return "RET"  
default : return "?"  
endswitch
```

Soldier.java

procedure init()

Inisialisasi pattern tower dan resource serta seed random.

Kamus:

-

Algorithm

```
towerPatterns  $\leftarrow$  array berisi pattern paint, money, defense tower  
resourcePattern  $\leftarrow$  G.rc.getResourcePattern()  
Rand.state  $\leftarrow$  G.rc.getID() * 1234567 + 89
```

procedure run()

Prosedur utama Soldier tiap round.

Kamus:

```
paint : integer  
paintCap : integer
```

Algorithm

```
call opportunisticComplete()  
call tryCompleteResourcePatterns()  
call tryWithdrawPaint()  
paint  $\leftarrow$  G.rc.getPaint()  
paintCap  $\leftarrow$  G.rc.getType().paintCapacity  
if paint < 35 and mode  $\neq$  RETREAT then  
    mode  $\leftarrow$  RETREAT  
endif  
if mode = RETREAT and paint  $\geq$  0.72 * paintCap then  
    mode  $\leftarrow$  EXPLORE  
endif  
if G.round > TOWER.BUILD.CUTOFF and mode  $\neq$  RETREAT and mode  $\neq$  ATTACK then  
    if mode  $\neq$  EXPLORE then
```

```
        call abandonBuild()
    endif
endif
if mode  $\neq$  RETREAT then
    switch mode
        case EXPLORE : call transitionExplore()
        case GOTO_RUIN : call transitionGotoRuin()
        case MARK : call transitionMark()
        case PAINT : call transitionPaint()
        case COMPLETE : call transitionComplete()
        case ATTACK : call transitionAttack()
    endswitch
endif
switch mode
    case EXPLORE : call doExplore()
    case GOTO_RUIN : call doGotoRuin()
    case MARK : call doMark()
    case PAINT : call doPaint()
    case COMPLETE : call doComplete()
    case RETREAT : call doRetreat()
    case ATTACK : call doAttack()
endswitch
append indicator
```

procedure transitionExplore()

Dari explore, tentukan apakah harus menyerang atau mulai membangun tower.

Kamus:

ruin : MapLocation

Algorithm

```
if G.round > TOWER_BUILD_CUTOFF then
    return
endif
```

```
    if G.lastKnownEnemyTower  $\neq$  null and G.round - G.lastEnemyTowerRound < 60 and  
    G.rc.getPaint() > 130 then  
        mode  $\leftarrow$  ATTACK  
        return  
    endif  
    ruin  $\leftarrow$  findBestRuinToBuild()  
    if ruin  $\neq$  null then  
        targetRuin  $\leftarrow$  ruin  
        targetType  $\leftarrow$  MapMemory.chooseTowerType()  
        markerPlaced  $\leftarrow$  false  
        buildStartRound  $\leftarrow$  G.round  
        mode  $\leftarrow$  GOTO_RUIN  
    endif
```

procedure transitionGotoRuin()

Periksa apakah target ruin masih valid dan apakah sudah cukup dekat untuk mark.

Kamus:

-

Algorithm

```
    if targetRuin = null then  
        call abandonBuild()  
        return  
    endif  
    if G.round - buildStartRound > BUILD_TIMEOUT then  
        call abandonBuild()  
        return  
    endif  
    call checkRuinStolen()  
    if mode  $\neq$  GOTO_RUIN then  
        return  
    endif  
    if G.rc.canSenseLocation(targetRuin) then  
        mode  $\leftarrow$  MARK
```

endif

procedure transitionMark()

Periksa apakah fase marking bisa lanjut.

Kamus:

-

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif

if G.round - buildStartRound > BUILD_TIMEOUT then
    call abandonBuild()
    return
endif

call checkRuinStolen()

if mode ≠ MARK then
    return
endif

if markerPlaced then
    mode ← PAINT
endif

endif
```

procedure transitionPaint()

Saat fase paint, cek apakah pattern sudah lengkap dan siap complete.

Kamus:

k : integer

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif

if G.round - buildStartRound > BUILD_TIMEOUT then
```

```
    call abandonBuild()
    return
endif
call checkRuinStolen()
if mode  $\neq$  PAINT then
    return
endif
if G.me.isWithinDistanceSquared(targetRuin, 20) then
    for k  $\leftarrow$  0 to length(TOWER_TYPES) - 1 do
        if G.rc.canCompleteTowerPattern(TOWER_TYPES[k], targetRuin) then
            targetType  $\leftarrow$  TOWER_TYPES[k]
            mode  $\leftarrow$  COMPLETE
            return
        endif
    endfor
endif
```

procedure transitionComplete()

Coba complete tower; jika gagal kembali ke fase paint.

Kamus:

-

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif
if not tryCompleteTower() then
    mode  $\leftarrow$  PAINT
endif
```

procedure transitionAttack()

Keluar dari attack jika info enemy tower sudah usang.

Kamus:

-

Algorithm

```
if G.lastKnownEnemyTower = null or G.round - G.lastEnemyTowerRound > 80 then
    mode  $\leftarrow$  EXPLORE
endif
```

procedure doExplore()

Mode eksplorasi umum Soldier.

Kamus:

-

Algorithm

```
if exploreTarget = null or G.me.isWithinDistanceSquared(exploreTarget, 4) or exploreAge >
EXPLORE_TIMEOUT then
    exploreTarget  $\leftarrow$  pickGreedyExploreTarget()
    exploreAge  $\leftarrow$  0
endif
exploreAge  $\leftarrow$  exploreAge + 1
if G.rc.isMovementReady() then
    call Nav.moveToward(exploreTarget)
endif
call paintNearby()
```

procedure doGotoRuin()

Bergerak menuju ruin target sambil tetap mengecat sekitar.

Kamus:

-

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif
if G.rc.isMovementReady() then
    call Nav.moveToward(targetRuin)
endif
```

call paintNearby()

procedure doMark()

Menempatkan marker tower pattern di ruin target.

Kamus:

-

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif

if not G.rc.canSenseLocation(targetRuin) then
    if G.rc.isMovementReady() then
        call Nav.moveToward(targetRuin)
    endif
    call paintNearby()
    return
endif

if not markerPlaced then
    if G.rc.getPaint() < 30 then
        call paintNearby()
        return
    endif
endif

if G.rc.canMarkTowerPattern(targetType, targetRuin) then
    G.rc.markTowerPattern(targetType, targetRuin)
    markerPlaced ← true
    mode ← PAINT
    return
endif

if G.rc.isMovementReady() and not G.me.isWithinDistanceSquared(targetRuin, 4) then
    call Nav.moveToward(targetRuin)
endif

return
```

```
endif  
mode ← PAINT
```

procedure doPaint()

Mengecat pattern tower seefisien mungkin.

Kamus:

urgent : MapLocation

Algorithm

```
if targetRuin = null then  
    call abandonBuild()  
    return  
endif  
if G.rc.isMovementReady() then  
    urgent ← findUrgentPatternTile()  
    if urgent ≠ null then  
        if not G.me.isWithinDistanceSquared(urgent, G.rc.getType().actionRadiusSquared) then  
            call Nav.moveToward(urgent)  
        endif  
    else  
        if not G.me.isWithinDistanceSquared(targetRuin, 4) then  
            call Nav.moveToward(targetRuin)  
        endif  
    endif  
endif  
if G.rc.isActionReady() then  
    if not paintBestPatternTile() then  
        call paintNearby()  
    endif  
endif
```

procedure doComplete()

Mendekat ke ruin lalu complete tower.

Kamus:

-

Algorithm

```
if targetRuin = null then
    call abandonBuild()
    return
endif
if not G.me.isWithinDistanceSquared(targetRuin, 2) then
    if G.rc.isMovementReady() then
        call Nav.moveToward(targetRuin)
    endif
    return
endif
call tryCompleteTower()
```

procedure doRetreat()

Mundur ke ally tower terdekat untuk refill paint.

Kamus:

tower : MapLocation
need : integer

Algorithm

```
tower ← findNearestAllyTowerLoc()
if tower = null then
    if G.rc.isMovementReady() then
        call Nav.moveToward(G.mapCenter)
    endif
    return
endif
if G.rc.isMovementReady() then
    call Nav.moveToward(tower)
endif
if G.me.isWithinDistanceSquared(tower, 2) then
    need ← G.rc.getType().paintCapacity - G.rc.getPaint()
    if need > 0 and G.rc.canTransferPaint(tower, -need) then
```

```
G.rc.transferPaint(tower, -need)

endif

endif
```

procedure doAttack()

Bergerak menuju enemy tower terakhir yang diketahui dan menyerangnya.

Kamus:

target : MapLocation

Algorithm

```
target ← G.lastKnownEnemyTower
if target = null then
    mode ← EXPLORE
    return
endif
if G.rc.isMovementReady() then
    call Nav.moveToward(target)
endif
if G.rc.isActionReady() then
    if G.me.isWithinDistanceSquared(target, G.rc.getType().actionRadiusSquared) then
        if G.rc.canAttack(target) then
            G.rc.attack(target)
        endif
    else
        call paintNearby()
    endif
endif
```

function paintBestPatternTile() → boolean

Memilih tile pattern paling prioritas untuk dicat.

Kamus:

bestTile : MapLocation

bestUseSecondary : boolean

bestPriority : integer

```
bestDist : integer
actionRad : integer
mopperNearby : boolean
dx, dy : integer
tile : MapLocation
info : MapInfo
mark : PaintType
paint : PaintType
needSecondary : boolean
priority : integer
dist : integer
```

Algorithm

```
if targetRuin = null then
    return false
endif

bestTile ← null
bestUseSecondary ← false
bestPriority ← -1
bestDist ←  $+\infty$ 

actionRad ← G.rc.getType().actionRadiusSquared
mopperNearby ← hasMopperNearby()

for dy ← -2 to 2 do
    for dx ← -2 to 2 do
        tile ← targetRuin.translate(dx, dy)

        if not G.me.isWithinDistanceSquared(tile, actionRad) then continue endif
        if not G.rc.canSenseLocation(tile) then continue endif
        if not G.rc.canAttack(tile) then continue endif

        info ← G.rc.senseMapInfo(tile)
        mark ← info.getMark()
        paint ← info.getPaint()

        if mark = EMPTY then continue endif

        needSecondary ← (mark = ALLY_SECONDARY)

        if needSecondary and paint = ALLY_SECONDARY then continue endif
```

```
    if not needSecondary and paint = ALLY_PRIMARY then continue endif
    if paint.isEnemy() then
        if mopperNearby and G.round - buildStartRound < 40 then
            priority ← 1
        else
            priority ← 2
        endif
    else
        priority ← 3
    endif
    dist ← G.me.distanceSquaredTo(tile)
    if priority > bestPriority or (priority = bestPriority and dist < bestDist) then
        bestPriority ← priority
        bestDist ← dist
        bestTile ← tile
        bestUseSecondary ← needSecondary
    endif
endfor
endif
if bestTile = null then
    return false
endif
if bestUseSecondary then
    if G.rc.canAttack(bestTile) then
        G.rc.attack(bestTile, true)
        return true
    endif
else
    if G.rc.canAttack(bestTile) then
        G.rc.attack(bestTile)
        return true
    endif
endif
endif
```

```
return false
```

function findUrgentPatternTile() → MapLocation

Mengembalikan tile pattern terdekat yang masih perlu dicat.

Kamus:

best : MapLocation
bestDist : integer
mopperNearby : boolean
dx, dy : integer
tile : MapLocation
info : MapInfo
mark : PaintType
paint : PaintType
needSecondary : boolean
d : integer

Algorithm

```
if targetRuin = null then
    return null
endif
best ← null
bestDist ←  $+\infty$ 
mopperNearby ← hasMopperNearby()
for dy ← -2 to 2 do
    for dx ← -2 to 2 do
        tile ← targetRuin.translate(dx, dy)
        if not G.rc.canSenseLocation(tile) then continue endif
        info ← G.rc.senseMapInfo(tile)
        mark ← info.getMark()
        paint ← info.getPaint()
        if mark = EMPTY then continue endif
        needSecondary ← (mark = ALLY_SECONDARY)
        if needSecondary and paint = ALLY_SECONDARY then continue endif
        if not needSecondary and paint = ALLY_PRIMARY then continue endif
```



```
    if paint.isEnemy() and mopperNearby and G.round - buildStartRound < 40 then
        continue
    endif
    d ← G.me.distanceSquaredTo(tile)
    if d < bestDist then
        bestDist ← d
        best ← tile
    endif
endfor
endfor
return best
```

function hasMopperNearby() → boolean

Mengecek apakah ada ally Mopper di sekitar.

Kamus:

i : integer

r : RobotInfo

Algorithm

```
for i ← length(G.nearbyAllies) - 1 downto 0 do
    r ← G.nearbyAllies[i]
    if r.type = MOPPER and G.me.isWithinDistanceSquared(r.location, 8) then
        return true
    endif
endfor
return false
```

procedure opportunisticComplete()

Menyelesaikan pattern tower siap-complete di ruin sekitar meskipun bukan target utama.

Kamus:

i, k : integer

ruin : MapLocation

ri : RobotInfo

sentinel : MapLocation

Algorithm

```
for i ← length(G.nearbyRuins) - 1 downto 0 do
  ruin ← G.nearbyRuins[i]
  if G.rc.canSenseRobotAtLocation(ruin) then
    ri ← G.rc.senseRobotAtLocation(ruin)
    if ri ≠ null then
      continue
    endif
  endif
  for k ← 0 to length(TOWER_TYPES) - 1 do
    if not G.rc.canCompleteTowerPattern(TOWER_TYPES[k], ruin) then
      continue
    endif
    sentinel ← ruin.add(SENTINEL[k])
    if not G.rc.canSenseLocation(sentinel) then
      continue
    endif
    if G.rc.senseMapInfo(sentinel).getMark() = EMPTY then
      continue
    endif
    G.rc.completeTowerPattern(TOWER_TYPES[k], ruin)
    call MapMemory.registerRuin(ruin, G.team, TOWER_TYPES[k])
    if ruin = targetRuin then
      call abandonBuild()
    endif
    break
  endfor
endfor
```

function tryCompleteTower() → boolean

Mencoba complete tower pada targetRuin.

Kamus:

k : integer

sentinel : MapLocation

Algorithm

if targetRuin = null then

 return false

endif

for k ← 0 to length(TOWER_TYPES) - 1 do

 if not G.rc.canCompleteTowerPattern(TOWER_TYPES[k], targetRuin) then

 continue

 endif

 sentinel ← targetRuin.add(SENTINEL[k])

 if G.rc.canSenseLocation(sentinel) then

 if G.rc.senseMapInfo(sentinel).getMark() = EMPTY then

 continue

 endif

 endif

 G.rc.completeTowerPattern(TOWER_TYPES[k], targetRuin)

 call MapMemory.registerRuin(targetRuin, G.team, TOWER_TYPES[k])

 call abandonBuild()

 return true

endfor

return false

function findBestRuinToBuild() → MapLocation

Memilih ruin terbaik secara greedy untuk dibangun.

Kamus:

best : MapLocation

bestScore : integer

ruin : MapLocation

ri : RobotInfo

enemyPaint : integer

score : integer

i : integer

Algorithm

```
best ← null
bestScore ←  $-\infty$ 
for i ← length(G.nearbyRuins)-1 downto 0 do
  ruin ← G.nearbyRuins[i]
  if G.rc.canSenseRobotAtLocation(ruin) then
    ri ← G.rc.senseRobotAtLocation(ruin)
    if ri ≠ null then
      continue
    endif
  endif
  enemyPaint ← countEnemyPaint(ruin, 12)
  if enemyPaint > 12 then
    continue
  endif
  score ← 1500 - G.me.distanceSquaredTo(ruin) - enemyPaint * 12
  if score > bestScore then
    bestScore ← score
    best ← ruin
  endif
endfor
for i ← MapMemory.numRuins - 1 downto 0 do
  if MapMemory.ruinStatus[i] ≠ 0 then continue endif
  if G.round - MapMemory.ruinClaimRound[i] < 100 then continue endif
  score ← 1000 - G.me.distanceSquaredTo(MapMemory.ruinLocs[i])
  if score > bestScore then
    bestScore ← score
    best ← MapMemory.ruinLocs[i]
  endif
endfor
return best
```

```
function countEnemyPaint(loc : MapLocation, radiusSq : integer) → integer
```

Menghitung jumlah enemy paint di sekitar loc.

Kamus:

count : integer

area : array of MapInfo

j : integer

Algorithm

count ← 0

area ← G.rc.senseNearbyMapInfos(loc, radiusSq)

for j ← length(area)-1 downto 0 do

 if area[j].getPaint().isEnemy() then

 count ← count + 1

 endif

endfor

return count

```
function pickGreedyExploreTarget() → MapLocation
```

Memilih target eksplorasi terbaik berdasarkan ruin netral dan random sampling.

Kamus:

best : MapLocation

bestScore : integer

i, t : integer

ruin, c : MapLocation

v, dist : integer

score : integer

Algorithm

best ← null

bestScore ← -1

if G.round ≤ TOWER_BUILD_CUTOFF then

 for i ← MapMemory.numRuins-1 downto 0 do

 if MapMemory.ruinStatus[i] ≠ 0 then continue endif

 ruin ← MapMemory.ruinLocs[i]

 v ← G.getVisited(ruin)

```
dist ← G.me.distanceSquaredTo(ruin)
score ← 250 - (if v = 0 then 0 else min(80, G.round - v)) - dist div 6
if score > bestScore then
    bestScore ← score
    best ← ruin
endif
endfor
endif
for t ← 0 to 7 do
    c ← MapLocation(Rand.nextInt(G.mapWidth), Rand.nextInt(G.mapHeight))
    v ← G.getVisited(c)
    dist ← G.me.distanceSquaredTo(c)
    score ← (if v = 0 then 160 else max(0, G.round - v - 30)) + dist div 8
    if score > bestScore then
        bestScore ← score
        best ← c
    endif
endfor
if best ≠ null then
    return best
endif
return MapLocation(Rand.nextInt(G.mapWidth), Rand.nextInt(G.mapHeight))
```

procedure paintNearby()

Memilih tile sekitar terbaik untuk dicat demi ekspansi territory.

Kamus:

best : MapLocation
bestScore : integer
info : MapInfo
loc : MapLocation
paint : PaintType
score : integer

Algorithm

```
if not G.rc.isActionReady() then
    return
endif
best ← null
bestScore ← 0
for each info in G.nearbyMapInfos do
    loc ← info.getMapLocation()
    if not G.me.isWithinDistanceSquared(loc, G.rc.getType().actionRadiusSquared) then continue
endif
    if not G.rc.canAttack(loc) then continue endif
    if info.isWall() or info.hasRuin() then continue endif
    paint ← info.getPaint()
    if paint = EMPTY then
        if loc = G.me then
            score ← 120
        else
            score ← 100
        endif
    else if paint.isEnemy() then
        score ← 80
    else
        continue
    endif
    score ← score - G.me.distanceSquaredTo(loc)
    if score > bestScore then
        bestScore ← score
        best ← loc
    endif
endfor
if best ≠ null then
    G.rc.attack(best)
endif
```

procedure tryCompleteResourcePatterns()

Menyelesaikan resource pattern di sekitar posisi Soldier.

Kamus:

dx, dy : integer

loc : MapLocation

Algorithm

```
for dx ← -2 to 2 do
  for dy ← -2 to 2 do
    loc ← G.me.translate(dx, dy)
    if G.rc.canCompleteResourcePattern(loc) then
      G.rc.completeResourcePattern(loc)
    endif
  endfor
endfor
```

procedure tryWithdrawPaint()

Mengambil paint dari nearby ally paint tower jika paint belum hampir penuh.

Kamus:

i : integer

ally : RobotInfo

need : integer

Algorithm

```
if G.rc.getPaint() ≥ 0.88 * G.rc.getType().paintCapacity then
  return
endif
if not G.rc.isActionReady() then
  return
endif
for i ← length(G.nearbyAllies)-1 downto 0 do
  ally ← G.nearbyAllies[i]
  if ally.type.isRobotType() then continue endif
  if ally.type.getBaseType() ≠ LEVEL.ONE_PAINT_TOWER then continue endif
  if not G.me.isWithinDistanceSquared(ally.location, 2) then continue endif
```



```
need ← G.rc.getType().paintCapacity - G.rc.getPaint()
if need > 0 and G.rc.canTransferPaint(ally.location, -need) then
    G.rc.transferPaint(ally.location, -need)
return
endif
endfor
```

function findNearestAllyTowerLoc() → MapLocation

Mengembalikan ally tower terdekat, dengan prioritas lebih tinggi untuk paint tower.

Kamus:

best : MapLocation
bestDist : integer
isPaint : boolean
d, i : integer

Algorithm

```
best ← null
bestDist ← +∞
for i ← MapMemory.numRuins-1 downto 0 do
    if MapMemory.ruinTeams[i] ≠ G.team or MapMemory.ruinTypes[i] = null then
        continue
    endif
    isPaint ← (MapMemory.ruinTypes[i].getBaseType() = LEVEL_ONE_PAINT_TOWER)
    d ← G.me.distanceSquaredTo(MapMemory.ruinLocs[i]) + (if isPaint then 0 else 400)
    if d < bestDist then
        bestDist ← d
        best ← MapMemory.ruinLocs[i]
    endif
endfor
return best
```

procedure checkRuinStolen()

Mengecek apakah target ruin sudah diambil musuh atau ally.

Kamus:

ri : RobotInfo

Algorithm

```
if targetRuin = null or not G.rc.canSenseLocation(targetRuin) then
    return
endif
if not G.rc.canSenseRobotAtLocation(targetRuin) then
    return
endif
ri ← G.rc.senseRobotAtLocation(targetRuin)
if ri = null then
    return
endif
if ri.team = G.opponentTeam then
    G.lastKnownEnemyTower ← targetRuin
    G.lastEnemyTowerRound ← G.round
    call abandonBuild()
    mode ← ATTACK
else if ri.team = G.team then
    call MapMemory.registerRuin(targetRuin, G.team, ri.type)
    call abandonBuild()
endif
```

procedure abandonBuild()

Menghapus state pembangunan tower dan kembali ke EXPLORE.

Kamus:

-

Algorithm

```
targetRuin ← null
targetType ← null
markerPlaced ← false
mode ← EXPLORE
```

function modeStr() → string

Mengubah mode Soldier menjadi string singkat.

Kamus:

-

Algorithm

```
switch mode
  case EXPLORE : return "EXP"
  case GOTO_RUIN : return "GTO"
  case MARK : return "MRK"
  case PAINT : return "PNT"
  case COMPLETE : return "CMP"
  case RETREAT : return "RET"
  case ATTACK : return "ATK"
  default : return "?"
endswitch
```

Splasher.java

procedure init()

Inisialisasi seed random Splasher.

Kamus:

-

Algorithm

```
Rand.state  $\leftarrow$  G.rc.getID() * 7654321 + 13
```

procedure run()

Prosedur utama Splasher tiap round.

Kamus:

paint : integer

cap : integer

Algorithm

```
call tryWithdrawPaint()
paint  $\leftarrow$  G.rc.getPaint()
cap  $\leftarrow$  G.rc.getType().paintCapacity
if paint < 50 and mode  $\neq$  RETREAT then
  mode  $\leftarrow$  RETREAT
```

```
endif
if mode = RETREAT and paint  $\geq$  0.75 * cap then
    mode  $\leftarrow$  EXPLORE
endif
call tryCompletePatterns()
switch mode
    case EXPLORE : call doExplore()
    case RETREAT : call doRetreat()
endswitch
append indicator
```

procedure doExplore()

Explore lalu lakukan splash attack terbaik.

Kamus:

moveTarget : MapLocation

Algorithm

```
if G.rc.isMovementReady() then
    moveTarget  $\leftarrow$  pickMoveTarget()
    if moveTarget  $\neq$  null then
        call Nav.moveToward(moveTarget)
    endif
endif
if G.rc.isActionReady() then
    call bestSplash()
endif
```

procedure bestSplash()

Memilih pusat splash dengan skor area terbaik.

Kamus:

bestLoc : MapLocation

bestScore : integer

actionRadius : integer

info, tile : MapInfo

```
center : MapLocation
score : integer
paint : PaintType
tileLoc : MapLocation
```

Algorithm

```
bestLoc ← null
bestScore ← 0
actionRadius ← G.rc.getType().actionRadiusSquared
for each info in G.nearbyMapInfos do
    center ← info.getMapLocation()
    if not G.me.isWithinDistanceSquared(center, actionRadius) then continue endif
    if not G.rc.canAttack(center) then continue endif
    score ← 0
    for each tile in G.nearbyMapInfos do
        tileLoc ← tile.getMapLocation()
        if not center.isWithinDistanceSquared(tileLoc, 2) then continue endif
        paint ← tile.getPaint()
        if paint = EMPTY then
            score ← score + 10
        else if paint.isEnemy() then
            score ← score + 15
        endif
    endfor
    if G.getVisited(center) = 0 then
        score ← score + 12
    endif
    for each tile in G.nearbyMapInfos do
        tileLoc ← tile.getMapLocation()
        if not center.isWithinDistanceSquared(tileLoc, 2) then continue endif
        if G.getVisited(tileLoc) = 0 then
            score ← score + 3
        endif
    endfor
endfor
```

```
    if score > bestScore then
        bestScore ← score
        bestLoc ← center
    endif
endfor
if bestLoc ≠ null then
    G.rc.attack(bestLoc)
endif
```

function pickMoveTarget() → MapLocation

Memilih target gerak berdasarkan target explore dan scoring local next step.

Kamus:

bestDir : Direction
bestScore : integer
d : Direction
next : MapLocation
score : integer
v : integer
i : integer

Algorithm

```
    if exploreTarget = null or G.me.isWithinDistanceSquared(exploreTarget, 4) or exploreAge >
EXPLORE_TIMEOUT then
        exploreTarget ← pickGreedyTarget()
        exploreAge ← 0
    endif
    exploreAge ← exploreAge + 1
    bestDir ← null
    bestScore ← -∞
    for each d in G.DIRECTIONS do
        if not G.rc.canMove(d) then
            continue
        endif
        next ← G.me.add(d)
```

```
    score ← 0
    score ← score + (G.me.distanceSquaredTo(exploreTarget) -
next.distanceSquaredTo(exploreTarget)) * 5
    v ← G.getVisited(next)
    if v = 0 then
        score ← score + 30
    else
        score ← score - (G.round - v) div 5
    endif
    for i ← length(G.nearbyAllies)-1 downto 0 do
        if G.nearbyAllies[i].type.isRobotType() and next.isWithinDistanceSquared(G.nearbyAllies[i].location,
2) then
            score ← score - 20
        endif
    endfor
    if score > bestScore then
        bestScore ← score
        bestDir ← d
    endif
endfor
if bestDir ≠ null then
    return G.me.add(bestDir)
endif
return exploreTarget
```

function pickGreedyTarget() → MapLocation

Memilih target eksplorasi acak yang paling menjanjikan.

Kamus:

best : MapLocation

bestScore : integer

attempt : integer

x, y : integer

c : MapLocation

v, dist : integer

score : integer

Algorithm

best $\leftarrow$ null

bestScore $\leftarrow$ -1

for attempt $\leftarrow$ 0 to 7 do

 x $\leftarrow$ Rand.nextInt(G.mapWidth)

 y $\leftarrow$ Rand.nextInt(G.mapHeight)

 c $\leftarrow$ MapLocation(x, y)

 v $\leftarrow$ G.getVisited(c)

 dist $\leftarrow$ G.me.distanceSquaredTo(c)

 score $\leftarrow$ (if v = 0 then 100 else max(0, G.round - v - 20)) + dist div 10

 if score > bestScore then

 bestScore $\leftarrow$ score

 best $\leftarrow$ c

 endif

endfor

if best $\neq$ null then

 return best

endif

return MapLocation(Rand.nextInt(G.mapWidth), Rand.nextInt(G.mapHeight))

procedure doRetreat()

Mundur ke paint tower terdekat.

Kamus:

tower : MapLocation

need : integer

Algorithm

tower $\leftarrow$ findNearestPaintTowerLoc()

if tower = null then

 mode $\leftarrow$ EXPLORE

 return

endif


```
if G.rc.isMovementReady() then
    call Nav.moveToward(tower)
endif
if G.me.isWithinDistanceSquared(tower, 2) then
    need ← G.rc.getType().paintCapacity - G.rc.getPaint()
    if need > 0 and G.rc.canTransferPaint(tower, -need) then
        G.rc.transferPaint(tower, -need)
    endif
endif
endif
```

procedure tryWithdrawPaint()

Mengambil paint dari nearby ally paint tower jika belum hampir penuh.

Kamus:

i : integer
ally : RobotInfo
need : integer

Algorithm

```
if G.rc.getPaint() ≥ 0.9 * G.rc.getType().paintCapacity then
    return
endif
if not G.rc.isActionReady() then
    return
endif
for i ← length(G.nearbyAllies)-1 downto 0 do
    ally ← G.nearbyAllies[i]
    if ally.type.isRobotType() then continue endif
    if ally.type.getBaseType() ≠ LEVEL.ONE_PAINT_TOWER then continue endif
    if not G.me.isWithinDistanceSquared(ally.location, 2) then continue endif
    need ← G.rc.getType().paintCapacity - G.rc.getPaint()
    if G.rc.canTransferPaint(ally.location, -need) then
        G.rc.transferPaint(ally.location, -need)
        return
    endif
endif
```

```
endfor
```

procedure tryCompletePatterns()

Menyelesaikan tower pattern dan resource pattern yang siap.

Kamus:

i, dx, dy : integer

ruin : MapLocation

tt : UnitType

loc : MapLocation

Algorithm

```
for i ← length(G.nearbyRuins)-1 downto 0 do
  ruin ← G.nearbyRuins[i]
  for each tt in Soldier.TOWER_TYPES do
    if G.rc.canCompleteTowerPattern(tt, ruin) then
      G.rc.completeTowerPattern(tt, ruin)
      call MapMemory.registerRuin(ruin, G.team, tt)
      break
    endif
  endfor
endfor
for dx ← -2 to 2 do
  for dy ← -2 to 2 do
    loc ← G.me.translate(dx, dy)
    if G.rc.canCompleteResourcePattern(loc) then
      G.rc.completeResourcePattern(loc)
    endif
  endfor
endfor
```

function findNearestPaintTowerLoc() → MapLocation

Mengembalikan ally paint tower terdekat.

Kamus:

best : MapLocation

```
bestDist : integer
```

```
d, i : integer
```

Algorithm

```
best ← null
```

```
bestDist ←  $+\infty$ 
```

```
for i ← MapMemory.numRuins-1 downto 0 do
```

```
  if MapMemory.ruinTeams[i] = G.team and MapMemory.ruinTypes[i] ≠ null then
```

```
    if MapMemory.ruinTypes[i].getBaseType() = LEVEL_ONE_PAINT_TOWER then
```

```
      d ← G.me.distanceSquaredTo(MapMemory.ruinLocs[i])
```

```
      if d < bestDist then
```

```
        bestDist ← d
```

```
        best ← MapMemory.ruinLocs[i]
```

```
      endif
```

```
    endif
```

```
  endif
```

```
endfor
```

```
return best
```

Tower.java**procedure init()**

Inisialisasi daftar lokasi spawn dan mendaftarkan tower ini ke MapMemory.

Kamus:

-

Algorithm

```
spawnLocs ← buildSpawnLocs()
```

```
call MapMemory.registerRuin(G.me, G.team, G.rc.getType())
```

procedure run()

Logic utama Tower setiap round.

Kamus:

```
isPhase2 : boolean
```

```
toSpawn : UnitType
```

shouldSpawnNow : boolean

Algorithm

```
call MapMemory.scanNearby()
isPhase2  $\leftarrow$  (G.round  $\geq$  PHASE2_ROUND)
if isPhase2 then
    toSpawn  $\leftarrow$  decideSpawnPhase2()
else
    toSpawn  $\leftarrow$  decideSpawnPhase1()
endif
shouldSpawnNow  $\leftarrow$  shouldSpawn(toSpawn, isPhase2)
if shouldSpawnNow and G.rc.isActionReady() then
    if trySpawn(toSpawn) then
        lastSpawnRound  $\leftarrow$  G.round
    endif
endif
call doAttack()
call tryUpgrade()
append indicator
```

function decideSpawnPhase1() $\rightarrow$ UnitType

Memilih tipe robot untuk fase awal secara greedy berdasarkan demand berbot.

Kamus:

wS, wSp, wM : real
sum : real
scoreS, scoreSp, scoreM : real

Algorithm

```
if G.round < 50 and spawnedTotal < 5 then
    return SOLDIER
endif
wS  $\leftarrow$  W_SOLDIER_BASE
wSp  $\leftarrow$  W_SPLASHER_BASE
wM  $\leftarrow$  W_MOPPER_BASE
wSp  $\leftarrow$  wSp + MapMemory.allyPaintTowers * 0.35
```

```
wS ← wS - min(1.0, G.rc.getNumberTowers() * 0.04)
if MapMemory.allyMoneyTowers ≥ 2 then
    wSp ← wSp + 0.4
endif
sum ← max(0.01, wS + wSp + wM)
wS ← wS / sum
wSp ← wSp / sum
wM ← wM / sum
scoreS ← dSoldier + wS - spawnedSoldiers
scoreSp ← dSplasher + wSp - spawnedSplashes
scoreM ← dMopper + wM - spawnedMoppers
if scoreS ≥ scoreSp and scoreS ≥ scoreM then
    return SOLDIER
endif
if scoreSp ≥ scoreM then
    return SPLASHER
endif
return MOPPER
```

function decideSpawnPhase2() → UnitType

Fase akhir: mayoritas spawn Splasher, sesekali Mopper.

Kamus:

-

Algorithm

```
if (spawnedTotal mod 7) = 6 then
    return MOPPER
endif
return SPLASHER
```

function shouldSpawn(toSpawn : UnitType, isPhase2 : boolean) → boolean

Menentukan apakah tower layak spawn sekarang.

Kamus:

chips : integer

towers : integer

cost : integer

Algorithm

chips $\leftarrow$ G.rc.getMoney()

towers $\leftarrow$ G.rc.getNumberTowers()

cost $\leftarrow$ toSpawn.moneyCost

if G.round < 10 then

 return true

endif

if towers $\geq$ 25 then

 return (chips $\geq$ cost)

endif

if chips < cost then

 return false

endif

if isPhase2 then

 return true

endif

if chips - cost $\geq$ 600 then return true endif

if G.round < 80 then return true endif

if lastSpawnRound + 2 < G.round and length(G.nearbyAllies) < 3 then return true endif

return false

function trySpawn(type : UnitType) $\rightarrow$ boolean

Mencoba spawn robot pada lokasi spawn pertama yang valid.

Kamus:

loc : MapLocation

Algorithm

for each loc in spawnLocs do

 if G.rc.canBuildRobot(type, loc) then

 G.rc.buildRobot(type, loc)

 spawnedTotal $\leftarrow$ spawnedTotal + 1

 switch type

```
        case SOLDIER : spawnedSoldiers  $\leftarrow$  spawnedSoldiers + 1 ; dSoldier  $\leftarrow$  dSoldier + 1
        case SPLASHER : spawnedSplashes  $\leftarrow$  spawnedSplashes + 1 ; dSplasher  $\leftarrow$  dSplasher +
1
        case MOPPER : spawnedMoppers  $\leftarrow$  spawnedMoppers + 1 ; dMopper  $\leftarrow$  dMopper + 1
    endswitch
    return true
endif
endfor
return false
```

procedure doAttack()

Menyerang area dengan AoE lalu memilih target tunggal terbaik.

Kamus:

bestTarget : MapLocation

bestScore : integer

atkStr : integer

i : integer

r : RobotInfo

score : integer

Algorithm

```
if G.rc.canAttack(null) then
    G.rc.attack(null)
endif
bestTarget  $\leftarrow$  null
bestScore  $\leftarrow$   $-\infty$ 
atkStr  $\leftarrow$  G.rc.getType().attackStrength
for i  $\leftarrow$  length(G.nearbyEnemies)-1 downto 0 do
    r  $\leftarrow$  G.nearbyEnemies[i]
    if not G.me.isWithinDistanceSquared(r.location, G.rc.getType().actionRadiusSquared) then
continue endif
    if not G.rc.canSenseRobotAtLocation(r.location) then continue endif
    if r.health  $\leq$  atkStr then
        score  $\leftarrow$  10000 - r.health
    else
```

```
        score ← 1000 - r.health
    endif
    if score > bestScore then
        bestScore ← score
        bestTarget ← r.location
    endif
endfor
if bestTarget ≠ null and G.rc.canAttack(bestTarget) then
    G.rc.attack(bestTarget)
endif
```

procedure tryUpgrade()

Upgrade tower bila uang cukup dan buffer aman.

Kamus:

upgradeCost : integer

buffer : integer

Algorithm

```
if towerLevel = 0 then
    upgradeCost ← 2500
else
    upgradeCost ← 5000
endif
if G.round ≥ PHASE2.ROUND then
    buffer ← 400
else
    buffer ← 1000
endif
if G.rc.getMoney() - upgradeCost ≥ buffer and G.rc.canUpgradeTower(G.me) then
    G.rc.upgradeTower(G.me)
    towerLevel ← towerLevel + 1
endif
```

function buildSpawnLocs() → array of MapLocation

Membangun daftar lokasi spawn di sekitar tower lalu mengurutkannya relatif terhadap pusat peta.

Kamus:

locs : array of MapLocation

center : MapLocation

Algorithm

```
locs ← {  
    G.me + NORTH,  
    G.me + NORTHEAST,  
    G.me + EAST,  
    G.me + SOUTHEAST,  
    G.me + SOUTH,  
    G.me + SOUTHWEST,  
    G.me + WEST,  
    G.me + NORTHWEST,  
    G.me + NORTH + NORTH,  
    G.me + EAST + EAST,  
    G.me + SOUTH + SOUTH,  
    G.me + WEST + WEST  
}  
center ← G.mapCenter  
sort locs descending by distanceSquaredTo(center)  
return locs
```

4.1.2 Algoritma Solusi Greedy 2

a. RobotPlayer.java

```
procedure run(rc : RobotController)  
{ Menjalankan robot utama dan memanggil perilaku sesuai tipe robot. }
```

Kamus:

robotType : RobotType

turnCount : integer

Algorithm

```
print "I'm alive"  
rc.setIndicatorString("Hello world")
```

```
while robot is alive do
    turnCount ← turnCount + 1
    try
        robotType ← rc.getType()
        if robotType = SOLDIER then
            Soldier.runSoldier(rc)
        else if robotType = MOPPER then
            Mopper.runMopper(rc)
        else if robotType = SPLASHER then
            Splasher.runSplasher(rc)
        else
            Tower.runTower(rc)
        endif
    catch GameActionException
        print "GameActionException"
    catch Exception
        print "Exception"
    finally
        Clock.yield()
endwhile
```

b. Soldier.java

```
procedure runSoldier(rc : RobotController)
{ Mengatur perilaku soldier berdasarkan fase permainan. }
Kamus:
    visible : RobotInfo
    round : integer
Algorithm
    Comms.startTurn(rc)
    captureSpawnTower(rc)
    if Comms.enemyTowerLoc ≠ null then
        knownEnemyTower ← Comms.enemyTowerLoc
    endif
```

```
visible ← findVisibleEnemyTower(rc)
if visible ≠ null then
    knownEnemyTower ← visible.getLocation()
    Comms.reportEnemyTower(rc, knownEnemyTower)
endif
round ← rc.getRoundNum()
if round ≤ 200 then
    runEarly(rc)
else if round ≤ 1000 then
    runMid(rc)
else
    runLate(rc)
endif
```

procedure runEarly(rc)

{ Strategi eksplorasi awal dan penyerangan tower musuh. }

Algorithm

```
if tryAttackEnemyTower(rc) then return endif
if knownEnemyTower ≠ null then
    target ← knownEnemyTower
else
    target ← getSymmetryTarget(rc)
endif
if target ≠ null then
    dir ← Motion.moveTowardLocation(rc, target)
    if dir ≠ null and rc.canMove(dir) then
        rc.move(dir)
        lastDir ← dir
    endif
endif
tryAttackEnemyTower(rc)
paintCurrentTileIfNeeded(rc)
```

procedure runMid(rc)

{ Fase tengah permainan dengan fokus pada tower musuh. }

Algorithm

```
if tryAttackEnemyTower(rc) then return endif
if knownEnemyTower  $\neq$  null then
    dir  $\leftarrow$  Motion.moveTowardLocation(rc, knownEnemyTower)
    if dir  $\neq$  null and rc.canMove(dir) then
        rc.move(dir)
        lastDir  $\leftarrow$  dir
    endif
    tryAttackEnemyTower(rc)
else
    runEarly(rc)
    return
endif
paintCurrentTileIfNeeded(rc)
```

procedure runLate(rc)

{ Fase akhir permainan dengan pembangunan tower dari ruin. }

Algorithm

```
if tryAttackEnemyTower(rc) then return endif
curRuin  $\leftarrow$  findNearbyRuin(rc)
if curRuin  $\neq$  null then
    buildRuin(rc, curRuin.getMapLocation())
    return
endif
dir  $\leftarrow$  Motion.moveTowardEnemyPaint(rc)
if dir  $\neq$  null and rc.canMove(dir) then
    rc.move(dir)
    lastDir  $\leftarrow$  dir
endif
for each enemy in rc.senseNearbyRobots(-1, rc.getTeam().opponent()) do
    if rc.canAttack(enemy.getLocation()) then
```

```
        rc.attack(enemy.getLocation())
    if isTowerType(enemy.getType()) then
        Comms.reportEnemyTower(rc, enemy.getLocation())
    endif
    break
endif
endfor
paintCurrentTileIfNeeded(rc)
function findNearbyRuin(rc) → MapInfo
Algorithm
    for each tile in rc.senseNearbyMapInfos() do
        if tile.hasRuin() then
            return tile
        endif
    endfor
    return null
```

c. Splasher.java

```
procedure runSplasher(rc : RobotController)
{ Mengatur perilaku unit splasher berdasarkan fase permainan. }
Kamus:
    round : integer
Algorithm
    Comms.startTurn(rc)
    if Comms.enemyTowerLoc ≠ null then
        knownEnemyTower ← Comms.enemyTowerLoc
    endif
    round ← rc.getRoundNum()
    if round ≤ 600 then
        runEarly(rc)
    else if round ≤ 1000 then
        runMid(rc)
    else
```

```
runLate(rc)
endif
```

procedure runEarly(rc)

{ Fase awal splasher bergerak menuju tower musuh atau pusat map. }

Algorithm

```
if knownEnemyTower  $\neq$  null then
    dir  $\leftarrow$  Motion.moveToLocation(rc, knownEnemyTower)
else
    dir  $\leftarrow$  Motion.moveToNearestEnemyTower(rc)
endif
if dir = null then
    dir  $\leftarrow$  Motion.moveToCenter(rc)
endif
if dir  $\neq$  null and rc.canMove(dir) then
    rc.move(dir)
endif
if rc.canAttack(rc.getLocation()) then
    rc.attack(rc.getLocation())
endif
Soldier.paintCurrentTileIfNeeded(rc)
```

procedure runMid(rc)

{ Fase tengah permainan splasher fokus mendekati tower musuh. }

Algorithm

```
if knownEnemyTower  $\neq$  null then
    dir  $\leftarrow$  Motion.moveToLocation(rc, knownEnemyTower)
else
    dir  $\leftarrow$  Motion.moveToNearestEnemyTower(rc)
endif
if dir = null then
    dir  $\leftarrow$  Motion.moveToCorner(rc)
endif
```

```
if dir  $\neq$  null and rc.canMove(dir) then
    rc.move(dir)
endif
if Soldier.tryAttackEnemyTower(rc) then return endif
Soldier.tryAttackEnemyTower(rc)
Soldier.paintCurrentTileIfNeeded(rc)
```

procedure runLate(rc)

{ Fase akhir permainan dengan fokus menyerang area musuh. }

Algorithm

```
if knownEnemyTower  $\neq$  null then
    dir  $\leftarrow$  Motion.moveTowardLocation(rc, knownEnemyTower)
else
    dir  $\leftarrow$  Motion.moveTowardNearestEnemyTower(rc)
endif
if Soldier.tryAttackEnemyTower(rc) then return endif
if dir = null then
    dir  $\leftarrow$  Motion.moveTowardCorner(rc)
endif
if dir  $\neq$  null and rc.canMove(dir) then
    rc.move(dir)
endif
if dir  $\neq$  null then
    ahead  $\leftarrow$  rc.getLocation().add(dir)
else
    ahead  $\leftarrow$  rc.getLocation()
endif
dirInfo  $\leftarrow$  rc.senseMapInfo(ahead)
if dirInfo.getPaint() = ENEMY_PRIMARY
    or dirInfo.getPaint() = ENEMY_SECONDARY
    or dirInfo.getPaint() = EMPTY then
    if rc.canAttack(rc.getLocation()) then
        rc.attack(rc.getLocation())
```

```
endif  
endif
```

d. Mopper.java

```
procedure runMopper(rc : RobotController)  
{ Mengatur perilaku unit mopper berdasarkan fase permainan. }  
Kamus:  
    round : integer  
Algorithm  
    Comms.startTurn(rc)  
    round  $\leftarrow$  rc.getRoundNum()  
    if round  $\leq$  600 then  
        Soldier.runEarly(rc)  
    else if round  $\leq$  1000 then  
        runMid(rc)  
    else  
        runLate(rc)  
    endif
```

```
procedure runMid(rc)  
{ Fase tengah permainan dengan fokus mengejar musuh terdekat. }  
Kamus:  
    enemies : RobotInfo array  
    nearestEnemy : MapLocation  
    closestDist : integer  
    dist : integer  
    dir : Direction  
    nextLoc : MapLocation  
Algorithm  
    enemies  $\leftarrow$  rc.senseNearbyRobots(-1, opponent team)  
    nearestEnemy  $\leftarrow$  null  
    closestDist  $\leftarrow \infty$ 
```



```
for each e in enemies do
    dist ← distanceSquared(rc location, e location)
    if dist < closestDist then
        closestDist ← dist
        nearestEnemy ← e location
    endif
endfor
if nearestEnemy ≠ null then
    dir ← Motion.moveTowardLocation(rc, nearestEnemy)
else
    dir ← Motion.moveTowardNearestAllySoldier(rc)
    if dir = null then
        dir ← Motion.moveTowardEnemyPaint(rc)
    endif
endif
nextLoc ← null
if dir ≠ null then
    nextLoc ← rc.getLocation().add(dir)
    if rc.canMove(dir) then
        rc.move(dir)
    endif
endif
if dir ≠ null and rc.canMopSwing(dir) then
    rc.mopSwing(dir)
else if nextLoc ≠ null and rc.canAttack(nextLoc) then
    rc.attack(nextLoc)
endif
```

procedure runLate(rc)

{ Fase akhir permainan dengan fokus membersihkan area musuh. }

Kamus:

dir : Direction

nextLoc : MapLocation

Algorithm

```
dir ← Motion.moveTowardNearestAllySoldier(rc)
if dir = null then
    dir ← Motion.moveTowardEnemyPaint(rc)
endif
nextLoc ← null
if dir ≠ null then
    nextLoc ← rc.getLocation().add(dir)
    if rc.canMove(dir) then
        rc.move(dir)
    endif
endif
if dir ≠ null and rc.canMopSwing(dir) then
    rc.mopSwing(dir)
else if nextLoc ≠ null and rc.canAttack(nextLoc) then
    rc.attack(nextLoc)
endif
```

e. Tower.java

procedure runTower(rc : RobotController)

{ Mengatur perilaku tower dalam menyerang musuh dan memproduksi robot. }

Kamus:

```
round : integer
mod : integer
nextType : UnitType
allowFallbackSoldier : boolean
messages : Message*array
```

Algorithm

```
Comms.startTurn(rc)
round ← rc.getRoundNum()
if there are nearby enemy robots then
    tryAttackEnemies(rc)
```

```
    trySpawn(rc, MOPPER, false)
endif
if round  $\leq$  300 then
    if buildCount mod 3 = 1 then
        trySpawn(rc, SOLDIER, true)
    else
        trySpawn(rc, SOLDIER, false)
    endif
    trySpawn(rc, SOLDIER, true)
else if round  $\leq$  1000 then
    mod  $\leftarrow$  buildCount mod 2
    if mod = 0 then
        nextType  $\leftarrow$  SPLASHER
    else
        nextType  $\leftarrow$  SOLDIER
    endif
    allowFallbackSoldier  $\leftarrow$  nextType  $\neq$  SPLASHER
    trySpawn(rc, nextType, allowFallbackSoldier)
else
    mod  $\leftarrow$  buildCount mod 2
    if mod = 0 then
        nextType  $\leftarrow$  SPLASHER
    else
        nextType  $\leftarrow$  MOPPER
    endif
    allowFallbackSoldier  $\leftarrow$  nextType  $\neq$  SPLASHER
    trySpawn(rc, nextType, allowFallbackSoldier)
endif
messages  $\leftarrow$  rc.readMessages(all)
for each m in messages do
    print message sender and content
endfor
```

```
function getBestSpawnLocation(rc, unitType) → MapLocation
```

```
{ Menentukan lokasi spawn terbaik berdasarkan posisi robot lain dan pusat map. }
```

Kamus:

center : MapLocation

bestScore : integer

bestLoc : MapLocation

score : integer

Algorithm

```
center ← middle of the map
```

```
bestScore ←  $-\infty$ 
```

```
bestLoc ← null
```

```
for each d in allDirections do
```

```
    nextLoc ← rc location + d
```

```
    if robot cannot be built at nextLoc then
```

```
        continue
```

```
    endif
```

```
    score ← 0
```

```
    score ← score − distance(nextLoc, center)
```

```
    for each ally robot do
```

```
        score ← score + distance(nextLoc, ally location)
```

```
    endfor
```

```
    for each enemy robot do
```

```
        score ← score − distance(nextLoc, enemy location)
```

```
        if enemy is a tower then
```

```
            report enemy tower location
```

```
        endif
```

```
    endfor
```

```
    if score > bestScore then
```

```
        bestScore ← score
```

```
        bestLoc ← nextLoc
```

```
    endif
```

```
endfor
```

```
return bestLoc
```

```
function trySpawn(rc, unitType, allowFallbackSoldier)  $\rightarrow$  boolean
```

Algorithm

```
nextLoc  $\leftarrow$  getBestSpawnLocation(rc, unitType)
if nextLoc = null and allowFallbackSoldier then
    nextLoc  $\leftarrow$  getBestSpawnLocation(rc, SOLDIER)
    unitType  $\leftarrow$  SOLDIER
endif
if nextLoc  $\neq$  null and rc.canBuildRobot(unitType, nextLoc) then
    rc.buildRobot(unitType, nextLoc)
    inform communication system about new spawn
    buildCount  $\leftarrow$  buildCount + 1
    return true
endif
return false
```

```
procedure tryAttackEnemies(rc)
```

Algorithm

```
for each enemy robot nearby do
    if rc.canAttack(enemy location) then
        rc.attack(enemy location)
        if enemy is a tower then
            report enemy tower location
        endif
        rc.attack(null)
    return
    endif
endfor
```

f. Comms.java

```
function encode(type, loc)  $\rightarrow$  integer
```

{ Mengubah tipe pesan dan lokasi menjadi satu nilai integer. }

Algorithm

```
return (type shifted left by 12) OR (loc.x shifted left by 6) OR loc.y
```

```
function decodeType(bytes) → integer
```

Algorithm

```
return (bytes shifted right by 12) AND 3
```

```
function decodeLoc(bytes) → MapLocation
```

Kamus:

x : integer

y : integer

Algorithm

```
x ← (bytes shifted right by 6) AND 63
```

```
y ← bytes AND 63
```

```
return MapLocation(x, y)
```

```
procedure startTurn(rc : RobotController)
```

```
{ Membaca pesan dari robot lain dan memperbarui informasi tower musuh. }
```

Kamus:

messages : Message array

bytes : integer

Algorithm

```
Motion.addToQueue(current robot location)
```

```
messages ← rc.readMessages(all)
```

```
for each m in messages do
```

```
    bytes ← m.getBytes()
```

```
    if decodeType(bytes) = TYPE.ENEMY_TOWER then
```

```
        enemyTowerLoc ← decodeLoc(bytes)
```

```
    endif
```

```
endfor
```

```
procedure broadcast(rc, bytes)
```

```
{ Mengirim pesan ke semua robot sekutu di sekitar. }
```

Kamus:

```
allies : RobotInfo array
```

Algorithm

```
allies ← rc.senseNearbyRobots(all, ally team)
for each ally in allies do
    if rc.canSendMessage(ally location, bytes) then
        rc.sendMessage(ally location, bytes)
    endif
endfor
```

```
procedure reportEnemyTower(rc, loc)
```

```
{ Melaporkan lokasi tower musuh ke robot sekutu. }
```

Kamus:

```
message : integer
```

Algorithm

```
if loc = null then return endif
enemyTowerLoc ← loc
message ← encode(TYPE_ENEMY_TOWER, loc)
broadcast(rc, message)
```

```
procedure informNewSpawn(rc, spawnLoc)
```

```
{ Mengirim informasi tower musuh kepada robot yang baru dibuat. }
```

Kamus:

```
message : integer
```

Algorithm

```
if enemyTowerLoc = null then return endif
message ← encode(TYPE_ENEMY_TOWER, enemyTowerLoc)
if rc.canSendMessage(spawnLoc, message) then
    rc.sendMessage(spawnLoc, message)
endif
```

g. Motion.java

```
function moveTowardLocation(rc, target) → Direction
```

```
{ Menggerakkan robot menuju lokasi target menggunakan kombinasi greedy dan bug navigation.  
}
```

Kamus:

myLoc : MapLocation

direct : Direction

curDist : integer

best : Direction

follow : Direction

Algorithm

```
if target = null then return null endif  
myLoc ← current robot location  
if myLoc = target then return null endif  
if bugTarget ≠ target then reset bug navigation variables endif  
direct ← direction from myLoc to target  
curDist ← distance(myLoc, target)  
if currently tracing wall AND robot can move directly to target AND curDist < bugStartDist  
then  
    stop tracing wall    endif  
if not tracing wall then  
    if robot can move directly toward target then return direct endif  
    best ← getBestDirection(rc, target)  
    if best ≠ null then return best endif  
    start wall tracing  
endif  
if tracing steps exceed limit then reverse tracing side endif  
follow ← traceWall(rc)  
if follow ≠ null then return follow endif  
if robot can move directly to target then return direct endif  
return null
```

```
function getBestDirection(rc, target) → Direction
```

Kamus:

bestDir : Direction

bestScore : integer

score : integer

Algorithm

```
bestDir ← null
bestScore ←  $-\infty$ 
for each direction d do
    if robot cannot move in direction d then continue endif
    nextLoc ← location after moving to d
    if nextLoc exists in recent location queue then continue endif
    score ← 0
    score ← score − distance(nextLoc, target) × 2
    for each ally robot do
        score ← score + distance(nextLoc, ally)
    endfor
    for each enemy robot do
        score ← score − distance(nextLoc, enemy)
    endfor
    for each nearby tile do
        if tile is wall then score ← score − large penalty endif
    endfor
    if tile paint is ally color then decrease score endif
    if tile paint is enemy or empty then increase score endif
    if score > bestScore then
        bestScore ← score
        bestDir ← d
    endif
endfor
return bestDir
```

function traceWall(rc) → Direction**Algorithm**

```
dir ← bugTraceDir
if bugSide is right then
    rotate direction clockwise until movable
else
    rotate direction counterclockwise until movable
endif
if movable direction found then
    update bugTraceDir
    increase trace step count
    return direction
endif
return null
```

function moveTowardEnemyPaint(rc) → **Direction****Algorithm**

```
bestTarget ← null
closestDist ← ∞
for each tile in rc.senseNearbyMapInfos() do
    if tile paint is ENEMY_PRIMARY OR ENEMY_SECONDARY then
        dist ← distance(robot location, tile location)
        if dist < closestDist then
            closestDist ← dist
            bestTarget ← tile location
        endif
    endif
endfor
if bestTarget ≠ null then
    return moveTowardLocation(rc, bestTarget)
endif
return moveTowardCorner(rc)
```

function moveTowardNearestEnemy(rc) → **Direction****Kamus:**

```
enemies : RobotInfo array
nearest : MapLocation
closestDist : integer
dist : integer
```

Algorithm

```
enemies ← rc.senseNearbyRobots(opponent team)
nearest ← null
closestDist ← ∞
for each enemy e in enemies do
    dist ← distance between robot location and enemy location
    if dist < closestDist then
        closestDist ← dist
        nearest ← enemy location
    endif
endfor
if nearest ≠ null then
    return moveTowardLocation(rc, nearest)
endif
return moveTowardCenter(rc)
```

```
function moveTowardNearestEnemyTower(rc) → Direction
```

Kamus:

enemies : RobotInfo array

nearest : MapLocation

closestDist : integer

dist : integer

Algorithm

```
enemies ← rc.senseNearbyRobots(opponent team)
```

```
nearest ← null
```

```
closestDist ← ∞
```

```
for each enemy e in enemies do
```

```
    if isTowerType(e.type) then
```

```
        dist ← distance between robot location and enemy location
```

```
        if dist < closestDist then
```

```
            closestDist ← dist
```

```
            nearest ← enemy location
```

```
        endif
```

```
    endif
```

```
endfor
```

```
if nearest ≠ null then
```

```
    return moveTowardLocation(rc, nearest)
```

```
endif
```

```
return moveTowardCorner(rc)
```

```
function moveTowardNearestAllySoldier(rc) → Direction
```

Kamus:

allies : RobotInfo array

nearest : MapLocation

closestDist : integer

dist : integer

Algorithm

```
allies ← rc.senseNearbyRobots(ally team)
nearest ← null
closestDist ← ∞
for each ally a in allies do
  if a.type = SOLDIER then
    dist ← distance between robot location and ally location
    if dist < closestDist then
      closestDist ← dist
      nearest ← ally location
    endif
  endif
endfor
if nearest ≠ null then
  return moveTowardLocation(rc, nearest)
endif
return null
```

function moveTowardCenter(rc) → **Direction****Algorithm**

```
mapH ← rc.getMapHeight()
mapW ← rc.getMapWidth()
center ← MapLocation(mapW / 2, mapH / 2)
return moveTowardLocation(rc, center)
```

function moveTowardCorner(rc) → **Direction****Kamus:**

```
corners : list of MapLocation
myLoc : MapLocation
targetCorner : MapLocation
farthestDist : integer
dist : integer
```

Algorithm

```
corners ← list of map corners and edge midpoints
myLoc ← robot location
targetCorner ← lastCornerTarget
if targetCorner = null then
    farthestDist ←  $-\infty$ 
    for each corner c in corners do
        dist ← distance between myLoc and c
        if dist > farthestDist then
            farthestDist ← dist
            targetCorner ← c
        endif
    endfor
endif
if distance between myLoc and targetCorner  $\leq$  threshold then
    idx ← index of targetCorner in corners
    if robot ID is even then
        step ← 1
    else
        step ← 3
    endif
    targetCorner ← corners[(idx + step) mod number of corners]
endif
lastCornerTarget ← targetCorner
return moveTowardLocation(rc, targetCorner)
```

procedure addtoQueue(loc)**Algorithm**

```
for i from last index of queue down to 1 do
    queueLastLocations[i] ← queueLastLocations[i - 1]
endfor
queueLastLocations[0] ← loc
```

4.1.3 Algoritma Solusi Greedy 3

a. RobotPlayer.java

procedure run(input rc : RobotController)

{ Menjalankan bot utama secara terus-menerus.

Input: rc, controller unit yang sedang aktif }

Kamus:

-

Algorithm

```
call G.init(rc)
while true do
  try
    call G.update()
    case rc.getType() of
      SOLDIER : call Robot.runSoldier()
      MOPPER : call Robot.runMopper()
      SPLASHER : call Robot.runSplasher()
      else : call Tower.run()
    endcase
  catch exception e
    print "Err:" + exception message
  endtry
  call Clock.yield()
endwhile
```

b. G.java

procedure init(input controller : RobotController)

{ Inisialisasi seluruh variabel global yang dipakai bot.

Input: controller robot atau tower yang aktif }

Kamus:

initAllies : array of RobotInfo

i : integer

Algorithm

rc ← controller

team ← rc.getTeam()

opponent ← team.opponent()

me ← rc.getLocation()

myType ← rc.getType()

isRobotUnit ← (myType = SOLDIER) or (myType = MOPPER) or (myType = SPLASHER)

moneyPattern ← rc.getTowerPattern(LEVEL.ONE_MONEY_TOWER)

```
paintPattern ← rc.getTowerPattern(LEVEL.ONE_PAINT_TOWER)
defensePattern ← rc.getTowerPattern(LEVEL.ONE_DEFENSE_TOWER)
recentLocs[0] ← me
histPtr ← 1
lastHistRound ← rc.getRoundNum()
if isRobotUnit then
    initAllies ← rc.senseNearbyRobots(-1, team)
    for i ← panjang(initAllies)-1 downto 0 do
        if Util.isTower(initAllies[i].type) then
            spawnBase ← initAllies[i].location
            lastSeenAllyTower ← spawnBase
            break
        endif
    endfor
else
    spawnBase ← me
    lastSeenAllyTower ← me
endif
if spawnBase ≠ null then
    inferredEnemyBase ← Util.inferEnemyBase(spawnBase)
endif
if inferredEnemyBase ≠ null then
    pushTarget ← inferredEnemyBase
else
    pushTarget ← Util.pickFrontierLikeTarget()
endif
pushTurnsLeft ← 60
```

procedure update()

{ Memperbarui state global setiap ronde. }

Kamus:

cur : MapLocation

Algorithm

```
cur ← rc.getLocation()
if cur ≠ me then
    prevPrevLoc ← prevLoc
    prevLoc ← me
endif
me ← cur
round ← rc.getRoundNum()
paint ← rc.getPaint()
chips ← rc.getChips()
nearbyAllies ← rc.senseNearbyRobots(-1, team)
nearbyEnemies ← rc.senseNearbyRobots(-1, opponent)
nearbyTiles ← rc.senseNearbyMapInfos()
nearbyAllyLen ← panjang(nearbyAllies)
nearbyEnemyLen ← panjang(nearbyEnemies)
nearbyTileLen ← panjang(nearbyTiles)
call rememberTowers()
call updatePushTarget()
if round ≠ lastHistRound then
    recentLocs[histPtr mod HIST] ← me
    histPtr ← histPtr + 1
    lastHistRound ← round
endif
```

procedure rememberTowers()

{ Mengingat lokasi tower sekutu dan musuh yang terlihat. }

Kamus:

i : integer

r : RobotInfo

Algorithm

```
for i ← nearbyAllyLen-1 downto 0 do
    r ← nearbyAllies[i]
    if Util.isTower(r.type) then
        lastSeenAllyTower ← r.location
```



```
endif
endfor
for i ← nearbyEnemyLen-1 downto 0 do
  r ← nearbyEnemies[i]
  if Util.isTower(r.type) then
    lastSeenEnemyTower ← r.location
    inferredEnemyBase ← r.location
  endif
endfor
```

procedure updatePushTarget()

{ Menentukan target dorongan berikutnya. }

Kamus:

needNew : boolean

Algorithm

```
needNew ← (pushTarget = null)
           or (pushTurnsLeft ≤ 0)
           or (jarakKuadrat(me, pushTarget) ≤ 9)
           or (prevLoc ≠ null and prevPrevLoc ≠ null and me = prevPrevLoc)
if needNew then
  if lastSeenEnemyTower ≠ null then
    pushTarget ← lastSeenEnemyTower
    pushTurnsLeft ← 30
  else if inferredEnemyBase ≠ null then
    pushTarget ← inferredEnemyBase
    pushTurnsLeft ← 40
  else
    pushTarget ← Util.pickFrontierLikeTarget()
    pushTurnsLeft ← 22
  endif
else
  pushTurnsLeft ← pushTurnsLeft - 1
endif
```

```
function timesVisited(input loc : MapLocation) → integer
```

```
{ Menghitung berapa kali sebuah lokasi muncul di riwayat kunjungan.
```

```
Input: loc, lokasi yang ingin dicek }
```

```
Kamus:
```

```
    cnt, i : integer
```

```
Algorithm
```

```
    cnt ← 0
```

```
    for i ← HIST-1 downto 0 do
```

```
        if recentLocs[i] ≠ null and recentLocs[i] = loc then
```

```
            cnt ← cnt + 1
```

```
        endif
```

```
    endfor
```

```
    return cnt
```

```
function paintPct() → integer
```

```
{ Mengembalikan persentase paint unit saat ini. }
```

```
Kamus:
```

```
    cap : integer
```

```
Algorithm
```

```
    cap ← myType.paintCapacity
```

```
    if cap ≤ 0 then
```

```
        return 100
```

```
    else
```

```
        return paint * 100 div cap
```

```
    endif
```

c. Util.java

```
function isTower(input type : UnitType) → boolean
```

```
{ Mengecek apakah sebuah tipe unit adalah tower.
```

```
Input: tipe unit }
```

```
Kamus:
```

-

Algorithm

```
return (type = LEVEL.ONE_PAINT_TOWER)
      or (type = LEVEL.TWO_PAINT_TOWER)
      or (type = LEVEL.THREE_PAINT_TOWER)
      or (type = LEVEL.ONE_MONEY_TOWER)
      or (type = LEVEL.TWO_MONEY_TOWER)
      or (type = LEVEL.THREE_MONEY_TOWER)
      or (type = LEVEL.ONE_DEFENSE_TOWER)
      or (type = LEVEL.TWO_DEFENSE_TOWER)
      or (type = LEVEL.THREE_DEFENSE_TOWER)
```

function inferEnemyBase(input allyBase : MapLocation) → MapLocation

{ Menginfer posisi base musuh dengan asumsi simetri rotasi peta.

Input: lokasi base sekutu }

Kamus:

w, h : integer

Algorithm

```
w ← G.rc.getMapWidth()
h ← G.rc.getMapHeight()
return MapLocation(w - 1 - allyBase.x, h - 1 - allyBase.y)
```

function randomMapLocation() → MapLocation

{ Menghasilkan lokasi acak di dalam peta. }

Kamus:

-

Algorithm

```
return MapLocation(
    G.rng.nextInt(G.rc.getMapWidth()),
    G.rng.nextInt(G.rc.getMapHeight())
)
```

function pickFrontierLikeTarget() → MapLocation

```
{ Memilih target acak yang cenderung menuju frontier atau area kontestasi. }
```

Kamus:

```
w, h : integer  
best, cand : MapLocation  
bestScore, score, i : integer
```

Algorithm

```
w ← G.rc.getMapWidth()  
h ← G.rc.getMapHeight()  
best ← null  
bestScore ←  $-\infty$   
for i ← 9 downto 0 do  
    cand ← MapLocation(G.rng.nextInt(w), G.rng.nextInt(h))  
    score ← 0  
    if G.lastSeenEnemyTower ≠ null then  
        score ← score - jarakKuadrat(cand, G.lastSeenEnemyTower) div 3  
    else  
        score ← score + jarakKuadrat(cand, G.me) div 4  
    endif  
    if G.lastSeenAllyTower ≠ null then  
        score ← score + jarakKuadrat(cand, G.lastSeenAllyTower) div 8  
    endif  
    if score > bestScore then  
        bestScore ← score  
        best ← cand  
    endif  
endfor  
if best ≠ null then  
    return best  
else  
    return randomMapLocation()  
endif
```

d. Logistics.java

```
function needRefillSoldier() → boolean
```

```
{ Menentukan apakah Soldier perlu refill paint. }
```

Kamus:

-

Algorithm

```
  if G.paint ≤ 35 then
```

```
    return true
```

```
  endif
```

```
  if G.paint ≤ 60 and farFromSupply() then
```

```
    return true
```

```
  endif
```

```
  if G.paint ≤ 80 and G.lastSeenAllyTower ≠ null
```

```
    and jarakKuadrat(G.me, G.lastSeenAllyTower) ≤ 2 then
```

```
      return true
```

```
  endif
```

```
  return false
```

```
function needRefillMopper() → boolean
```

```
{ Menentukan apakah Mopper perlu refill paint. }
```

Kamus:

-

Algorithm

```
  return (G.paint ≤ 20) or (G.paint ≤ 35 and farFromSupply())
```

```
function needRefillSplasher() → boolean
```

```
{ Menentukan apakah Splasher perlu refill paint. }
```

Kamus:

-

Algorithm

```
  return (G.paint ≤ 80) or (G.paint ≤ 120 and farFromSupply())
```

```
function farFromSupply() → boolean
```

```
{ Mengecek apakah unit jauh dari sumber paint sekutu terdekat. }
```

Kamus:

-

Algorithm

```
return G.lastSeenAllyTower  $\neq$  null  
and jarakKuadrat(G.me, G.lastSeenAllyTower)  $\leq$  30
```

procedure goRefill()

```
{ Mencoba refill paint atau bergerak menuju tower sekutu. }
```

Kamus:

-

Algorithm

```
if tryWithdrawAdjacent() then  
    return  
endif  
  
if G.lastSeenAllyTower  $\neq$  null then  
    call Motion.moveToward(G.lastSeenAllyTower, false)  
else  
    call Motion.safeFallbackMove(false)  
endif
```

function tryWithdrawAdjacent() $\rightarrow$ boolean

```
{ Mengambil paint dari tower sekutu yang cukup dekat. }
```

Kamus:

cap, want, i : integer

ally : RobotInfo

Algorithm

```
if not G.isRobotUnit then  
    return false  
endif  
  
cap  $\leftarrow$  G.myType.paintCapacity  
if G.paint  $\geq 9 * cap \div 10$  then  
    return false
```

```
endif
for i ← G.nearbyAllyLen-1 downto 0 do
  ally ← G.nearbyAllies[i]
  if not Util.isTower(ally.type) then
    continue
  endif
  if jarakKuadrat(G.me, ally.location) > 2 then
    continue
  endif
  want ← cap - G.paint
  if G.rc.canTransferPaint(ally.location, -want) then
    G.rc.transferPaint(ally.location, -want)
    G.paint ← G.rc.getPaint()
    return true
  endif
endif
endfor
return false
```

function tryTransferPaintToAlly() → **boolean**
{ Mopper menyalurkan paint ke ally yang kekurangan paint. }

Kamus:

bestAlly : RobotInfo
bestScore, i, pct, score, amount : integer
at : UnitType
dir : Direction
ally : RobotInfo

Algorithm

```
if G.myType ≠ MOPPER then
  return false
endif
if G.paint ≤ 30 then
  return false
endif
```

```
bestAlly ← null
bestScore ←  $-\infty$ 
for i ← G.nearbyAllyLen-1 downto 0 do
    ally ← G.nearbyAllies[i]
    if ally.location = G.me then
        continue
    endif
    at ← ally.type
    if at ≠ SOLDIER and at ≠ SPLASHER and at ≠ MOPPER then
        continue
    endif
    pct ← ally.paintAmount * 100 div at.paintCapacity
    if pct < 55 then
        continue
    endif
    if pct ≤ 10 then
        score ← 80
    else if pct ≤ 25 then
        score ← 50
    else if pct ≤ 40 then
        score ← 25
    else
        score ← 10
    endif
    if at = SOLDIER then
        score ← score + 15
    else if at = SPLASHER then
        score ← score + 10
    endif
    score ← score - 2 * jarakKuadrat(G.me, ally.location)
    if score > bestScore then
        bestScore ← score
        bestAlly ← ally
    endif
endfor
```



```
        endif
    endfor

    if bestAlly = null then
        return false
    endif

    amount ← minimum(minimum(G.paint - 30, bestAlly.type.paintCapacity -
bestAlly.paintAmount), 70)

    if amount ≤ 5 then
        return false
    endif

    if jarakKuadrat(G.me, bestAlly.location) ≤ 2
        and G.rc.canTransferPaint(bestAlly.location, amount) then
            G.rc.transferPaint(bestAlly.location, amount)
            return true
        endif
    endif

    dir ← arah dari G.me ke bestAlly.location
    if dir ≠ CENTER and G.rc.canMove(dir) then
        call Motion.executeMove(dir)
        return true
    endif

    return false
```

e. Motion.java

procedure executeMove(input dir : Direction)

{ Melakukan gerakan satu langkah jika valid, lalu memperbarui histori posisi.

Input: dir, arah gerak }

Kamus:

-

Algorithm

```
    if dir = null or dir = CENTER or not G.rc.canMove(dir) then
        return
    endif

    G.rc.move(dir)
```

```
G.lastMoveDir ← dir
G.prevPrevLoc ← G.prevLoc
G.prevLoc ← G.me
G.me ← G.rc.getLocation()
```

```
function moveToward(input target : MapLocation, input aggressive : boolean) →
boolean
```

{ Bergerak ke arah target dengan memilih arah ber-skor terbaik.

Input: target, target gerak; aggressive, mode gerak }

Kamus:

```
bestDir, dir : Direction
bestScore, curDist, nxtDist, score, di : integer
nxt : MapLocation
```

Algorithm

```
if target = null then
    return false
endif

bestDir ← null
bestScore ←  $-\infty$ 
curDist ← jarakKuadrat(G.me, target)
for di ← 7 downto 0 do
    dir ← G.DIRS[di]
    if not G.rc.canMove(dir) then
        continue
    endif
    nxt ← G.me + dir
    nxtDist ← jarakKuadrat(nxt, target)
    score ← Greedy.scoreMove(nxt, aggressive)
    if nxtDist < curDist then
        if aggressive then
            score ← score + 42
        else
            score ← score + 32
        end if
    end if
end for
```

```
        endif
    else if nxtDist > curDist then
        if aggressive then
            score ← score - 32
        else
            score ← score - 24
        endif
    endif
    if score > bestScore then
        bestScore ← score
        bestDir ← dir
    endif
endfor
if bestDir ≠ null then
    call executeMove(bestDir)
    return true
endif
return false
```

function followWeakAlly() → boolean

{ Mencari ally yang paint-nya rendah lalu mendekatinya. }

Kamus:

best, ally : RobotInfo

bestScore, i, pct, score : integer

at : UnitType

Algorithm

best ← null

bestScore ← $-\infty$

for i ← G.nearbyAllyLen-1 downto 0 do

ally ← G.nearbyAllies[i]

if ally.location = G.me then

continue

endif

```
at ← ally.type
if at ≠ SOLDIER and at ≠ SPLASHER and at ≠ MOPPER then
    continue
endif
pct ← ally.paintAmount * 100 div at.paintCapacity
if pct < 40 then
    continue
endif
if pct ≤ 10 then
    score ← 50
else if pct ≤ 25 then
    score ← 30
else
    score ← 15
endif
if at = SOLDIER then
    score ← score + 8
endif
score ← score - jarakKuadrat(G.me, ally.location)
if score > bestScore then
    bestScore ← score
    best ← ally
endif
endfor
return best ≠ null and moveToward(best.location, false)
```

procedure safeFallbackMove(input aggressive : boolean)

{ Gerakan fallback jika tidak ada gerakan utama yang dipilih.

Input: aggressive, mode scoring gerak }

Kamus:

bestDir, dir : Direction

bestScore, score, d, di : integer

```
nxt : MapLocation
```

Algorithm

```
bestDir ← null
bestScore ←  $-\infty$ 
for di ← 7 downto 0 do
    dir ← G.DIRS[di]
    if not G.rc.canMove(dir) then
        continue
    endif
    nxt ← G.me + dir
    score ← Greedy.scoreMove(nxt, aggressive)
    if G.pushTarget  $\neq$  null then
        d ← jarakKuadrat(G.me, G.pushTarget) - jarakKuadrat(nxt, G.pushTarget)
        if d > 0 then
            score ← score + 16
        else if d < 0 then
            score ← score - 12
        endif
    endif
    if score > bestScore then
        bestScore ← score
        bestDir ← dir
    endif
endfor
if bestDir  $\neq$  null then
    call executeMove(bestDir)
endif
```

f. Greedy.java

```
function tryAttackEnemyTower() → boolean
```

```
{ Menyerang tower musuh terbaik yang ada dalam jangkauan. }
```

Kamus:

```
best, e : RobotInfo
```

```
bestScore, score, i : integer
```

Algorithm

```
best ← null
```

```
bestScore ←  $-\infty$ 
```

```
for i ← G.nearbyEnemyLen-1 downto 0 do
```

```
    e ← G.nearbyEnemies[i]
```

```
    if not Util.isTower(e.type) or not G.rc.canAttack(e.location) then
```

```
        continue
```

```
    endif
```

```
    score ← 120
```

```
    if e.health  $\leq$  100 then
```

```
        score ← score + 60
```

```
    endif
```

```
    score ← score - jarakKuadrat(G.me, e.location)
```

```
    if score > bestScore then
```

```
        bestScore ← score
```

```
        best ← e
```

```
    endif
```

```
endfor
```

```
if best  $\neq$  null then
```

```
    G.rc.attack(best.location)
```

```
    G.paint ← G.paint - 5
```

```
    return true
```

```
endif
```

```
return false
```

```
function tryAttackEnemyRobotEfficient() → boolean
```

```
{ Menyerang robot musuh non-tower bila tile target bukan tile ally. }
```

Kamus:

```
e : RobotInfo
```

```
tileInfo : MapInfo
```

```
tp : PaintType
```

```
i : integer
```

Algorithm

```
for i ← G.nearbyEnemyLen-1 downto 0 do
  e ← G.nearbyEnemies[i]
  if Util.isTower(e.type) then
    continue
  endif
  if not G.rc.canAttack(e.location) then
    continue
  endif
  tileInfo ← findCachedTileOrSense(e.location)
  if tileInfo = null then
    continue
  endif
  tp ← tileInfo.getPaint()
  if tp.isAlly() then
    continue
  endif
  G.rc.attack(e.location, false)
  G.paint ← G.paint - 5
  return true
endfor
return false
```

function tryBestPaintAttack() → **boolean**

{ Memilih tile terbaik untuk diwarnai oleh Soldier. }

Kamus:**bestTarget** : MapLocation**bestSec** : boolean**bestScore**, **s1**, **s2**, **i** : integer**tile** : MapInfo**Algorithm****bestTarget** ← null

```
bestSec ← false
bestScore ←  $-\infty$ 
for i ← G.nearbyTileLen-1 downto 0 do
  tile ← G.nearbyTiles[i]
  if not G.rc.canAttack(tile.getMapLocation()) then
    continue
  endif
  s1 ← scorePaintTarget(tile, false)
  if s1 > bestScore then
    bestScore ← s1
    bestTarget ← tile.getMapLocation()
    bestSec ← false
  endif
  s2 ← scorePaintTarget(tile, true)
  if s2 > bestScore then
    bestScore ← s2
    bestTarget ← tile.getMapLocation()
    bestSec ← true
  endif
endfor
if bestTarget ≠ null and bestScore ≥ 5 then
  G.rc.attack(bestTarget, bestSec)
  G.paint ← G.paint - 5
  return true
endif
return false
```

function tryCleanEnemyPaint() → **boolean**

{ Mopper membersihkan cat musuh terbaik atau melakukan mop swing terbaik. }

Kamus:

bestTarget, loc : MapLocation

bestScore, score, bestSwingScore, i, j, di : integer

tile : MapInfo

p : PaintType

bestSwing, dir : Direction

Algorithm

bestTarget $\leftarrow$ null

bestScore $\leftarrow -\infty$

for i $\leftarrow$ G.nearbyTileLen-1 downto 0 do

 tile $\leftarrow$ G.nearbyTiles[i]

 p $\leftarrow$ tile.getPaint()

 if p = EMPTY or p.isAlly() then

 continue

 endif

 loc $\leftarrow$ tile.getMapLocation()

 if not G.rc.canAttack(loc) then

 continue

 endif

 score $\leftarrow$ 30 - jarakKuadrat(G.me, loc) + lightFrontierBonus(loc)

 for j $\leftarrow$ G.nearbyEnemyLen-1 downto 0 do

 if G.nearbyEnemies[j].location = loc then

 score $\leftarrow$ score + 25

 break

 endif

 endfor

 if score > bestScore then

 bestScore $\leftarrow$ score

 bestTarget $\leftarrow$ loc

 endif

endfor

if bestTarget $\neq$ null then

 G.rc.attack(bestTarget)

 return true

endif

bestSwing $\leftarrow$ null

bestSwingScore $\leftarrow$ 5

```
for di ← 3 downto 0 do
    dir ← G.DIRS[2*di]
    if not G.rc.canMopSwing(dir) then
        continue
    endif
    score ← estimateSwingScore(dir)
    if score > bestSwingScore then
        bestSwingScore ← score
        bestSwing ← dir
    endif
endfor
if bestSwing ≠ null then
    G.rc.mopSwing(bestSwing)
    return true
endif
return false
```

function tryBestSplashAttack() → **boolean**

{ Memilih pusat splash attack terbaik untuk Splasher. }

Kamus:

bestTarget, loc : MapLocation

bestScore, score, i : integer

Algorithm

```
if G.paint < 55 then
    return false
endif
bestTarget ← null
bestScore ←  $-\infty$ 
for i ← G.nearbyTileLen-1 downto 0 do
    loc ← G.nearbyTiles[i].getMapLocation()
    if not G.rc.canAttack(loc) then
        continue
    endif
```

```
score ← splashValue(loc) - 35
if score > bestScore then
    bestScore ← score
    bestTarget ← loc
endif
endfor
if bestTarget ≠ null and bestScore > 0 then
    G.rc.attack(bestTarget)
    G.paint ← G.paint - 50
    return true
endif
return false
```

function tryExpandMove() → boolean

{ Memilih gerakan ekspansi terbaik untuk Soldier. }

Kamus:

bestDir, dir : Direction

bestScore, score, d, di : integer

nxt : MapLocation

Algorithm

bestDir ← null

bestScore ← $-\infty$

for di ← 7 downto 0 do

 dir ← G.DIRS[di]

 if not G.rc.canMove(dir) then

 continue

 endif

 nxt ← G.me + dir

 score ← scoreMove(nxt, false)

 + 5 * frontierBonusLight(nxt)

 + 2 * localPaintOpportunity(nxt)

 if G.pushTarget ≠ null then

 d ← jarakKuadrat(G.me, G.pushTarget) - jarakKuadrat(nxt, G.pushTarget)

```
    if d > 0 then
        score ← score + 30
    else if d < 0 then
        score ← score - 25
    endif
endif
if score > bestScore then
    bestScore ← score
    bestDir ← dir
endif
endfor
if bestDir ≠ null and bestScore ≠ 0 then
    call Motion.executeMove(bestDir)
    return true
endif
return false
```

function tryContestMove() → **boolean**

{ Memilih gerakan kontestasi terbaik untuk Splasher. }

Kamus:

bestDir, dir : Direction

bestScore, score, d, di : integer

nxt : MapLocation

Algorithm

bestDir ← null

bestScore ← $-\infty$

for di ← 7 downto 0 do

dir ← G.DIRS[di]

if not G.rc.canMove(dir) then

continue

endif

nxt ← G.me + dir

score ← scoreMove(nxt, true)

```
if G.pushTarget  $\neq$  null then
    d  $\leftarrow$  jarakKuadrat(G.me, G.pushTarget) - jarakKuadrat(nxt, G.pushTarget)
    if d > 0 then
        score  $\leftarrow$  score + 26
    else if d < 0 then
        score  $\leftarrow$  score - 20
    endif
endif
if score > bestScore then
    bestScore  $\leftarrow$  score
    bestDir  $\leftarrow$  dir
endif
endfor
if bestDir  $\neq$  null then
    call Motion.executeMove(bestDir)
    return true
endif
return false
```

function scoreMove(input nxt : MapLocation, input aggressive : boolean) $\rightarrow$ integer

{ Memberi skor pada lokasi gerak kandidat.

Input: nxt, lokasi kandidat; aggressive, mode penilaian }

Kamus:

nxtInfo : MapInfo

p, tp : PaintType

paintPct, score, frontier, adjacent, drain, futurePaint, revisit : integer

candDir : Direction

d, dist, i : integer

a : RobotInfo

t1 : MapLocation

Algorithm

nxtInfo $\leftarrow$ findCachedTileOrSense(nxt)

```
if nxtInfo  $\neq$  null then
    p  $\leftarrow$  nxtInfo.getPaint()
else
    p  $\leftarrow$  EMPTY
endif
paintPct  $\leftarrow$  G.paintPct()
if paintPct  $\leq$  15 then
    if p.isAlly() then
        score  $\leftarrow$  30
    else if p = EMPTY then
        score  $\leftarrow$  5
    else
        score  $\leftarrow$  -40
    endif
else if paintPct  $\leq$  40 then
    if p.isAlly() then
        score  $\leftarrow$  15
    else if p = EMPTY then
        score  $\leftarrow$  18
    else
        score  $\leftarrow$  -20
    endif
else
    if p.isAlly() then
        score  $\leftarrow$  -8
    else if p = EMPTY then
        score  $\leftarrow$  20
    else if aggressive then
        score  $\leftarrow$  12
    else
        score  $\leftarrow$  6
    endif
endif
```

```
frontier  $\leftarrow$  0
adjacent  $\leftarrow$  0
if p.isAlly() then
    drain  $\leftarrow$  0
else if p = EMPTY then
    drain  $\leftarrow$  1
else
    drain  $\leftarrow$  2
endif
for i  $\leftarrow$  G.nearbyTileLen-1 downto 0 do
    t1  $\leftarrow$  G.nearbyTiles[i].getMapLocation()
    dist  $\leftarrow$  jarakKuadrat(nxt, t1)
    if dist > 0 and dist  $\leq$  2 then
        tp  $\leftarrow$  G.nearbyTiles[i].getPaint()
        if tp = EMPTY then
            frontier  $\leftarrow$  frontier + 2
        else if not tp.isAlly() then
            frontier  $\leftarrow$  frontier + 3
        endif
    endif
endfor
for i  $\leftarrow$  G.nearbyAllyLen-1 downto 0 do
    a  $\leftarrow$  G.nearbyAllies[i]
    if a.location  $\neq$  G.me and jarakKuadrat(nxt, a.location)  $\leq$  2 then
        adjacent  $\leftarrow$  adjacent + 1
    endif
endfor
score  $\leftarrow$  score + 3 * frontier
score  $\leftarrow$  score - 6 * adjacent
futurePaint  $\leftarrow$  G.paint - drain - adjacent
if futurePaint  $\leq$  5 then
    score  $\leftarrow$  score - 90
else if futurePaint  $\leq$  12 then
```

```
    score ← score - 40
else if futurePaint ≤ 20 then
    score ← score - 15
endif
revisit ← G.timesVisited(nxt)
score ← score - 55 * revisit
if G.prevLoc ≠ null and nxt = G.prevLoc then
    score ← score - 100
endif
if G.prevPrevLoc ≠ null and nxt = G.prevPrevLoc then
    score ← score - 35
endif
candDir ← arah dari G.me ke nxt
if G.lastMoveDir ≠ CENTER then
    if candDir = opposite(G.lastMoveDir) then
        score ← score - 60
    else if candDir = G.lastMoveDir then
        score ← score + 10
    endif
endif
if G.lastSeenAllyTower ≠ null and paintPct ≤ 40 then
    score ← score - jarakKuadrat(nxt, G.lastSeenAllyTower) div 3
endif
for i ← G.nearbyEnemyLen-1 downto 0 do
    d ← jarakKuadrat(nxt, G.nearbyEnemies[i].location)
    if d ≤ 4 then
        score ← score - 6
    else if d ≤ 9 then
        score ← score - 2
    endif
endfor
return score
```



```
function scorePaintTarget(input tile : MapInfo, input useSec : boolean) →  
integer
```

{ Memberi skor pada sebuah tile untuk aksi pewarnaan Soldier.

Input: tile, tile target; useSec, pilihan warna }

Kamus:

p, ap : PaintType

loc, al : MapLocation

score, allyN, enemyN, d, after, i : integer

Algorithm

```
p ← tile.getPaint()
```

```
loc ← tile.getMapLocation()
```

```
if p.isAlly() then
```

```
    return -999
```

```
endif
```

```
if p = EMPTY then
```

```
    score ← 22
```

```
    if G.round ≤ 140 then
```

```
        score ← score + 15
```

```
    endif
```

```
else
```

```
    score ← 30
```

```
endif
```

```
allyN ← 0
```

```
enemyN ← 0
```

```
for i ← G.nearbyTileLen-1 downto 0 do
```

```
    al ← G.nearbyTiles[i].getMapLocation()
```

```
    d ← jarakKuadrat(loc, al)
```

```
    if d = 0 or d > 2 then
```

```
        continue
```

```
    endif
```

```
    ap ← G.nearbyTiles[i].getPaint()
```

```
    if ap.isAlly() then
```

```
        allyN ← allyN + 1
```

```
    else if ap  $\neq$  EMPTY then
        enemyN  $\leftarrow$  enemyN + 1
    endif
endfor

if allyN  $\geq$  2 then score  $\leftarrow$  score + 8 endif
if allyN  $\geq$  4 then score  $\leftarrow$  score + 12 endif
if allyN  $\geq$  6 then score  $\leftarrow$  score + 18 endif
score  $\leftarrow$  score - 3 * enemyN
score  $\leftarrow$  score + 2 * lightFrontierBonus(loc)
for i  $\leftarrow$  G.nearbyTileLen-1 downto 0 do
    if      G.nearbyTiles[i].hasRuin()      and      jarakKuadrat(loc,
G.nearbyTiles[i].getMapLocation())  $\leq$  8 then
        score  $\leftarrow$  score + 15
        break
    endif
endfor

if G.inferredEnemyBase  $\neq$  null then
    if      jarakKuadrat(loc, G.inferredEnemyBase)      ;      jarakKuadrat(G.me,
G.inferredEnemyBase) then
        score  $\leftarrow$  score + 6
    endif
endif

after  $\leftarrow$  G.paint - 5
if after  $\leq$  0 then
    return -999
else if after  $\leq$  10 then
    score  $\leftarrow$  score - 80
else if after  $\leq$  20 then
    score  $\leftarrow$  score - 30
endif

score  $\leftarrow$  score - jarakKuadrat(G.me, loc) div 2
return score
```

```
function lightFrontierBonus(input loc : MapLocation) → integer
```

```
{ Menghitung bonus frontier ringan di sekitar satu tile.
```

```
Input: loc, lokasi yang dinilai }
```

```
Kamus:
```

```
    bonus, i : integer
```

```
    adj : MapLocation
```

```
    p : PaintType
```

```
Algorithm
```

```
    bonus ← 0
```

```
    for i ← G.nearbyTileLen-1 downto 0 do
```

```
        adj ← G.nearbyTiles[i].getMapLocation()
```

```
        if jarakKuadrat(loc, adj) ≠ 1 then
```

```
            continue
```

```
        endif
```

```
        p ← G.nearbyTiles[i].getPaint()
```

```
        if p = EMPTY then
```

```
            bonus ← bonus + 3
```

```
        else if not p.isAlly() then
```

```
            bonus ← bonus + 5
```

```
        endif
```

```
    endfor
```

```
    return bonus
```

```
function frontierBonusLight(input loc : MapLocation) → integer
```

```
{ Bonus frontier untuk lokasi gerak dalam radius 1 atau 2.
```

```
Input: loc, lokasi yang dinilai }
```

```
Kamus:
```

```
    bonus, d, i : integer
```

```
    adj : MapLocation
```

```
    p : PaintType
```

```
Algorithm
```

```
    bonus ← 0
```

```
    for i ← G.nearbyTileLen-1 downto 0 do
```

```
adj ← G.nearbyTiles[i].getMapLocation()
d ← jarakKuadrat(loc, adj)
if d ≠ 1 and d ≠ 2 then
    continue
endif
p ← G.nearbyTiles[i].getPaint()
if p = EMPTY then
    bonus ← bonus + 3
else if not p.isAlly() then
    bonus ← bonus + 5
endif
endfor
return bonus
```

function localPaintOpportunity(input center : MapLocation) → integer

{ Menghitung potensi pewarnaan di sekitar satu lokasi.

Input: center, pusat evaluasi }

Kamus:

v, d, i : integer

adj : MapLocation

p : PaintType

Algorithm

v ← 0

for i ← G.nearbyTileLen-1 downto 0 do

adj ← G.nearbyTiles[i].getMapLocation()

d ← jarakKuadrat(center, adj)

if d ≠ 1 and d ≠ 2 then

continue

endif

p ← G.nearbyTiles[i].getPaint()

if p = EMPTY then

v ← v + 2

else if not p.isAlly() then

```
        v ← v + 3
    endif
endfor
return v
```

function countAdjacentAllies(input loc : MapLocation) → integer

{ Menghitung jumlah ally di sekitar satu lokasi.

Input: loc, lokasi yang dicek }

Kamus:

cnt, i : integer

a : RobotInfo

Algorithm

cnt ← 0

for i ← G.nearbyAllyLen-1 downto 0 do

a ← G.nearbyAllies[i]

if a.location ≠ G.me and jarakKuadrat(loc, a.location) ≤ 2 then

cnt ← cnt + 1

endif

endfor

return cnt

function estimateSwingScore(input dir : Direction) → integer

{ Menghitung nilai mop swing ke satu arah.

Input: dir, arah swing }

Kamus:

step1, step2, loc : MapLocation

score, d1, d2, i : integer

p : PaintType

Algorithm

step1 ← G.me + dir

step2 ← step1 + dir

score ← 0

for i ← G.nearbyTileLen-1 downto 0 do

```
loc ← G.nearbyTiles[i].getMapLocation()
if jarakKuadrat(step1, loc) ≤ 2 or jarakKuadrat(step2, loc) ≤ 2 then
    p ← G.nearbyTiles[i].getPaint()
    if p ≠ EMPTY and not p.isAlly() then
        score ← score + 10
    endif
endif
endfor
for i ← G.nearbyEnemyLen-1 downto 0 do
    d1 ← jarakKuadrat(G.nearbyEnemies[i].location, step1)
    d2 ← jarakKuadrat(G.nearbyEnemies[i].location, step2)
    if d1 ≤ 2 or d2 ≤ 2 then
        score ← score + 15
    endif
endfor
if G.paint ≤ 20 then
    score ← score - 15
endif
return score
```

function splashValue(input center : MapLocation) → integer

{ Menghitung nilai serangan splash untuk satu pusat serangan.

Input: center, pusat splash }

Kamus:

score, i : integer

loc : MapLocation

p : PaintType

Algorithm

score ← 0

for i ← G.nearbyTileLen-1 downto 0 do

loc ← G.nearbyTiles[i].getMapLocation()

if jarakKuadrat(center, loc) ≤ 4 then

continue

```
endif  
p ← G.nearbyTiles[i].getPaint()  
if p = EMPTY then  
    score ← score + 4  
else if not p.isAlly() then  
    score ← score + 8  
else  
    score ← score - 2  
endif  
endfor  
return score
```

function findCachedTileOrSense(input loc : MapLocation) → MapInfo

{ Mencari info tile dari cache nearby, atau sense langsung jika bisa.

Input: loc, lokasi yang ingin dicek }

Kamus:

i : integer

Algorithm

```
for i ← G.nearbyTileLen-1 downto 0 do  
    if G.nearbyTiles[i].getMapLocation() = loc then  
        return G.nearbyTiles[i]  
    endif  
endfor  
if G.rc.canSenseLocation(loc) then  
    return G.rc.senseMapInfo(loc)  
else  
    return null  
endif
```

g. Robot.java

procedure runSoldier()

{ Menjalankan perilaku Soldier setiap ronde. }

Kamus:

moved : boolean

Algorithm

```
call Logistics.tryWithdrawAdjacent()
if TowerBuilder.tryTowerRoutine() then
    return
endif
if Logistics.needRefillSoldier() then
    call Logistics.goRefill()
    return
endif
if G.rc.isActionReady() and Greedy.tryAttackEnemyTower() then
    call Greedy.tryExpandMove()
    return
endif
moved ← Greedy.tryExpandMove()
if not moved then
    moved ← Motion.moveToward(G.pushTarget, true)
    if not moved then
        call Motion.safeFallbackMove(true)
    endif
endif
if G.rc.isActionReady() then
    call Logistics.tryWithdrawAdjacent()
    if not Greedy.tryAttackEnemyRobotEfficient() then
        if not Greedy.tryAttackEnemyTower() then
            if not TowerBuilder.tryTowerRoutine() then
                call Greedy.tryBestPaintAttack()
            endif
        endif
    endif
endif
endif
```

procedure runMopper()


```
{ Menjalankan perilaku Mopper setiap ronde. }
```

Kamus:

```
closest : MapLocation
```

```
best, d, i : integer
```

Algorithm

```
if Logistics.tryTransferPaintToAlly() then
    return
endif

if Logistics.needRefillMopper() then
    call Logistics.goRefill()
    return
endif

if G.rc.isActionReady() and Greedy.tryCleanEnemyPaint() then
    return
endif

if Motion.followWeakAlly() then
    return
endif

if G.nearbyEnemyLen > 0 then
    closest ← null
    best ←  $+\infty$ 
    for i ← G.nearbyEnemyLen-1 downto 0 do
        d ← jarakKuadrat(G.me, G.nearbyEnemies[i].location)
        if d < best then
            best ← d
            closest ← G.nearbyEnemies[i].location
        endif
    endfor
    if closest ≠ null and Motion.moveToward(closest, false) then
        return
    endif
endif

if Motion.moveToward(G.pushTarget, false) then
    return
endif
```

```
endif  
call Motion.safeFallbackMove(false)
```

procedure runSplasher()

{ Menjalankan perilaku Splasher setiap ronde. }

Kamus:

moved : boolean

Algorithm

```
call Logistics.tryWithdrawAdjacent()  
if Logistics.needRefillSplasher() then  
    call Logistics.goRefill()  
    return  
endif  
if G.rc.isActionReady() and Greedy.tryAttackEnemyTower() then  
    return  
endif  
if G.rc.isActionReady() and Greedy.tryBestSplashAttack() then  
    return  
endif  
moved ← Greedy.tryContestMove()  
if not moved then  
    moved ← Motion.moveToward(G.pushTarget, false)  
endif  
if not moved then  
    call Motion.safeFallbackMove(false)  
    return  
endif  
if G.rc.isActionReady() then  
    call Logistics.tryWithdrawAdjacent()  
    if not Greedy.tryAttackEnemyTower() then  
        call Greedy.tryBestSplashAttack()  
    endif  
endif  
endif
```

h. Tower.java

procedure run()

{ Menjalankan perilaku tower. }

Kamus:

-

Algorithm

call tryAttack()

call trySpawn()

procedure tryAttack()

{ Menyerang musuh dengan skor tertinggi yang ada dalam jangkauan tower. }

Kamus:

best, e : RobotInfo

bestScore, score, i : integer

Algorithmbest $\leftarrow$ nullbestScore $\leftarrow -\infty$ for i $\leftarrow$ G.nearbyEnemyLen-1 downto 0 doe $\leftarrow$ G.nearbyEnemies[i]

if not G.rc.canAttack(e.location) then

continue

endif

if e.health $\leq$ 50 thenscore $\leftarrow$ 80else if e.health $\leq$ 150 thenscore $\leftarrow$ 40

else

score $\leftarrow$ 10

endif

if Util.isTower(e.type) then

score $\leftarrow$ score + 50

else if e.type = SOLDIER then

score $\leftarrow$ score + 25

```
    else if e.type = SPLASHER then
        score ← score + 20
    else
        score ← score + 10
    endif
    score ← score - jarakKuadrat(G.me, e.location)
    if score > bestScore then
        bestScore ← score
        best ← e
    endif
endfor
if best ≠ null and G.rc.canAttack(best.location) then
    G.rc.attack(best.location)
endif
```

procedure trySpawn()

{ Menentukan apakah tower akan spawn unit lalu menambah counter spawn. }

Kamus:

tp : integer

toBuild : UnitType

Algorithm

```
if not G.rc.isActionReady() then
    return
endif
tp ← G.rc.getPaint()
toBuild ← chooseUnit(tp)
if toBuild = null then
    return
endif
if tryBuild(toBuild) then
    if toBuild = SOLDIER then
        spawnedSoldiers ← spawnedSoldiers + 1
    else if toBuild = MOPPER then
```

```
spawnedMoppers  $\leftarrow$  spawnedMoppers + 1
else if toBuild = SPLASHER then
    spawnedSplashers  $\leftarrow$  spawnedSplashers + 1
endif
endif
```

function chooseUnit(input tp : integer) $\rightarrow$ UnitType

{ Memilih jenis unit yang paling layak dibangun.

Input: tp, jumlah paint tower saat ini }

Kamus:

wantMoppers, wantSplashers : integer

Algorithm

```
if G.round  $\leq$  40 then
    if tp  $\geq$  200 and G.chips  $\geq$  250 then
        return SOLDIER
    else
        return null
    endif
endif

wantMoppers  $\leftarrow$  maksimum(1, spawnedSoldiers div 3)
if spawnedSoldiers  $\geq$  3 then
    wantSplashers  $\leftarrow$  maksimum(1, spawnedSoldiers div 5)
else
    wantSplashers  $\leftarrow$  0
endif

if spawnedMoppers < wantMoppers and tp  $\geq$  100 and G.chips  $\geq$  300 then
    return MOPPER
endif

if spawnedSplashers < wantSplashers and tp  $\geq$  300 and G.chips  $\geq$  400 then
    return SPLASHER
endif

if tp  $\geq$  200 and G.chips  $\geq$  250 then
    return SOLDIER
```

```
endif  
return null
```

```
function tryBuild(input type : UnitType) → boolean
```

```
{ Memilih arah spawn terbaik untuk unit baru.
```

```
Input: type, tipe unit yang ingin dibangun }
```

```
Kamus:
```

```
bestDir, dir, toEnemy : Direction
```

```
bestScore, score, di : integer
```

```
nxt : MapLocation
```

```
info : MapInfo
```

```
p : PaintType
```

```
Algorithm
```

```
bestDir ← null
```

```
bestScore ←  $-\infty$ 
```

```
for di ← 7 downto 0 do
```

```
    dir ← G.DIRS[di]
```

```
    nxt ← G.me + dir
```

```
    if not G.rc.canBuildRobot(type, nxt) then
```

```
        continue
```

```
    endif
```

```
    info ← TowerBuilder.findTileInfo(nxt)
```

```
    if info ≠ null then
```

```
        p ← info.getPaint()
```

```
    else
```

```
        p ← EMPTY
```

```
    endif
```

```
    if p.isAlly() then
```

```
        score ← 20
```

```
    else if p = EMPTY then
```

```
        score ← 8
```

```
    else
```

```
        score ← -15
```

```
endif
score ← score - 4 * Greedy.countAdjacentAllies(nxt)
if G.inferredEnemyBase ≠ null then
    toEnemy ← arah dari G.me ke G.inferredEnemyBase
    if dir = toEnemy or dir = rotateLeft(toEnemy) or dir =
rotateRight(toEnemy) then
        score ← score + 15
    endif
endif
if score > bestScore then
    bestScore ← score
    bestDir ← dir
endif
endfor
if bestDir ≠ null then
    G.rc.buildRobot(type, G.me + bestDir)
    return true
endif
return false
```

function tryTowerRoutine() → **boolean**

{ Menjalankan seluruh routine pembangunan atau upgrade tower oleh Soldier. }

Kamus:

ruin : MapLocation

goalType : UnitType

Algorithm

if G.myType ≠ SOLDIER then

return false

endif

if G.paint < 35 then

return false

endif

if tryUpgradeAdjacentTower() then

```
        return true
    endif
    if cachedRuin  $\neq$  null and (G.round - cachedRound)  $\leq$  3 then
        ruin  $\leftarrow$  cachedRuin
        goalType  $\leftarrow$  cachedGoalType
    else
        ruin  $\leftarrow$  findBestNearbyRuin()
        if ruin = null then
            return false
        endif
        goalType  $\leftarrow$  chooseTowerType(ruin)
        cachedRuin  $\leftarrow$  ruin
        cachedGoalType  $\leftarrow$  goalType
        cachedRound  $\leftarrow$  G.round
    endif
    if G.rc.canCompleteTowerPattern(goalType, ruin) then
        G.rc.completeTowerPattern(goalType, ruin)
        cachedRuin  $\leftarrow$  null
        return true
    endif
    if tryPaintTowerPattern(ruin, goalType) then
        return true
    endif
    if Motion.moveToward(ruin, true) then
        return true
    endif
    return false
```

function tryUpgradeAdjacentTower() $\rightarrow$ **boolean**

{ Meng-upgrade tower level 1 yang berada di dekat Soldier. }

Kamus:

ally : RobotInfo


```
    i : integer
Algorithm
    if G.chips < 2600 then
        return false
    endif
    for i ← G.nearbyAllyLen-1 downto 0 do
        ally ← G.nearbyAllies[i]
        if not Util.isTower(ally.type) then
            continue
        endif
        if jarakKuadrat(G.me, ally.location) > 2 then
            continue
        endif
        if ally.type = LEVEL.ONE.PAINT.TOWER or ally.type = LEVEL.ONE.MONEY.TOWER
then
            if G.rc.canUpgradeTower(ally.location) then
                G.rc.upgradeTower(ally.location)
                return true
            endif
        endif
    endfor
    return false
```

```
function findBestNearbyRuin() → MapLocation
{ Mencari ruin terbaik di sekitar untuk dibangun tower. }
```

Kamus:

```
best, loc : MapLocation
bestScore, score, i : integer
info : MapInfo
onRuin : RobotInfo
```

Algorithm

```
best ← null
bestScore ←  $-\infty$ 
for i ← G.nearbyTileLen-1 downto 0 do
```

```
info ← G.nearbyTiles[i]
if not info.hasRuin() then
    continue
endif
loc ← info.getMapLocation()
if G.rc.canSenseLocation(loc) then
    onRuin ← G.rc.senseRobotAtLocation(loc)
else
    onRuin ← null
endif
if onRuin ≠ null and Util.isTower(onRuin.type) then
    continue
endif
score ← -3 * jarakKuadrat(G.me, loc)
score ← score + 6 * quickPatternCount(loc, chooseTowerType(loc))
if G.round ≤ 140 then
    score ← score + 30
else if G.round ≤ 650 then
    score ← score + 15
endif
if G.inferredEnemyBase ≠ null and jarakKuadrat(loc, G.inferredEnemyBase) ≤ 100 then
    score ← score + 20
endif
if score > bestScore then
    bestScore ← score
    best ← loc
endif
endfor
return best
```

```
function quickPatternCount(input ruin : MapLocation, input type : UnitType)
→ integer
```

{ Menghitung jumlah sel pola tower yang sudah benar di sekitar ruin.

Input: ruin, pusat ruin; type, tipe tower target }

Kamus:

pat : array[0..4][0..4] of boolean

correct, dx, dy, i : integer

tile : MapInfo

loc : MapLocation

ns : boolean

p : PaintType

Algorithm

pat $\leftarrow$ getPattern(type)

correct $\leftarrow$ 0

for i $\leftarrow$ G.nearbyTileLen-1 downto 0 do

 tile $\leftarrow$ G.nearbyTiles[i]

 loc $\leftarrow$ tile.getMapLocation()

 dx $\leftarrow$ loc.x - ruin.x

 dy $\leftarrow$ loc.y - ruin.y

 if dx < -2 or dx > 2 or dy < -2 or dy > 2 then

 continue

 endif

 if dx = 0 and dy = 0 then

 continue

 endif

 if tile.hasRuin() or tile.isWall() then

 continue

 endif

 ns $\leftarrow$ pat[dx+2][dy+2]

 p $\leftarrow$ tile.getPaint()

 if p.isAlly() and ((ns and p = ALLY_SECONDARY) or ((not ns) and p = ALLY_PRIMARY)) then

 correct $\leftarrow$ correct + 1

 endif

endfor

return correct

```
function chooseTowerType(input ruin : MapLocation) → UnitType
```

```
{ Menentukan tipe tower yang paling sesuai untuk ruin tertentu.
```

```
Input: ruin, lokasi ruin kandidat }
```

```
Kamus:
```

```
-
```

```
Algorithm
```

```
    if G.lastSeenEnemyTower ≠ null and jarakKuadrat(ruin, G.lastSeenEnemyTower) ≤  
36 then
```

```
        return LEVEL_ONE_DEFENSE_TOWER
```

```
    endif
```

```
    if G.inferredEnemyBase ≠ null and jarakKuadrat(ruin, G.inferredEnemyBase) ≤ 50  
then
```

```
        return LEVEL_ONE_DEFENSE_TOWER
```

```
    endif
```

```
    if G.round ≤ 80 then
```

```
        return LEVEL_ONE_MONEY_TOWER
```

```
    endif
```

```
    if G.round ≤ 600 then
```

```
        return LEVEL_ONE_PAINT_TOWER
```

```
    endif
```

```
    return LEVEL_ONE_MONEY_TOWER
```

```
function tryPaintTowerPattern(input ruin : MapLocation, input type :  
UnitType) → boolean
```

```
{ Mengecat sel pola tower yang paling menguntungkan terlebih dahulu.
```

```
Input: ruin, pusat ruin; type, tipe tower target }
```

```
Kamus:
```

```
    pat : array[0..4][0..4] of boolean
```

```
    progressBonus : integer
```

```
    bestLoc, loc : MapLocation
```

```
    bestSec, ns : boolean
```

```
    bestScore, dx, dy, score : integer
```

```
    info : MapInfo
```

```
cur : PaintType
```

Algorithm

```
if not G.rc.isActionReady() then
    return false
endif

pat ← getPattern(type)
progressBonus ← 4 * quickPatternCount(ruin, type)

bestLoc ← null
bestSec ← false
bestScore ←  $-\infty$ 

for dx ← -2 to 2 do
    for dy ← -2 to 2 do
        if dx = 0 and dy = 0 then
            continue
        endif
        loc ← ruin.translate(dx, dy)
        if not G.rc.canAttack(loc) then
            continue
        endif
        info ← findTileInfo(loc)
        if info = null or info.hasRuin() or info.isWall() then
            continue
        endif
        ns ← pat[dx+2][dy+2]
        cur ← info.getPaint()

        if cur.isAlly() and ((ns and cur = ALLY_SECONDARY) or ((not ns) and cur = ALLY_PRIMARY)) then
            continue
        endif
        score ← 50 + progressBonus
        if cur ≠ EMPTY and not cur.isAlly() then
            score ← score + 20
        endif
        score ← score - jarakKuadrat(G.me, loc)
```

```
        if score > bestScore then
            bestScore ← score
            bestLoc ← loc
            bestSec ← ns
        endif
    endfor
endfor
if bestLoc ≠ null then
    G.rc.attack(bestLoc, bestSec)
    G.paint ← G.paint - 5
    return true
endif
return false
```

function findTileInfo(input loc : MapLocation) → MapInfo

{ Mengambil MapInfo dari cache nearby tiles jika ada.

Input: loc, lokasi yang dicari }

Kamus:

i : integer

Algorithm

```
for i ← G.nearbyTileLen-1 downto 0 do
    if G.nearbyTiles[i].getMapLocation() = loc then
        return G.nearbyTiles[i]
    endif
endfor
return null
```

function getPattern(input type : UnitType) → array[0..4][0..4] of boolean

{ Mengambil pola 5×5 sesuai tipe tower.

Input: type, tipe tower }

Kamus:

-

Algorithm

```
if type = LEVEL.ONE_PAINT_TOWER then
    return G.paintPattern
endif
if type = LEVEL.ONE_DEFENSE_TOWER then
    return G.defensePattern
endif
return G.moneyPattern
```

4.2 Struktur Data dan Method

4.2.1 G.java

Table 182: Struktur Data di G.java

Nama	Penjelasan
RobotController rc	Referensi utama ke API Battlecode untuk semua aksi dan sensor robot.
int mapWidth, mapHeight, mapArea	Dimensi peta yang disimpan saat inisialisasi agar tidak perlu di-query ulang tiap ronde.
MapLocation mapCenter	Titik tengah peta, digunakan sebagai tujuan fallback saat robot tidak tahu harus ke mana.
Team team, opponentTeam	Referensi tim sendiri dan tim musuh agar perbandingan tim tidak perlu memanggil rc setiap saat.
int roundSpawned	Ronde saat robot ini pertama kali di-spawn, untuk menghitung usia robot.
Direction[] DIRECTIONS[8]	Array 8 arah mata angin (tanpa CENTER) untuk iterasi pergerakan dan scanning.
Direction[] ALL_DIRS[9]	Array 9 arah termasuk CENTER, jarang dipakai dalam kode saat ini.
MapLocation me	Lokasi robot saat ini, di-update setiap ronde agar tidak memanggil rc.getLocation() berulang.
int round	Nomor ronde saat ini, di-update di awal setiap giliran.
RobotInfo[] nearbyAllies	Cache daftar robot sekutu dalam jangkauan sensor, di-refresh tiap ronde.
RobotInfo[] nearbyEnemies	Cache daftar robot musuh dalam jangkauan sensor, di-refresh tiap ronde.
MapInfo[] nearbyMapInfos	Cache informasi tile-tile di sekitar robot (paint, mark, wall), di-refresh tiap ronde.

Table 182: Struktur Data di G.java

Nama	Penjelasan
<code>MapLocation[] nearbyRuins</code>	Cache daftar ruin yang terdeteksi di sekitar robot pada ronde ini.
<code>int[][] visited[30][30]</code>	Grid 30×30 yang mencatat ronde terakhir setiap sel 2×2 dikunjungi, untuk panduan eksplorasi.
<code>MapLocation lastKnownEnemyTower</code>	Lokasi terakhir tower musuh yang pernah terlihat, digunakan Soldier untuk mode ATTACK.
<code>int lastEnemyTowerRound</code>	Ronde saat <code>lastKnownEnemyTower</code> terakhir diperbarui, untuk menentukan apakah info masih relevan.
<code>int lastPaint</code>	Field yang dideklarasikan tapi tidak pernah digunakan di mana pun dalam kode.
<code>StringBuilder indicator</code>	Buffer teks indikator debug yang ditampilkan di atas robot setiap ronde.

Table 183: Function/Procedure di G.java

Function	Penjelasan
<code>void markVisited(MapLocation loc)</code>	Mencatat lokasi yang diberikan ke grid visited dengan nomor ronde saat ini.
<code>int getVisited(MapLocation loc)</code>	Mengembalikan ronde terakhir lokasi tersebut dikunjungi, atau 0 jika belum pernah.
<code>int cheby(MapLocation a, MapLocation b)</code>	Menghitung jarak Chebyshev (max dari selisih x dan y) antara dua lokasi.

4.2.2 RobotPlayer.java

Table 184: Function/Procedure di RobotPlayer.java

Function	Penjelasan
<code>void run(RobotController rc)</code>	Melakukan inisialisasi satu kali lalu menjalankan loop utama yang memanggil logika robot sesuai tipenya setiap ronde.
<code>void updateRound()</code>	Memperbarui semua field global di G (ronde, lokasi, cache sensor, ruin) setiap awal giliran.

4.2.3 Soldier.java

Table 185: Struktur Data di Soldier.java

Nama	Penjelasan
<code>int TOWER_BUILD_CUTOFF = 600</code>	Ronde batas setelah itu Soldier berhenti membangun tower dan beralih ke ekspansi murni.
<code>int EXPLORE=0 ...ATTACK=6</code>	Konstanta integer yang merepresentasikan tujuh mode state machine Soldier.
<code>int mode</code>	State machine aktif saat ini, menentukan perilaku Soldier pada ronde ini.
<code>MapLocation targetRuin</code>	Ruin yang sedang menjadi target pembangunan tower oleh Soldier ini.
<code>UnitType targetType</code>	Tipe tower yang direncanakan akan dibangun pada targetRuin saat ini.
<code>boolean markerPlaced</code>	Flag yang menandakan apakah pola tower sudah ditandai (mark) pada ruin target.
<code>int buildStartRound</code>	Ronde saat proses pembangunan tower dimulai, untuk mengukur timeout.
<code>int BUILD_TIMEOUT = 110</code>	Batas maksimal ronde yang diperbolehkan untuk menyelesaikan satu proses pembangunan tower.
<code>UnitType[] TOWER_TYPES[3]</code>	Array tiga tipe tower level-1 yang digunakan saat iterasi pengecekan dan penyelesaian pola.
<code>Direction[] SENTINEL[3]</code>	Arah tile sentinel per tipe tower untuk memverifikasi bahwa pola sudah di-mark sebelum diselesaikan.
<code>boolean[][][] towerPatterns</code>	Pola boolean 5×5 untuk ketiga tipe tower, di-fetch saat init untuk digunakan dalam painting.
<code>boolean[][] resourcePattern</code>	Pola boolean resource pattern yang di-fetch saat init (tersimpan tapi tidak digunakan langsung).
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi saat ini yang diperbarui saat target sudah tercapai atau kadaluarsa.
<code>int exploreAge</code>	Penghitung ronde sejak exploreTarget terakhir diperbarui, untuk timeout eksplorasi.
<code>int EXPLORE_TIMEOUT = 30</code>	Batas ronde sebelum exploreTarget dipilih ulang agar tidak stuck mengejar lokasi yang tidak bisa dicapai.

Table 186: Function/Procedure di Soldier.java

Function	Penjelasan
<code>void init()</code>	Mengambil pola tower dan resource dari rc, lalu menyetel seed PRNG robot ini.
<code>void run()</code>	Fungsi utama yang dijalankan tiap ronde: cek paint, jalankan transisi state, lalu eksekusi aksi mode aktif.
<code>void transitionExplore()</code>	Dari EXPLORE, mencari ruin terbaik untuk dibangun dan beralih ke GOTO_RUIN, atau ke ATTACK jika musuh terdeteksi.
<code>void transitionGotoRuin()</code>	Memvalidasi target ruin masih valid dan belum dicuri, lalu beralih ke MARK saat ruin sudah bisa disensor.
<code>void transitionMark()</code>	Menunggu <code>markerPlaced</code> menjadi true sebelum beralih ke mode PAINT.
<code>void transitionPaint()</code>	Memeriksa apakah pola sudah bisa diselesaikan dan beralih ke COMPLETE jika sudah siap.
<code>void transitionComplete()</code>	Mencoba menyelesaikan tower; jika gagal kembali ke mode PAINT.
<code>void transitionAttack()</code>	Kembali ke EXPLORE jika informasi tower musuh sudah terlalu lama atau tidak ada.
<code>void doExplore()</code>	Bergerak ke target eksplorasi dan mengecat tile di sekitar sepanjang jalan.
<code>void doGotoRuin()</code>	Bergerak menuju targetRuin sambil mengecat tile di sekitar sepanjang jalan.
<code>void doMark()</code>	Mendekati ruin dan memanggil <code>markTowerPattern</code> untuk menandai pola tower di peta.
<code>void doPaint()</code>	Bergerak ke tile pola yang paling mendesak dan mengecat tile sesuai pola tower.
<code>void doComplete()</code>	Mendekati ruin dan memanggil <code>tryCompleteTower</code> untuk menyelesaikan pembangunan tower.
<code>void doRetreat()</code>	Bergerak ke tower sekutu terdekat dan mengisi ulang paint dari tower tersebut.
<code>void doAttack()</code>	Bergerak ke lokasi tower musuh terakhir diketahui dan menyerangnya jika dalam jangkauan.
<code>boolean paintBestPatternTile()</code>	Memilih dan mengecat tile pola tower berprioritas tertinggi dalam jangkauan aksi.

Table 186: Function/Procedure di Soldier.java

Function	Penjelasan
<code>MapLocation findUrgentPatternTile()</code>	Menemukan tile pola yang belum dicat dan paling dekat dengan robot untuk dijadikan navigasi.
<code>boolean hasMopperNearby()</code>	Mengembalikan true jika ada Mopper sekutu dalam radius 8 dari posisi robot ini.
<code>void opportunisticComplete()</code>	Menyelesaikan pola tower mana pun yang sudah siap di sekitar robot tanpa harus dalam mode COMPLETE.
<code>boolean tryCompleteTower()</code>	Mencoba menyelesaikan tower pada targetRuin dengan memverifikasi sentinel tile terlebih dahulu.
<code>MapLocation findBestRuinToBuild()</code>	Memilih ruin terbaik untuk dibangun berdasarkan jarak, jumlah cat musuh, dan status klaim.
<code>int countEnemyPaint(MapLocation loc, int radiusSq)</code>	Menghitung jumlah tile bercat musuh dalam radius tertentu dari lokasi yang diberikan.
<code>MapLocation pickGreedyExploreTarget()</code>	Memilih target eksplorasi terbaik secara greedy dengan memprioritaskan ruin netral dan sel yang belum dikunjungi.
<code>void paintNearby()</code>	Mengecat tile terbaik dalam jangkauan aksi dengan memprioritaskan tile sendiri, tile kosong, lalu tile musuh.
<code>void tryCompleteResourcePatterns()</code>	Mencoba menyelesaikan resource pattern di semua tile dalam radius 2 dari posisi robot.
<code>void tryWithdrawPaint()</code>	Menarik paint dari tower paint terdekat jika robot berada dalam jangkauan dan paint belum penuh.
<code>MapLocation findNearestAllyTowerLoc()</code>	Menemukan tower sekutu terdekat dengan bonus preferensi pada paint tower untuk retreat.
<code>void checkRuinStolen()</code>	Memeriksa apakah ruin target sudah diambil musuh atau sekutu lain dan memperbarui state sesuai hasilnya.
<code>void abandonBuild()</code>	Mereset semua variabel build dan kembali ke mode EXPLORE.
<code>String modeStr()</code>	Mengembalikan string singkat mode aktif untuk ditampilkan di indikator debug.

4.2.4 Splasher.java

Table 187: Struktur Data di .java

Nama	Penjelasan
<code>int EXPLORE=0, RETREAT=1</code>	Dua konstanta mode state machine Splasher: eksplorasi aktif atau mundur ke tower.
<code>int mode</code>	Mode aktif Splasher saat ini.
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi yang diperbarui secara berkala.
<code>int exploreAge</code>	Penghitung ronde sejak <code>exploreTarget</code> terakhir diganti.
<code>int EXPLORE.TIMEOUT = 25</code>	Batas ronde sebelum <code>exploreTarget</code> Splasher dipilih ulang.

Table 188: Function/Procedure di `Splasher.java`

Function	Penjelasan
<code>void init()</code>	Menyetel seed PRNG unik untuk robot Splasher ini.
<code>void run()</code>	Fungsi utama tiap ronde: cek paint, mode, complete pattern, lalu eksekusi aksi.
<code>void doExplore()</code>	Bergerak ke target terbaik dan melakukan splash attack pada lokasi paling menguntungkan.
<code>void bestSplash()</code>	Memilih pusat splash yang memberikan skor tertinggi berdasarkan jumlah tile kosong dan cat musuh dalam radius ledakan.
<code>MapLocation pickMoveTarget()</code>	Menilai 8 arah gerak dan memilih yang paling menguntungkan menuju area belum dikunjungi.
<code>MapLocation pickGreedyTarget()</code>	Memilih target eksplorasi acak terbaik dari 8 kandidat berdasarkan skor kunjungan dan jarak.
<code>void doRetreat()</code>	Bergerak ke paint tower terdekat dan menarik paint saat sudah berada dalam jangkauan.
<code>void tryWithdrawPaint()</code>	Menarik paint dari paint tower sekutu terdekat jika berada dalam jangkauan dan kapasitas belum penuh.
<code>void tryCompletePatterns()</code>	Menyelesaikan pola tower dan resource pattern yang sudah siap di sekitar robot secara oportunistik.
<code>MapLocation findNearestPaintTowerLoc()</code>	Mencari paint tower sekutu yang paling dekat dari posisi robot saat ini.

4.2.5 Mopper.java

Table 189: Struktur Data di Mopper.java

Nama	Penjelasan
<code>int EXPLORE=0, SUPPORT=1, RETREAT=2</code>	Tiga konstanta mode state machine Mopper.
<code>int mode</code>	Mode aktif Mopper saat ini.
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi Mopper yang diperbarui secara berkala.
<code>int exploreAge</code>	Penghitung ronde sejak <code>exploreTarget</code> Mopper terakhir diganti.
<code>int EXPLORE_TIMEOUT = 20</code>	Batas ronde sebelum <code>exploreTarget</code> Mopper dipilih ulang.

Table 190: Function/Procedure di Mopper.java

Function	Penjelasan
<code>void init()</code>	Menyetel seed PRNG unik untuk robot Mopper ini.
<code>void run()</code>	Fungsi utama tiap ronde: cek paint untuk retreat, complete pattern, lalu eksekusi mode aktif.
<code>void doExplore()</code>	Mengecek ally yang butuh paint, bergerak ke area cat musuh terpadat, dan melakukan aksi mop.
<code>Direction pickMopDirection()</code>	Memilih arah gerak terbaik dengan menghitung skor berdasarkan kepadatan cat musuh dan robot musuh di sekitarnya.
<code>void doMopAction()</code>	Memilih aksi terbaik: swing jika banyak musuh dalam satu garis, atau mop tile/robot musuh terbaik.
<code>Direction findBestSwing()</code>	Menemukan arah kardinal terbaik untuk swing attack yang mengenai paling banyak robot musuh.
<code>RobotInfo findNeedyAlly()</code>	Mencari robot sekutu dalam radius 2 yang paint-nya di bawah 30% kapasitas untuk diberi support.
<code>void doSupport()</code>	Mendekati ally yang butuh paint dan mentransfer paint kepadanya.
<code>void doRetreat()</code>	Menarik paint dari paint tower terdekat atau bergerak menuju paint tower untuk isi ulang.
<code>void tryCompletePatterns()</code>	Menyelesaikan pola tower yang sudah siap di sekitar robot secara oportunistik.

Table 190: Function/Procedure di Mopper.java

Function	Penjelasan
<code>MapLocation findNearestPaintTowerLoc()</code>	Mencari paint tower sekutu yang paling dekat dari posisi Mopper saat ini.
<code>String modeStr()</code>	Mengembalikan string singkat mode aktif Mopper untuk indikator debug.

4.2.6 Tower.java

Table 191: Struktur Data di Tower.java

Nama	Penjelasan
<code>int PHASE2_ROUND = 601</code>	Ronde di mana strategi spawn beralih ke mode agresif fase 2 (dominasi Splasher).
<code>double W_SOLDIER.BASE = 1.6</code>	Bobot dasar Soldier dalam kalkulasi proporsi spawn fase 1.
<code>double W_SPLASHER.BASE = 0.5</code>	Bobot dasar Splasher dalam kalkulasi proporsi spawn fase 1.
<code>double W_MOPPER.BASE = 0.9</code>	Bobot dasar Mopper dalam kalkulasi proporsi spawn fase 1.
<code>double dSoldier, dSplasher, dMopper</code>	Akumulator desimal untuk menyeimbangkan jumlah spawn aktual vs target proporsi setiap tipe unit.
<code>int spawnedTotal, spawnedSoldiers, ...</code>	Counter jumlah unit yang sudah di-spawn oleh tower ini per tipe dan total.
<code>int lastSpawnRound</code>	Ronde terakhir tower berhasil men-spawn unit, untuk logika throttling spawn.
<code>MapLocation[] spawnLocs</code>	Array lokasi spawn yang sudah diurutkan dari yang terjauh dari pusat peta ke terdekat.
<code>int towerLevel</code>	Level upgrade tower saat ini, bertambah tiap kali <code>upgradeTower</code> berhasil dipanggil.

Table 192: Function/Procedure di Tower.java

Function	Penjelasan
<code>void init()</code>	Membangun daftar lokasi spawn dan mendaftarkan tower ini ke MapMemory.

Table 192: Function/Procedure di Tower.java

Function	Penjelasan
<code>void run()</code>	Fungsi utama tiap ronde: putuskan unit yang di-spawn, lakukan spawn, serang musuh, dan coba upgrade.
<code>UnitType decideSpawnPhase1()</code>	Memilih tipe unit terbaik untuk di-spawn di fase 1 berdasarkan bobot dinamis dan jumlah tower yang ada.
<code>UnitType decideSpawnPhase2()</code>	Memilih tipe unit untuk fase 2 dengan pola sederhana: setiap unit ke-7 adalah Mopper, sisanya Splasher.
<code>boolean shouldSpawn(UnitType t, boolean isPhase2)</code>	Menentukan apakah tower boleh spawn sekarang berdasarkan jumlah chip, jumlah tower, dan fase permainan.
<code>boolean trySpawn(UnitType type)</code>	Mencoba membangun robot di salah satu lokasi spawn yang tersedia dan memperbarui counter.
<code>void doAttack()</code>	Melakukan serangan AoE otomatis lalu menyerang musuh berprioritas tertinggi dalam jangkauan.
<code>void tryUpgrade()</code>	Mengupgrade tower jika chip mencukupi threshold dengan buffer sesuai fase permainan.
<code>MapLocation[] buildSpawnLocs()</code>	Membuat dan mengurutkan array 12 lokasi potensial spawn di sekitar tower berdasarkan jarak dari pusat peta.

4.2.7 Nav.java

Table 193: Struktur Data di Nav.java

Nama	Penjelasan
<code>Direction bugDir</code>	Arah dinding yang sedang diikuti dalam algoritma bug navigation, null jika sedang jalan lurus.
<code>int bugRot</code>	Arah rotasi bug nav: +1 searah jarum jam, -1 berlawanan jarum jam.
<code>int bugSteps</code>	Jumlah langkah yang sudah dilakukan dalam mode bug nav untuk deteksi timeout.

Table 193: Struktur Data di Nav.java

Nama	Penjelasan
MapLocation bugTarget	Target terakhir yang digunakan bug nav untuk mendeteksi perubahan tujuan.
int bugStartDist	Jarak kuadrat ke target saat bug nav dimulai, untuk mendeteksi kemajuan.
int BUG_RESET = 20	Batas langkah maksimal sebelum bug nav di-reset agar tidak berputar terus.

Table 194: Function/Procedure di Nav.java

Function	Penjelasan
void moveToward(MapLocation target)	Menggerakkan robot menuju target dengan bug navigation untuk menghindari rintangan.
void moveAway(MapLocation avoid)	Menggerakkan robot menjauhi lokasi yang diberikan dengan mencoba arah berlawanan dan variannya.
void spreadFromAllies()	Menggerakkan robot menjauh dari kerumunan ally dengan menghitung vektor repulsi.
boolean tryMove(Direction d)	Mencoba bergerak ke arah tertentu, memperbarui G.me dan visited jika berhasil, mengembalikan true/false.
void greedyExploreTo(MapLocation target)	Bergerak menuju target dengan memilih arah yang memaksimalkan kemajuan sekaligus mengutamakan sel belum dikunjungi.
boolean paintUnderSelf()	Mengecat tile di bawah robot sendiri jika tile tersebut kosong atau bercat musuh.
MapLocation pickExploreTarget()	Memilih target eksplorasi terbaik dari 8 kandidat acak berdasarkan skor kunjungan dan jarak.

4.2.8 MapMemory.java

Table 195: Struktur Data di MapMemory.java

Nama	Penjelasan
int MAX_RUINS = 100	Batas maksimal ruin/tower yang bisa disimpan dalam memori.
int numRuins	Jumlah ruin/tower yang saat ini tersimpan dalam array.

Table 195: Struktur Data di MapMemory.java

Nama	Penjelasan
<code>MapLocation[] ruinLocs[100]</code>	Array lokasi semua ruin dan tower yang pernah ditemukan.
<code>Team[] ruinTeams[100]</code>	Array tim pemilik setiap ruin/tower yang diketahui.
<code>UnitType[] ruinTypes[100]</code>	Array tipe tower untuk setiap ruin/tower yang sudah dibangun.
<code>int[] ruinStatus[100]</code>	Status setiap ruin: 0=netral, 1=milik sekutu, 2=milik musuh.
<code>int[] ruinClaimRound[100]</code>	Ronde saat robot mengklaim ruin untuk dibangun, untuk mencegah klaim ganda dalam 80 ronde.
<code>int allyPaintTowers</code>	Jumlah paint tower sekutu yang diketahui saat ini.
<code>int allyMoneyTowers</code>	Jumlah money tower sekutu yang diketahui saat ini.
<code>int allyDefenseTowers</code>	Jumlah defense tower sekutu yang diketahui saat ini.
<code>int totalAllyTowers</code>	Total semua tower sekutu yang diketahui saat ini.
<code>boolean[] symmetry[3]</code>	Tiga flag simetri peta yang masih mungkin berlaku: horizontal, vertikal, dan rotasional.
<code>int[][] towerGrid[12][12]</code>	Grid indeks 12×12 untuk lookup cepat apakah sudah ada ruin pada sel 5×5 tertentu.

Table 196: Function/Procedure di MapMemory.java

Function	Penjelasan
<code>void registerRuin(MapLocation loc, Team t, UnitType u)</code>	Menambahkan atau memperbarui data ruin/tower pada lokasi yang diberikan dan menyetel ulang hitungan tower.
<code>boolean tryClaimRuin(MapLocation loc)</code>	Mencoba mengklaim ruin netral untuk dibangun; gagal jika sudah diklaim dalam 80 ronde terakhir.
<code>int teamToStatus(Team t)</code>	Mengonversi objek Team menjadi integer status (0=netral, 1=sekutu, 2=musuh).
<code>void recomputeAllyCount()</code>	Menghitung ulang semua counter tower sekutu per tipe dari awal berdasarkan data <code>ruinTeams</code> dan <code>ruinTypes</code> .
<code>UnitType chooseTowerType()</code>	Memilih tipe tower terbaik untuk dibangun berikutnya berdasarkan rasio paint/money/defense yang ada.

Table 196: Function/Procedure di MapMemory.java

Function	Penjelasan
<code>void updateSymmetry(MapLocation loc, boolean isWall)</code>	Placeholder untuk memperbarui estimasi simetri peta berdasarkan tile yang dilihat, saat ini belum diimplementasi.
<code>MapLocation getEnemyMirror(MapLocation allyLoc)</code>	Memprediksi posisi tower musuh dengan mencerminkan posisi tower sekutu berdasarkan simetri yang paling mungkin.
<code>void scanNearby()</code>	Memindai semua ruina dan robot musuh di sekitar robot lalu mendaftarkannya ke memori peta.
<code>MapLocation closestNeutralRuina(MapLocation from)</code>	Mencari ruina netral terdekat dari titik yang diberikan berdasarkan jarak kuadrat.
<code>MapLocation closestEnemyTower(MapLocation from)</code>	Mencari tower musuh terdekat yang diketahui dari titik yang diberikan berdasarkan jarak kuadrat.

4.2.9 Rand.java

Table 197: Struktur Data di Rand.java

Nama	Penjelasan
<code>int state = 12345</code>	Seed dan state internal PRNG xorshift32 yang diubah setiap pemanggilan <code>next()</code> .

Table 198: Function/Procedure di Rand.java

Function	Penjelasan
<code>int next()</code>	Menghasilkan bilangan pseudorandom positif berikutnya menggunakan algoritma xorshift32.
<code>int nextInt(int bound)</code>	Mengembalikan bilangan pseudorandom dalam rentang $[0, bound)$ dengan memanggil <code>next()</code> lalu modulo.

4.3 Pengujian

Main Bot vs Alternative 1 Bot

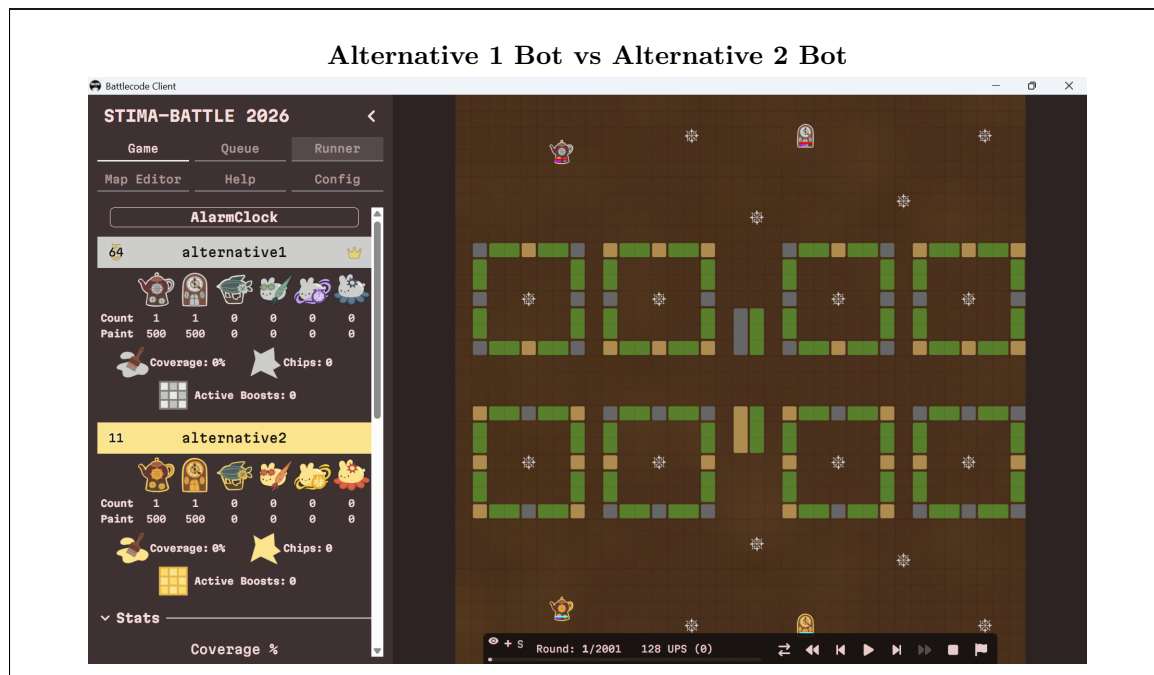
The screenshot shows the Battlecode Client interface for a match between 'main' and 'alternative1'. The left sidebar displays the game status, including the 'AlarmClock' timer, the 'main' bot's score (61), and the 'alternative1' bot's score (14). The main area shows the game map with various obstacles and bots. The bottom status bar indicates the round is 1/2001 and the UPS is 0.

Bot	Count	Paint	Coverage	Chips	Active Boosts
main	1	500	0%	0	0
alternative1	1	500	0%	0	0

Main Bot vs Alternative 2 Bot

The screenshot shows the Battlecode Client interface for a match between 'main' and 'alternative2'. The left sidebar displays the game status, including the 'AlarmClock' timer, the 'main' bot's score (75), and the 'alternative2' bot's score (14). The main area shows the game map with various obstacles and bots. The bottom status bar indicates the round is 1/716 and the UPS is 0.

Bot	Count	Paint	Coverage	Chips	Active Boosts
main	1	500	0%	0	0
alternative2	1	500	0%	0	0



4.4 Analisis Hasil

4.4.1 Main Bot

Arsitektur paling terstruktur: State Machine 7 state pada Soldier, MapMemory terpusat, phase switching deterministik di ronde 600, dan spawn-location sorting ke arah map center.

Arsitektur & Alur Kontrol

Soldier menjalankan state machine eksplisit. Tower membagi spawn ke dua phase (build vs flood). Mopper memiliki EXPLORE, SUPPORT, dan RETREAT mode. Splasher menggunakan greedy area scoring.

Tipe	Faktor	Analisis & Dampak
kekuatan	State Machine 7 State (Soldier)	Alur kontrol eksplisit: EXPLORE → GOTO_RUIN → MARK → PAINT → COMPLETE → RETREAT → ATTACK. Setiap transisi memiliki kondisi penjaga (guard) tersendiri sehingga tidak ada state ambiguitas. BUILD_TIMEOUT 110 ronde mencegah deadlock, jika ruin tidak bisa diselesaikan, soldier segera cari ruin lain.
kekuatan	MapMemory Terpusat	ruinClaimRound tracking mencegah beberapa soldier mengklaim ruin yang sama secara bersamaan. towerGrid (resolusi 5×5) memungkinkan lookup O(1) per ruin. recomputeAllyCount dipanggil setiap kali tower status berubah, data selalu fresh.

Tipe	Faktor	Analisis & Dampak
kekuatan	Phase Switching Deterministik	Ronde 1-600 = build tower. Ronde 601+ = flood splasher. Batas tegas memudahkan prediksi & tuning. Phase 2 pola 6 splasher + 1 mopper memberikan tekanan teritorial masif.
kekuatan	Opportunistic Complete	Robot APAPUN (soldier, splasher, mopper) yang berada dekat ruin dengan pattern siap langsung memanggil completeTowerPattern. Meminimalkan turn yang terbuang, tidak perlu menunggu soldier yang sedang PAINT selesai.
kekuatan	Spawn Location Sorted ke Center	buildSpawnLocs() mengurutkan 12 posisi spawn berdasarkan jarak ke map center (terjauh dulu). Robot langsung bergerak menjauhi tower ke arah tengah/musuh sejak lahir.
kelemahan	ATTACK Mode Single Target	lastKnownEnemyTower hanya menyimpan 1 lokasi, hanya tower musuh terakhir terlihat yang dikejar. Jika ada 3 enemy tower di lokasi berbeda, hanya 1 yang diserang, 2 lainnya dibiarkan. Mode ATTACK kadaluarsa setelah 80 ronde tanpa sighting baru → kembali ke EXPLORE tanpa benar-benar menyerang.
kelemahan	Explore Sampling Hanya 8 Titik	pickGreedyExploreTarget melakukan random sampling 8 titik saja per turn. Di peta 60×60 (3600 tile), coverage eksplorasi sangat acak, bisa ada area kosong besar yang lama tidak dijelajahi. Beberapa soldier bisa tiba di area yang sama karena target random bisa berulang.
kelemahan	Splasher Phase 2 Rentan Counter Mopper	Rasio 6 splasher : 1 mopper di phase 2 terlalu condong ke splasher. Jika musuh mengirim banyak mopper, splasher kehilangan paint dengan cepat (mopper enemy menyedot paint dari splasher). Splasher tidak memiliki logika melarikan diri dari mopper enemy.
kelemahan	Tidak Ada Inter- Robot Communication	Tidak ada messaging (rc.sendMessage) antar robot. 5 soldier bisa semua melihat ruin yang sama pada ronde yang sama, 4 akan waste turn karena ruinClaimRound baru di-update setelah 100 ronde. Dalam kondisi ruin cluster, ini menyebabkan bottleneck signifikan.
kelemahan	Inferensi Simetri Tidak Adaptif	getEnemyMirror memprioritaskan horizontal reflection dan tidak pernah mengoreksinya. Jika peta rotasional, semua estimasi posisi musuh salah sampai robot benar-benar melihat tower musuh.

Heuristik Greedy

Heuristik utama difokuskan pada pemilihan ruin (score berbasis ancaman), pemilihan tower type (rasio dinamis), dan paint action (ekspansi sambil jalan).

Tipe	Faktor	Analisis & Dampak
kekuatan	findBestRuinToBuild, Score Berbasis Ancaman	$\text{score} = 1500 - \text{distSq} - \text{enemyPaint} \times 12$. Faktor enemyPaint memberikan penalti progresif: ruin dengan 10 tile cat musuh di sekitarnya dapat penalti 120 poin. Hard cutoff: ruin dengan enemyPaint > 12 di radius 12 langsung di-skip, mencegah soldier masuk ke area berbahaya. Fallback ke MapMemory untuk ruin yang sudah lama tidak diklaim (>100 ronde) dengan bonus 1000.
kekuatan	chooseTowerType, Rasio Dinamis	Prioritas: (1) paint tower jika belum ada, (2) money tower jika ratio money < 70% dari paint, (3) defense tower jika enemy terlihat < 60 ronde lalu. Sistem rasio (paintRatio < 0.45 → build paint, moneyRatio < 0.35 → build money) menciptakan keseimbangan ekonomi otomatis. Lebih responsif dari threshold tetap karena menyesuaikan dengan jumlah tower yang sudah ada.
kekuatan	paintNearby, Ekspansi Teritorial Sambil Jalan	Dipanggil di mode EXPLORE dan GOTO RUIN, soldier tidak pernah kosong tangan saat berjalan. Score: 120 (tile sendiri kosong), 100 (tile kosong), 80 (tile cat musuh) – distSq. Memastikan setiap langkah soldier selalu berkontribusi ke ekspansi wilayah.
kekuatan	paintBestPatternTile, Prioritas Berdasarkan Konteks	Priority 3: tile pattern kosong (tertinggi). Priority 2: tile cat musuh tanpa mopper nearby, langsung cat sendiri. Priority 1: tile cat musuh jika ada mopper nearby DAN build baru dimulai (<40 ronde), biarkan mopper bersihkan dulu. Koordinasi implisit antara soldier dan mopper tanpa messaging.
kelemahan	Mopper Retreat Threshold Terlalu Rendah	Mopper retreat hanya saat paint < 20 (dari kapasitas yang tidak disebutkan eksplisit, diasumsikan standar). Nilai ini sangat rendah, mopper bisa sudah jauh dari tower saat paint mencapai 20, tidak sempat refill. Bandingkan: Alt2 memopper retreat di paint ≤ 20 ATAU ≤ 35 jika jauh dari supply.
kelemahan	hasMopperNearby, Detection Radius Terlalu Kecil	Deteksi mopper ally dalam radius $\text{distSq} \leq 8$ (≈ 2.8 tile). Mopper yang bergerak cepat bisa keluar radius ini dalam 1–2 turn, koordinasi cat ulang tile musuh tidak konsisten.

Kondisi Map

Kondisi map yang menguntungkan dan merugikan berdasarkan karakteristik unik algoritma Main Bot.

Tipe	Faktor	Analisis & Dampak
kekuatan	Peta Besar & Banyak Ruin Tersebar	MapMemory dapat memanfaatkan seluruh database ruin. Phase 1 yang panjang (600 ronde) terbayar dengan jaringan tower yang luas. Splasher flood phase 2 sangat efektif mengisi area peta yang besar.
kekuatan	Peta Simetri Horizontal	getEnemyMirror memprioritaskan horizontal reflection, estimasi posisi musuh akurat. Soldier mode ATTACK langsung menuju lokasi yang benar.
kelemahan	Peta Kecil ($< 30 \times 30$)	Phase 2 di ronde 600 mungkin terlambat, game bisa sudah berakhir atau musuh sudah dominan lebih awal. BUILD_TIMEOUT 110 ronde sangat lama di peta kecil di mana ruin berjarak dekat.
kelemahan	Peta dengan Ruin Cluster di Pusat	enemyPaint > 12 di sekitar ruin center menyebabkan findBestRuinToBuild men-skip semua ruin strategis. Soldier tidak membangun di posisi paling penting pada peta jenis ini.
kelemahan	Peta Simetri Rotasional	Horizontal mirror diprioritaskan padahal salah, semua soldier/splasher menuju posisi yang keliru. Koreksi hanya terjadi saat robot benar-benar melihat enemy tower dalam sense range.
kelemahan	Banyak Obstacle / Dinding	Nav.moveToward dipanggil di semua mode tapi tidak ada bug-nav eksplisit terlihat. Splasher dan mopper bisa terjebak di belakang dinding panjang tanpa bisa memutar.

4.4.2 Alternative 1

Arsitektur berbasis phase round. Tidak ada MapMemory, tidak ada state machine. Soldier agresif sejak ronde 1. Kelemahan kritis: tidak ada money tower dan tidak ada paint management.

Arsitektur & Alur Kontrol

Soldier dibagi tiga fase (runEarly/runMid/runLate). Tower alternating soldier $\leftrightarrow$ splasher/mopper. Mopper hanya runLate aktif.

Tipe	Faktor	Analisis & Dampak
kekuatan	Agresivitas Soldier Sejak Ronde 1	runEarly: soldier langsung bergerak ke symmetry target (estimasi posisi enemy base) sejak lahir. tryAttackEnemyTower dipanggil di AWAL setiap turn, jika enemy tower dalam range, langsung serang tanpa delay. Jika berhasil merusak spawn tower musuh di ronde 1–50, keseluruhan army musuh terhambat.
kekuatan	Bug Navigation Solid (Motion.java)	moveTowardLocation mengimplementasikan wall tracing (bug navigation) dengan stuck detection (3 turn tanpa gerak). Ganti sisi tracing secara otomatis saat stuck ≥ 3 turn, tidak stuck permanen di obstacle apapun. getBestDirection memperhitungkan: jarak ke target ($-10 \times \text{distSq}$), spread dari ally ($+\text{distSq}$), dekati enemy ($-\text{distSq}$), hindari wall (-50), preferensi paint (± 20).
kekuatan	Spawn Location Scoring di Tower	getBestSpawnLocation: $\text{score} = -\text{distSq}(\text{center}) + \sum \text{distSq}(\text{ally}) - \sum \text{distSq}(\text{enemy})$. Robot dispawn: mendekati center, menjauhi ally yang sudah ada (spread), mendekati enemy yang terlihat. Mencegah robot numpuk di satu spot.
kekuatan	Bytecode Rendah, Stabilitas Tinggi	Logika sederhana berarti konsumsi bytecode per turn rendah. Tidak ada iterasi kompleks atau nested loop berat. Robot tidak pernah overflow bytecode, eksekusi selalu lengkap setiap turn.
kelemahan	TIDAK ADA Money Tower (Fatal)	buildRuin hardcoded: SELALU membangun LEVEL.ONE.PAINT.TOWER. Tidak ada income chips dari money tower, bot tidak bisa upgrade tower maupun spawn agresif di late game. Di ronde 500+ saat musuh sudah punya 5+ money tower, gap chips income bisa 5–10 $\times$ lipat. Ini adalah kelemahan paling kritis yang membuat Alt1 kalah secara sistematis di game panjang.
kelemahan	TIDAK ADA Paint Management / Retreat	Tidak ada threshold paint yang memicu retreat ke tower untuk refill. Soldier, mopper, dan splasher bisa terus bergerak hingga paint = 0 dan menjadi tidak berguna. Robot yang kehabisan paint masih berjalan tapi tidak bisa mewarnai tile apapun, pure waste movement.

Tipe	Faktor	Analisis & Dampak
kelemahan	TIDAK ADA MapMemory / Shared State	Setiap robot tidak tahu tower mana sudah dibangun oleh robot lain. Beberapa soldier bisa mencoba build ruin yang sama secara bersamaan, salah satunya pasti fail. Tidak ada koordinasi tower type, semua orang build paint tower tanpa mempertimbangkan kebutuhan tim.
kelemahan	Splasher Hanya Cat 1 Tile per Turn	runEarly: rc.attack(rc.getLocation()), hanya mewarnai tile posisi splasher sendiri. Splasher yang mahal (biaya lebih tinggi dari soldier) pada dasarnya berfungsi sebagai soldier lambat. Kekuatan utama splasher (area splash radius 2) tidak pernah digunakan di early game.
kelemahan	Mopper Hanya runLate (Code Phase Lain Di-comment)	Seluruh logika runEarly dan runMid mopper di-comment out. Mopper hanya: followNearestAllySoldier → moveTowardEnemyPaint → mopSwing/attack. Tidak ada swing scoring, tidak ada support transfer paint, tidak ada retreat.

Heuristik Greedy

Heuristik utama: symmetry target berbasis ID%3, buildRuin hardcoded paint tower, motion scoring getBestDirection.

Tipe	Faktor	Analisis & Dampak
kekuatan	getSymmetryTarget, Distribusi Eksplorasi Otomatis	Setiap soldier menggunakan simetri berbeda berdasarkan ID%3: horizontal, vertikal, atau rotasional. Secara tidak sengaja mendistribusikan soldier ke 3 estimasi posisi musuh yang berbeda. Salah satunya pasti mendekati posisi yang benar pada peta apapun.
kekuatan	moveTowardCorner, Anti-Stuck Berbasis Penjelajahan	Splasher yang tidak menemukan target bergerak ke corner terjauh, lalu cycle ke corner berikutnya. Memastikan splasher menutup seluruh area peta dari corner ke corner secara sistematis. Step berbeda (1 atau 3) berdasarkan ID%2, dua splasher tidak cycle corner dengan pola yang sama.
kelemahan	buildRuin, Tower Type Tidak Pernah Dinilai	Tidak ada evaluasi tipe tower apa yang dibutuhkan tim saat ini. LEVEL_ONE_PAINT_TOWER selalu dipilih terlepas dari situasi ekonomi, jumlah tower, atau ancaman musuh. Tidak ada defense tower → sangat rentan terhadap serangan ke tower sendiri.

Tipe	Faktor	Analisis & Dampak
kelemahan	tryAttackEnemyTower, Tidak Ada Prioritisasi Target	Iterasi enemy list, attack enemy tower PERTAMA yang bisa diserang, tidak ada scoring HP atau jarak. Bisa menyerang tower yang HP-nya masih tinggi padahal ada tower lain yang hampir hancur. Tower attack dari Alt1 kurang efisien per aksi.
kelemahan	paintCurrentTileIfNeeded, Ekspansi Minimal	Hanya mewarnai tile posisi robot sendiri jika belum dicat ally. Tidak mewarnai tile sekitar yang kosong atau tile cat musuh meskipun dalam range. Kontribusi ekspansi teritorial per soldier sangat kecil.

Kondisi Map

Kondisi map yang menguntungkan dan merugikan berdasarkan karakteristik unik algoritma Alternative 1.

Tipe	Faktor	Analisis & Dampak
kekuatan	Peta Kecil & Jarak Spawn ke Enemy Base Pendek	Soldier mencapai enemy base dalam ronde 10-30, rush bisa efektif sebelum musuh membangun army. Di peta $\leq 20 \times 20$, agresivitas awal Alt1 bisa langsung game-deciding.
kekuatan	Enemy Spawn Tower dengan HP Rendah di Ronde Awal	Jika desain game memberikan tower HP rendah di awal, soldier rush Alt1 bisa destroy sebelum musuh spawn unit. Ini menciptakan snowball effect: musuh tanpa tower = tidak bisa spawn unit = Alt1 menang mudah.
kelemahan	Peta Besar ($> 40 \times 40$)	Soldier butuh 30-60+ ronde hanya untuk mencapai musuh, selama itu musuh membangun economy dan army. Alt1 tidak melakukan apa-apa yang produktif selama perjalanan panjang ini (tidak build tower, tidak expand territory).
kelemahan	Game Panjang (> 800 Ronde)	Tanpa money tower, Alt1 kehabisan chips untuk spawn unit baru di late game. Musuh dengan money tower bisa terus regenerasi army sementara Alt1 tidak. Semakin lama game berlangsung, semakin lebar gap antara Alt1 dan bot lain.
kelemahan	Peta dengan Banyak Ruin di Center (Contested Zone)	Alt1 baru build tower di ronde 1001+ (runLate). Ruin-strategis di center sudah diambil musuh jauh sebelum Alt1 mulai membangun. Ekspansi teritorial Alt1 sangat terlambat.

4.4.3 Alternative 2

Arsitektur greedy reaktif murni. Tidak ada hard phase, semua adaptif. Unggul di paint management, anti-stuck, dan mopper support system. Kelemahan: simetri inference kaku (hanya rotasional).

Arsitektur & Alur Kontrol

Soldier menjalankan pipeline prioritas tanpa state machine. Mopper paling lengkap dari ketiga bot. Splasher balanced dengan threshold konservatif.

Tipe	Faktor	Analisis & Dampak
kekuatan	inferredEnemyBase dari Ronde Pertama	Saat init, setiap robot langsung menghitung rotational mirror: $(w-1-x, h-1-y)$. pushTarget langsung mengarah ke enemy base estimasi, seluruh army bergerak terarah sejak lahir. Tidak perlu melihat musuh dulu untuk bergerak ke arah yang benar (pada peta rotasional).
kekuatan	Anti-Stuck Scoring Granular (G.scoreMove)	$-55 \times \text{timesVisited}(\text{nxt})$: penalti kumulatif untuk tile yang sering dikunjungi. -100 jika $\text{nxt} == \text{prevLoc}$: tidak pernah balik ke tile tepat sebelumnya. -35 jika $\text{nxt} == \text{prevPrevLoc}$: momentum gerak ke depan dipertahankan. -60 jika arah berlawanan dengan lastMoveDir: tidak berbalik tiba-tiba. Praktis tidak pernah stuck di infinite loop tanpa butuh wall tracing eksplisit.
kekuatan	Paint Management 3-Level (Logistics.java)	Level 1 (CRITICAL): $\text{paint} \leq 35 \rightarrow$ langsung retreat ke tower terdekat. Level 2 (PREVENTIVE): $\text{paint} \leq 60$ DAN farFromSupply ($\text{distSq} > 30$ dari tower) $\rightarrow$ pulang sebelum benar-benar habis. Level 3 (OPPORTUNISTIC): $\text{paint} \leq 80$ DAN adjacent ke tower $\rightarrow$ isi ulang sambil lewat. Sistem 3-level ini sangat efisien: robot tidak pernah benar-benar kehabisan paint di tengah peta.
kekuatan	Mopper Transfer Paint ke Ally Aktif	tryTransferPaintToAlly: mopper secara proaktif mencari ally dengan $\text{paint} < 55\%$ kapasitas. Scoring prioritas: soldier $>$ splasher (lebih penting diisi), $\text{paint} \leq 10\%$ mendapat $+80$ score. Mopper bisa transfer hingga 70 paint per aksi, mempersingkat downtime robot lain di lapangan. followWeakAlly: jika ally $\text{paint} < 40\%$, mopper bergerak mendekati untuk transfer.

Tipe	Faktor	Analisis & Dampak
kekuatan	Tower Spawn Terarah ke Enemy Base	tryBuild memberi +15 bonus untuk spawn direction ke arah inferredEnemyBase. Robot baru langsung bergerak menuju perbatasan musuh, tidak perlu reorientasi. Efek compound: setiap robot baru memberi tekanan ke arah yang sama dari ronde pertamanya.
kekuatan	TowerBuilder, Upgrade Terintegrasi di Soldier	tryUpgradeAdjacentTower: jika soldier ada di sebelah paint/money tower ally dan chips ≥ 2600 , langsung upgrade. Tower tidak perlu menunggu gilirannya sendiri untuk upgrade, soldier bisa melakukannya. Upgrade lebih cepat terjadi karena banyak soldier yang berpotensi melakukannya.
kelemahan	inferredEnemyBase Hanya Rotasional, Tidak Adaptif	inferEnemyBase SELALU menghitung $(w-1-x, h-1-y)$, hanya simetri rotasional. Jika peta horizontal atau vertikal simetri, SEMUA robot bergerak ke pojok yang salah. Koreksi hanya terjadi saat robot benar-benar melihat enemy tower langsung (lastSeenEnemyTower). Di peta besar, bisa butuh 100-200 ronde sebelum melihat enemy tower, sangat merugikan.
kelemahan	Splasher Terlalu Konservatif di Paint	needRefillSplasher: retreat saat paint ≤ 80 dari kapasitas 300 = hanya 26% kapasitas tersisa. Splasher menghabiskan porsi besar waktunya bolak-balik ke tower untuk refill. Di game yang cepat, splasher mungkin lebih sering di perjalanan pulang-pergi daripada benar-benar menyerang.
kelemahan	Tidak Ada MapMemory Persisten	cachedRuin hanya valid 3 ronde ($\text{round} - \text{cachedRound} \leq 3$). Setiap robot re-evaluasi ruin secara mandiri setiap 3 ronde. Beberapa soldier bisa memilih ruin yang sama, keduanya berusaha mark dan paint $\rightarrow$ konflik pattern. Tidak ada mekanisme “ruin ini sudah diklaim” seperti MapMemory di Main Bot.
kelemahan	Mopper Terlalu Sering Transfer, Kurang Mop	tryTransferPaintToAlly adalah prioritas PERTAMA di runMopper. Jika ada 3 ally dengan paint rendah sekaligus, mopper terus-menerus transfer paint dan tidak pernah sempat membersihkan cat musuh. Fungsi utama mopper (membersihkan territory) terabaikan saat ada banyak ally butuh support.

Heuristik Greedy

Heuristik utama: scoreMove multi-faktor (paint%, frontier, anti-stuck, drain prediction), splashValue threshold, dan chooseTowerType kontekstual.

Tipe	Faktor	Analisis & Dampak
kekuatan	Greedy.scoreMove, Multi-Faktor Gerak	$\text{paintPct} \leq 15\%$: +30 ally, +5 empty, -40 enemy = mode bertahan ketat. $\text{paintPct} \leq 40\%$: +15 ally, +18 empty, -20 enemy = mode konservatif. $\text{paintPct} > 40\%$: -8 ally, +20 empty, +6/12 enemy = mode ekspansi agresif. frontier bonus (+3 empty, +5 enemy per tile adjacent) mendorong robot ke tepi territory. futurePaint prediction: cek dampak paint setelah gerak, penalti -90 jika akan habis.
kekuatan	Greedy.splashValue, Splash Hanya Jika Worth It	$\text{splashValue} = +4$ per tile kosong, +8 per tile cat musuh, -2 per tile ally dalam radius 4. tryBestSplashAttack hanya execute jika score > 35, tidak membuang paint untuk splash yang hasilnya kecil. Kondisi tambahan: $\text{paint} \geq 55$ sebelum berani splash, tidak splash saat hampir habis.
kekuatan	Greedy.scorePaintTarget, Kontekstual	Bonus +15 jika tile dekat ruin ($\text{distSq} \leq 8$), prioritaskan cat pattern area tower. Bonus berdasarkan jumlah tile ally di sekitar (+8 jika ≥ 2 , +12 jika ≥ 4 , +18 jika ≥ 6), cat cluster. allyN bonus mendorong flood fill dari area yang sudah dicat, bukan scatter acak.
kekuatan	TowerBuilder.chooseTowerType, Paling Kontekstual	Defense tower jika enemy tower terlihat dalam $\text{distSq} \leq 36$, reaktif terhadap ancaman. Defense tower jika ruin dekat inferred enemy base ($\text{distSq} \leq 50$), preventif di frontier. Money tower di ronde ≤ 80 , membangun ekonomi sedini mungkin. Paint tower ronde 81-600, money tower kembali setelah 600, siklus ekonomi yang dipikirkan.
kelemahan	Util.inferEnemyBase, Asumsi Tunggal	Hanya menghitung 1 kandidat (rotasional) dan tidak pernah membandingkan dengan kandidat simetri lain. Tidak ada mekanisme eliminasi simetri (seperti Main Bot yang bisa check konsistensi). pushTarget bisa salah terus selama ratusan ronde pada peta non-rotasional.
kelemahan	Logistics.farFromSupply, Definisi Kaku	farFromSupply: $\text{distSq} > 30$ dari lastSeenAllyTower. Distansi 30 $\text{distSq} = \approx 5.5$ tile Chebyshev, cukup ketat. Jika tower terakhir terlihat sudah jauh di belakang, banyak robot masuk mode preventive retreat terlalu awal.

Tipe	Faktor	Analisis & Dampak
kelemahan	G.updatePushTarget, Target Terlalu Sering Berganti	pushTurnsLeft: 22 ronde untuk frontier target, 30 untuk last seen enemy tower, 40 untuk inferred base. Setiap 22–40 ronde, pushTarget bisa berubah total → robot bisa terlihat bimbang berganti arah. Jika robot belum mencapai target, penggantian target menyebabkan jalan memutar tidak efisien.

Kondisi Map

Kondisi map yang menguntungkan dan merugikan berdasarkan karakteristik unik algoritma Alternative 2.

Tipe	Faktor	Analisis & Dampak
kekuatan	Peta Simetri Rotasional	inferredEnemyBase langsung akurat, seluruh army menuju lokasi yang tepat sejak ronde pertama. chooseUnit tower menghasilkan money tower di ronde 1–80, paint tower setelahnya, money lagi setelah 600. Dengan arah yang benar sejak awal, Alt2 mendominasi peta rotasional.
kekuatan	Peta dengan Ruin Dekat Spawn	TowerBuilder.findBestNearbyRuin aktif dari ronde pertama. Money tower bisa dibangun sebelum ronde 80, ekonomi terbangun sangat cepat. Upgrade tower (chips ≥ 2600) bisa terjadi lebih awal karena income money tower berjalan.
kekuatan	Peta dengan Cluster Enemy Paint	tryCleanEnemyPaint dengan lightFrontierBonus dan swing scoring sangat efektif di area padat. splashValue tinggi di area cat musuh yang berdekatan, splasher sangat efisien. Mopper + splasher combo memaksimalkan pembersihan area sekaligus.
kelemahan	Peta Simetri Horizontal atau Vertikal	inferredEnemyBase = $(w-1-x, h-1-y)$ akan mengarah ke pojok yang salah. Soldier, mopper, splasher semua bergerak ke area kosong sampai bisa melihat enemy tower langsung. Kerugian temporal bisa 100–300 ronde, waktu yang dipakai musuh untuk membangun keunggulan.
kelemahan	Peta dengan Banyak Obstacle di Jalur Utama	Motion.moveToward menggunakan greedy direction scoring tanpa wall tracing eksplisit. Di lorong sempit atau area obstacle padat, robot bisa terjebak di balik obstacle karena tidak ada trace logic. Berbeda dengan Alt1 yang memiliki bug-nav solid.

Tipe	Faktor	Analisis & Dampak
kelemahan	Peta dengan Ruin Tersebar Jauh dari Spawn	TowerBuilder hanya melihat nearbyTiles (dalam sense range). Ruin di luar range tidak terdeteksi, robot tidak tahu harus kemana untuk build. Bergantung pada pushTarget random frontier jika tidak ada ruin visible.

4.4.4 Hasil Pengujian

Main Bot vs Alternative 1

Aspek	Main Bot	Alternative 1
Ekonomi Chips	unggul: money+paint balance, chips income tinggi	kritis lemah: hanya paint tower, zero chips growth
Ekspansi Teritorial	unggul: splasher flood phase 2 sistematis	lemah: splasher hanya cat 1 tile per turn
Early Aggression	lemah: fokus build tower, serangan tertunda	unggul: soldier rush ke enemy base dari ronde 1
Defense Tower	unggul: defense tower selection + MapMemory	lemah: tidak ada defense tower sama sekali
Paint Management	unggul: 2-level retreat threshold	kritis lemah: tidak ada retreat → unit mati kehabisan paint
Koordinasi Build	unggul: MapMemory mencegah konflik ruin	lemah: tidak ada shared state → beberapa soldier ke ruin sama

Main Bot vs Alternative 2

Aspek	Main Bot	Alternative 2
Arah Gerak Awal	lemah: menunggu info/build sebelum menuju musuh	unggul: inferredEnemyBase dari ronde pertama
Tower Type Selection	unggul: rasio dinamis responsif terhadap jumlah tower	baik: kontekstual round & enemy proximity
Paint Management	baik: 2-level threshold	unggul: 3-level + mopper transfer aktif di lapangan
Anti-Stuck	lemah: hanya EXPLORE_TIMEOUT 30 ronde	unggul: scoring revisit -55 + prevLoc penalty

Aspek	Main Bot	Alternative 2
Simetri Inference	unggul: 3 kandidat simetri (horiz/vert/rotas)	lemah: hanya rotasional, salah total di peta lain
Koordinasi Ruin Build	unggul: MapMemory persisten, ruinClaimRound	lemah: cache 3 ronde, beberapa soldier ke ruin sama
Ekonomi Chips	baik: rasio tower dinamis	unggul: money tower ronde ≤ 80 = income sangat awal

Alternative 1 vs Alternative 2

Aspek	Alternative 1	Alternative 2
Ekonomi Chips	kritis lemah: tidak ada money tower	unggul: money tower dari ronde ≤ 80
Early Aggression	unggul: soldier rush langsung dari ronde 1	baik: push ke inferred base dari lahir
Paint Management	kritis lemah: tidak ada retreat/refill logic	unggul: 3-level threshold sangat granular
Mopper System	lemah: hanya follow+swing dasar	unggul: transfer paint + follow weak ally + swing score
Splasher Effectiveness	lemah: single tile per turn	unggul: area splash dengan value threshold
Bug Navigation	unggul: wall tracing + stuck detection solid	lemah: greedy direction tanpa trace eksplisit

5 Kesimpulan dan Saran

5.1 Kesimpulan

Melalui tugas besar ini, dilakukan perancangan dan implementasi bot permainan Battlecode menggunakan pendekatan algoritma greedy. Permasalahan dalam permainan dimodelkan ke dalam komponen-komponen algoritma greedy, yaitu himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, fungsi solusi, dan fungsi objektif. Dengan pemodelan tersebut, setiap robot dan tower dapat memilih aksi terbaik dari berbagai kemungkinan aksi yang tersedia berdasarkan prioritas strategi yang telah ditentukan.

Penulis telah merancang tiga alternatif strategi greedy dengan pendekatan heuristik yang berbeda. Alternatif pertama berfokus pada stabilitas ekonomi melalui pembangunan tower dan pemanfaatan sumber daya jangka panjang. Alternatif kedua menggunakan strategi agresif yang memprioritaskan ekspansi wilayah serta tekanan terhadap wilayah musuh sejak awal permainan. Alternatif ketiga menggunakan pendekatan berbasis state yang memungkinkan robot beradaptasi terhadap kondisi permainan seperti jumlah paint, posisi musuh, dan jarak terhadap tower sekutu.

Berdasarkan analisis efisiensi dan efektivitas setiap algoritma, beserta pengujian yang dilakukan antar bot, strategi greedy yang berfokus pada pembangunan menara menunjukkan performa yang paling stabil dan efektif dalam mencapai tujuan permainan, yaitu menguasai wilayah peta dan memenangkan pertandingan. Pembangunan menara memberikan keuntungan jangka panjang berupa peningkatan produksi sumber daya serta memperkuat pertahanan wilayah yang telah dikuasai. Keunggulan ekonomi yang dihasilkan dari strategi ini memungkinkan tim untuk memproduksi lebih banyak robot dan mempertahankan wilayah secara lebih konsisten selama permainan berlangsung. Hal ini juga sejalan dengan tujuan utama permainan Battlecode yaitu memperluas wilayah yang diwarnai pada peta.

Dengan demikian, dapat disimpulkan bahwa strategi greedy berbasis pembangunan menara merupakan strategi yang paling optimal di antara alternatif yang dirancang dalam penelitian ini. Pendekatan ini memberikan keseimbangan yang baik antara efisiensi penggunaan sumber daya, stabilitas ekonomi, serta kemampuan ekspansi wilayah. Oleh karena itu, strategi tersebut dipilih sebagai strategi utama yang diimplementasikan pada bot utama dalam tugas besar ini.

5.2 Saran

Penggunaan strategi algoritma Greedy yang berfokus pada pengambilan keputusan secara lokal untuk mencapai hasil yang maksimal secara global memiliki beberapa keterbatasan. Dalam praktiknya, pendekatan ini dapat menyebabkan penurunan performa bot ketika berhadapan dengan bot lain yang lebih adaptif dan mampu mengenali kondisi lingkungan permainan dengan lebih baik. Hal ini terjadi karena algoritma Greedy hanya mempertimbangkan kondisi yang sedang dihadapi saat itu tanpa memperhitungkan kemungkinan perubahan situasi di langkah berikutnya. Oleh karena itu, agar performa bot yang menggunakan strategi Greedy dapat lebih optimal, diperlukan pengembangan lebih lanjut dengan memanfaatkan algoritma lain yang lebih kompleks dan adaptif. Dengan menggabungkan Greedy dengan algoritma tambahan yang lebih baik, proses pengambilan keputusan oleh bot diharapkan menjadi lebih efektif sehingga dapat meningkatkan performa bot secara keseluruhan selama permainan berlangsung.



Lampiran

- Tautan Menuju Repositori GitHub Kelompok: [Repositori GitHub](#)
- Tautan Menuju Video Algoritma: [Video Algoritma](#)

References

- [1] R. Munir, “Algoritma Greedy,” Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf).
- [2] erikji, “battlecode25/src at main · erikji/battlecode25,” GitHub, 2025. Available: <https://github.com/erikji/battlecode25/tree/main/src>. [Accessed: Mar. 15, 2026].
- [3] “Direction (Battlecode 2025),” Battlecode.org, 2025. Available: <https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/Direction.html>. [Accessed: Mar. 15, 2026].